

EXECUTABLE SCIENTIFIC INTELLIGENCE

WITH DYNAMIC SOP LANG CIRCUITS



A SURVEY, RESEARCH VISION, AND PROGRAMME FOR
TRUSTWORTHY SYMBOLIC LANGUAGE SYSTEMS AND EXECUTABLE SCIENCE

Executable Scientific Intelligence with Dynamic SOP Lang Circuits

**A Survey, Research Vision, and Programme for Trustworthy Symbolic
Language Systems and Executable Science**

Axiologic Research

State of research: August 2026

Copyright © [2026] Axiologic Research

All rights reserved.

KDP publishing rights held by Outfinity SRL (Iași, Romania).

Available for free on Axiologic website (www.axiologic.net).

Author's Note

This work was created under the umbrella of Axiologic Research as part of two interconnected research programs, one dedicated to Artificial Intelligence and the other to Outfinitism, both led by Sînică Alboaie. To explore the details of how these two programs intersect and build upon each other, we invite readers to visit the Outfinity Venture Studio page at www.outfinity.ch.

While Artificial Intelligence language models were used to assist with text generation, the foundational philosophy, thematic structure, and analytical approach derive exclusively from the author's decades of dedicated study of social phenomena, social technologies, and genuine hard technologies resulting from various European and self funded research projects. Recently, within the Achilles research project, we have been deeply studying AI generated literature and its automated verification using neuro symbolic approaches; all the free books made available to the public are part of the suite of works we use to test and popularise our latest ideas and technologies.

Table of Contents

Preface - Scope, status, and method

Abstract

Part I - The Case for Executable Scientific Intelligence

Chapter 1 - Scientific Knowledge Between Narrative and Execution

- 1.1 The document remains indispensable and computationally insufficient
- 1.2 Agentic science and the problem of justified authority

Chapter 2 - Research Environments That Expose the Need

- 2.1 Scientific peer review as a layered verification problem
- 2.2 Literary analysis as theory-constrained plural reasoning
- 2.3 Specification generation from theories and document corpora
- 2.4 Symbolic language models inside agentic systems

Chapter 3 - Human-Readable Formalization and Executable Language

- 3.1 Controlled natural languages and executable specifications
- 3.2 From executable prose to a dynamic circuit intermediate representation

Part II - Foundations for Dynamic Symbolic Execution

Chapter 4 - From Graphs to Executable Circuits

- 4.1 Graph, knowledge graph, dependency graph, dataflow, workflow, and circuit
- 4.2 Lessons from compiler IRs, computational graphs, and cryptographic circuits

Chapter 5 - Symbolic, Neuro-Symbolic, and Formal Systems

- 5.1 Logic programming, rewriting, cognitive architectures, and bounded reasoning
- 5.2 Probabilistic, differentiable, solver-backed, and graph-based approaches

Chapter 6 - SOP Lang as a Circuit Intermediate Representation

- 6.1 The minimal declaration model and command-owned grammars
- 6.2 Values, references, alternatives, subcircuits, templates, and definitions
- 6.3 Knowledge as declarations, circuits, and indexed external stores

Chapter 7 - Constructing Dynamic Circuits Under Explicit Resource Budgets

- 7.1 Deterministic agenda construction and incremental activation
- 7.2 Scaling persistent symbolic models without combinatorial explosion

Chapter 8 - Provenance, Research Objects, and Trust

- 8.1 From rerunnable computation to semantically inspectable research
- 8.2 Authority, uncertainty, abstention, and governance

Part III - Experiments and a Programme of Research

Chapter 9 - Three Prototypes and What They Establish

- 9.1 SOPLangCircuits
- 9.2 dynamicCircuits
- 9.3 eSLM

Chapter 10 - A Unified Architecture for Dynamic Symbolic Systems

- 10.1 Slow acquisition and compilation, fast bounded execution
- 10.2 Stable contracts, implementation profiles, and artefact governance
- 10.3 The constructor as the central unresolved scientific problem

Chapter 11 - Experimental Programme, Falsification, and Maturity

11.1 Hypotheses, baselines, and measurements

11.2 Use-case experiments and credible research outcomes

Chapter 12 - Toward Trustworthy Executable Science

12.1 Systematic scientific search with explicit evidence boundaries

12.2 Research position in August 2026

Conclusion - A research instrument rather than an autonomous oracle

Appendix A - SOP Lang Reference Model

Appendix B - Prerequisite Encyclopedia

Appendix C - Acronyms and Abbreviations

References

Preface - Scope, Status, and Method

This book is a survey, an architectural synthesis, and a programme of research. It does not present SOP Lang dynamic circuits as a completed theory of language or as a production-ready replacement for large language models. It asks whether a persistent symbolic system can acquire reusable linguistic and reasoning competence, construct a small executable circuit for each request, and expose enough of that construction to support inspection, replay, bounded execution, and deliberate assignment of authority.

The programme is grounded in several evolving prototypes rather than in one frozen implementation. The prototypes already demonstrate executable SOP declarations, bounded symbolic methods, persistent knowledge catalogues, task-local circuit generation, coding-agent synthesis of candidate artefacts, structured failures, provenance receipts, abstention, and separation between proposal and promotion. They also disagree in useful ways about language coverage, storage, execution profiles, security boundaries, and how much knowledge should be executable. These differences are evidence of an active research process. They should not be disguised as a settled standard.

The argument is deliberately broader than SOP Lang syntax. A language cannot become a useful scientific substrate merely because it has readable declarations. The book therefore begins with the environments in which an executable symbolic layer would matter, examines controlled and executable natural languages, distinguishes several kinds of graphs and circuits, surveys symbolic and neuro-symbolic systems, and studies the lessons of compilers, cryptography, provenance, and research-object infrastructure. SOP Lang is then introduced as the particular intermediate representation around which our experiments can be unified.

The status date matters. Agentic research systems, AI-assisted peer review, AI governance, and machine-actionable scholarly communication are moving quickly. The survey reflects public material available on 18 August 2026. Claims about the three AssistOS-AI repositories describe the public repositories as consulted on that date. Claims about the research programme are architectural proposals unless a prototype or cited experiment is explicitly identified as evidence.

Research-status convention. “Implemented” means present in at least one reviewed prototype and exercised by tests or documented runs. “Supported by precedent” means that mature neighbouring technologies establish the feasibility of a mechanism, but not its success in SOP Lang. “Open” identifies a question for which neither the prototypes nor the literature currently provide adequate evidence.

Abstract

Scientific communication remains centred on prose documents even when the underlying work depends on data, software, formal models, statistical tests, simulation pipelines, and complex chains of evidence. Large language models have made those documents easier to search, summarize, critique, and transform, and the rapid development of scientific agents in 2025 and 2026 shows that models can already propose hypotheses, use tools, run experiments, analyse results, and draft manuscripts. Yet a fluent agent transcript is not a stable scientific object. It can conceal which statements came from sources, which were inferred, which were proposed, which checks actually ran, which alternatives were left unexplored, and whether the system stopped because it had adequate evidence or because its budget expired.

This book investigates dynamic SOP Lang circuits as a possible substrate for more controllable symbolic language systems. The central proposal is not that unrestricted natural language should execute directly, nor that one universal logic should represent every domain. The proposal is that language can be progressively compiled into a human-readable and machine-executable intermediate representation. In that representation, commands produce named values, dependencies are explicit, subcircuits are composable, knowledge and procedures remain versioned, and request-time execution is bounded by declared resources. Neural models may still interpret difficult language or propose new artefacts, but their outputs can be marked as candidates and subjected to deterministic parsing, testing, verification, and promotion.

The research motivation is developed through four demanding environments. Scientific peer review requires exact checks, evidence tracing, computational replay, methodological criticism, and human judgment to coexist without being conflated. Literary analysis requires a system to preserve theory dependence and defensible plurality rather than force every question into a single deductive answer. Specification generation requires prose theories, regulations, and technical documents to be transformed into stable obligations without silently inventing missing conditions. Agentic systems require recurrent LLM roles to be replaced gradually by explicit symbolic capabilities while retaining neural components where open-ended language remains necessary.

The survey places this programme in relation to controlled natural languages, natural-language-oriented programming and executable specifications, logic programming, deductive databases, rewriting systems, cognitive architectures, probabilistic logic, temporal reasoning, differentiable logic, proof assistants, SMT solvers, computational graphs, workflow systems, compiler intermediate representations, equality saturation, and cryptographic circuits. Each tradition solves a substantial part of the problem. None by itself supplies a general mechanism for selecting, composing, executing, revising, and governing heterogeneous reasoning procedures in response to ordinary language under a hard work budget.

SOP Lang is positioned as the circuit intermediate representation rather than as a complete natural-language grammar. Its stable core can remain small because individual commands own the grammar of their argument regions. Detailed syntax is important, but it is not the scientific contribution. The difficult problem is dynamic circuit construction: how a deterministic runtime can activate a relevant, small slice of a persistent symbolic model containing potentially millions of facts and thousands of procedures without scanning or instantiating them all. The book develops an agenda based on compiled descriptors, typed capability indexes, incremental activation, memoization, hierarchical subcircuits, bounded alternatives, explicit stopping conditions, and separation between fast runtime inference and slower acquisition or compilation.

Three public experimental lines provide concrete evidence and expose unresolved tensions. SOPLangCircuits studies bounded request-time selection over persistent indexed SOP knowledge. dynamicCircuits studies coding agents as compilers of documents into candidate SOP artefacts while reserving official execution to a separate deterministic runtime. eSLM studies controlled language,

declarative knowledge, bounded symbolic methods, explicit abstention, reviewable witnesses, and strict authority boundaries. A unified architecture can combine these lessons without mechanically merging the codebases.

The long-term objective is trustworthy executable science. Existing notebooks, workflow standards, research-object packages, provenance models, scholarly knowledge graphs, and nanopublications make important portions of research rerunnable or machine-actionable. Dynamic SOP circuits could add a semantic execution layer that connects claims, assumptions, theories, methods, data, evidence, counterexamples, and validation gates. The programme does not claim that empirical observation can be reduced to code or that automated review should replace scientific communities. It aims to make the formalizable and computational parts of research explicit enough to be rerun, challenged, recomposed, and audited, while representing rather than concealing the boundary where interpretation and human authority remain necessary.

Research thesis. A trustworthy symbolic language system should be judged not by how much computation it can express, but by whether it can construct the right bounded computation, preserve the status and origin of its intermediate results, and make the boundary between proposal, evidence, verification, and authority operationally enforceable.

Part I - The Case for Executable Scientific Intelligence

The opening part begins from scientific and institutional needs rather than from syntax. It asks what is missing when research remains only prose, what current agents can and cannot justify, and why a human-readable executable layer is worth investigating.

Chapter 1 - Scientific Knowledge Between Narrative and Execution

1.1 The document remains indispensable and computationally insufficient

The scientific article is one of the most successful compression technologies ever created. It can place a question, a method, a result, a comparison with prior work, an interpretation, and a set of limitations into a form that other humans can read across institutions and decades. Its flexibility is not accidental. Scientific novelty often appears before a community has agreed on a vocabulary precise enough for a database schema or a formal theory. Prose lets researchers name provisional objects, qualify claims, describe exceptional circumstances, and argue about the meaning of an observation.

The same flexibility makes the article a poor execution format. The computational meaning of a method is distributed across equations, captions, source code, supplementary files, tacit conventions, and undocumented choices. A result may depend on a dataset version that is mentioned only indirectly, on an exclusion rule buried in a paragraph, or on a software environment that no longer exists. Even when data and code are available, the relation between a reported claim and the computation that supports it is often implicit. Reproducing the code does not necessarily reproduce the argument.

Contemporary reproducibility infrastructure addresses important parts of this problem. Jupyter notebooks combine narrative, code, data, equations, and results. The Common Workflow Language describes command-line tools and their composition in a portable workflow. REANA preserves and reruns reusable analyses. Whole Tale packages code, data, computational environments, and narrative as executable research objects. RO-Crate packages research artefacts with machine-readable metadata, while W3C PROV describes the entities, activities, and agents involved in derivation. These systems make computation portable and provenance interoperable. [R32][R33][R34][R49][R50][R51]

A different line of work makes scholarly claims more machine-actionable. The Open Research Knowledge Graph represents research contributions in structured form so that studies can be compared and explored computationally. Nanopublications package small assertions with provenance and publication information. Such infrastructures challenge the assumption that the PDF must remain the only authoritative shape of scientific communication. [R52][R53]

None of these achievements should be minimized. They establish that research can be packaged, identified, cited, rerun, and linked. They also clarify what remains missing. A workflow knows that one tool consumes the output of another, but it does not automatically know that a particular table supports a causal claim only under a stated assumption. A knowledge graph can store that a paper reports an effect, but it does not automatically know which statistical test, exclusion criterion, or theory makes the effect relevant to a new question. Provenance can record that an artefact was generated by an activity without deciding whether the activity was methodologically adequate.

The gap is semantic execution. Scientific objects need a representation in which claims, evidence, assumptions, transformations, and checks can be connected by operations whose meaning is sufficiently explicit to inspect. This does not require every sentence to become formal. It requires that the parts precise enough for computation acquire executable companions and that the remaining informal parts be marked as interpretation rather than silently simulated as certainty.

A useful system would therefore support several levels of machine actionability. At the lowest level, it should locate stable source spans and artefacts. At a stronger level, it should identify claims and the evidence they cite. At a stronger level still, it should rerun or reconstruct computations, compare a method with a protocol, propagate a correction through dependent results, or instantiate an alternative theory. These levels should not be collapsed. A citation is not a proof, a rerun is not a

replication, and a formally valid derivation is not evidence that the premises accurately represent the world.

The research target can be described as an executable semantic companion to the article. The human document remains primary for explanation, novelty, judgment, and concepts not yet formalized. The companion contains machine-addressable objects and procedures: claim identities, source bindings, method descriptions, computational workflows, assumptions, tests, counterexamples, and receipts. A question about the paper activates only the relevant portion of this object. The system need not execute a universal model of the entire article each time.

This proposal changes the unit of reproducibility. Today reproducibility often means that the same script can be run in the same environment. In an executable semantic system, one can also ask which claims depend on a changed source, which conclusions survive an alternative preprocessing choice, which assumptions were never tested, and which parts of the argument remain interpretive. The desired output is not merely a repeated number but an inspectable dependency structure around the number.

Existing capability	What remains unresolved
Notebook or workflow execution	Replays a declared computation, but usually does not identify the semantic claim supported by each result or the assumptions that make the result applicable.
Research-object packaging	Preserves artefacts and environment, but does not by itself provide a language for composing claim-level reasoning and validation.
Provenance standards	Describe origin and derivation, but do not decide methodological adequacy or epistemic authority.
Scholarly knowledge graphs	Make contributions comparable and queryable, but normally do not execute the full reasoning procedure required by a new question.
Large language models	Interpret and synthesize prose flexibly, but their generated explanations and citations do not automatically constitute faithful execution traces.

1.2 Agentic science and the problem of justified authority

The state of scientific AI changed materially during 2025 and 2026. Systems described as AI scientists or co-scientists now combine literature search, hypothesis generation, coding, experiment execution, analysis, and manuscript production. In 2026, Nature published work on the AI Scientist as an end-to-end automation system for AI research, on Co-Scientist as a system for structured hypothesis generation and experimental verification, and on Robin as a multi-agent system for automated discovery in experimental biology. These results show that long research loops can be automated to a degree that would have seemed speculative only a few years earlier. [R54][R55][R56]

The central question is no longer whether an agent can perform a sequence that resembles research. It is whether the resulting sequence supports justified scientific authority. A system can

generate an interesting hypothesis, find relevant papers, run code, and produce a plausible manuscript while still failing in ways that are difficult to see from the final prose. It may select evidence because it resembles the desired conclusion, omit a negative result, conflate correlation with mechanism, misapply a method outside its domain, or stop after a convenient rather than systematic search.

Agentic systems intensify a familiar problem of large language models: the same component may propose a plan, select evidence, judge its own success, and write the explanation. A fluent final report can therefore hide a concentration of authority. The model that made an interpretive choice also narrates why the choice was reasonable. Even when tool calls are logged, the log may show only API-level events, not the semantic status of the values that moved between them.

Scientific trust cannot be reduced to internal consistency. A perfectly consistent agent may be consistently wrong because its premises are false, its measurements are biased, or its theory excludes the relevant alternative. Trustworthy automation must expose the difference between imported evidence, machine interpretation, generated hypotheses, executed tests, independent validation, and social acceptance by a scientific community.

A second difficulty is incompleteness. Open-ended research rarely permits a proof that no relevant alternative exists. A system that fails to find a counterexample under a ten-minute search has not established that no counterexample exists. The stopping condition is part of the result. The scientific record should distinguish a constructive witness, a completed finite search, a truncated search frontier, a failed tool call, and an abstention caused by inadequate formalization.

A third difficulty is evolution. Scientific knowledge changes after publication. Sources are corrected or retracted, datasets are reprocessed, methods are criticized, and theories are refined. A trustworthy system needs persistent identities for claims and dependencies so that it can determine what should be reconsidered. A conversational memory summary is inadequate because it can overwrite the path by which an earlier conclusion was reached.

These requirements suggest an architecture in which the scientific agent is not one autonomous model but a governed composition of capabilities. Open-ended models may generate candidates, but candidates do not become facts merely because they are fluent. Deterministic or formally checked components can verify some subproblems, but their authority is limited to the formalization they received. Human reviewers and scientific institutions remain authorities for judgments that depend on novelty, significance, ethics, and domain interpretation.

The contribution proposed in this book is infrastructural. A dynamic circuit system can give each transformation an identity, make dependencies inspectable, preserve alternatives, bind execution to resource budgets, and produce receipts. SOP Lang is investigated as the human-readable intermediate representation in which these responsibilities can be composed. The language is not intended to legislate scientific truth. It is intended to make the process by which a machine claims to have reached a result more explicit and contestable.

This position is intentionally less dramatic than the promise of fully autonomous science. It may also be more useful. The near-term opportunity is to build scientific systems whose automated operations can be replayed, whose limitations are visible, whose proposals are separated from authority, and whose recurrent successful procedures can be compiled into persistent symbolic competence. A system with these properties can accelerate science without requiring society to trust an opaque artificial scientist as a single epistemic actor.

Authority distinction

Source discovery

Required treatment

Produces candidate sources and search receipts; relevance is not equivalent to support.

Authority distinction	Required treatment
Claim extraction	Produces propositions bound to source spans; extraction confidence is not truth.
Hypothesis generation	Produces candidates whose novelty and plausibility must be evaluated independently.
Computational execution	Produces results relative to code, data, environment, and parameters; it does not validate empirical grounding by itself.
Formal verification	Establishes a property of a formal statement; it does not establish that the statement faithfully captures the scientific question.

Chapter 2 - Research Environments That Expose the Need

2.1 Scientific peer review as a layered verification problem

Scientific peer review is an appropriate first high-value environment because it contains both tasks that can be operationalized and judgments that resist full formalization. A paper presents claims, evidence, methods, comparisons, and interpretations. Some review questions are exact: whether totals are internally consistent, whether a reported test can be recomputed, whether a cited source actually contains the attributed result, whether a required artefact exists, or whether a conclusion contradicts a definition elsewhere in the manuscript. Other questions concern novelty, significance, explanatory adequacy, or whether an experimental design is persuasive. These judgments depend on disciplinary knowledge and legitimate disagreement.

The year 2026 made this distinction urgent. The AAAI-26 AI Review Pilot generated a clearly labelled AI review for every one of 22,977 papers that entered full review, while reporting that naive prompting of off-the-shelf LLMs does not yield high-quality reviews. NeurIPS 2026 launched an experiment in which an AI assistant is intended to help reviewers understand submissions and perform background analysis without replacing reviewer judgment. These efforts move automated review from laboratory benchmarks into live institutional settings. [R57][R58]

The appropriate goal is not an artificial reviewer that emits one authoritative score. A stronger design decomposes review into inspectable functions. The system can identify claims, locate the evidence each claim invokes, test internal consistency, inspect computational artefacts, compare methods with reporting standards, search for relevant prior results, and construct counterarguments. Each result can state whether it was mechanically verified, supported by sources, inferred under a theory, or left to human judgment.

This decomposition also improves security. Papers can contain adversarial text, hidden instructions, misleading citations, or strategically selected evidence. An LLM that reads the submission and produces the final review in one context is exposed to prompt injection and role confusion. A circuit-based architecture can treat the manuscript as untrusted data, restrict which components can interpret it as instructions, and separate the ingestion of claims from the authority to accept a review conclusion.

A machine-assisted review should first produce a structured review object. Each issue in that object identifies a target claim or artefact, the evidence examined, the check performed, the execution status, the limitation of the check, and the scope of any recommendation. Human-readable prose is then generated from the object. This ordering reduces the risk that criticism is invented during rhetorical generation and permits different views for authors, reviewers, editors, and artifact evaluators.

The review process can become incremental. When authors revise a paper, only checks whose inputs changed need to run again. If a dataset is corrected, dependent statistical claims can receive new versions. If a cited source is retracted, support relations can be reconsidered without erasing the historical record. Review becomes a maintained and versioned set of procedures rather than a one-time document.

A serious benchmark should contain controlled faults as well as naturally occurring papers. Researchers can create variants with citation mismatches, swapped labels, inconsistent sample counts, missing baselines, invalid statistical interpretations, unsupported causal language, contradictory definitions, incomplete artefacts, and ambiguous claims. Because the mutation is known, evaluation can distinguish detection from explanation and from final editorial judgment.

The limits must remain explicit. A system cannot determine from code alone whether a biological sample was handled correctly, whether an archaeological observation was recorded honestly, or

whether a theoretical contribution is genuinely profound. Automated review can strengthen the factual and computational substrate on which reviewers deliberate. It should not erase the social, ethical, and interpretive character of scientific evaluation.

Review layer	Appropriate machine role
Document and artefact integrity	Check availability, identifiers, versions, internal references, schemas, and reproducibility prerequisites.
Claim-evidence alignment	Bind claims to manuscript spans, cited sources, tables, figures, datasets, and computations.
Method and computation	Recompute feasible results, compare declared procedures with execution, and report deviations.
Literature context	Retrieve candidate prior work and expose search scope, rather than declaring novelty from a hidden search.
Interpretive criticism	Generate theory- and evidence-bound candidate critiques whose assumptions remain visible.
Editorial authority	Remain human or institutionally governed except for narrowly defined mechanical gates.

2.2 Literary analysis as theory-constrained plural reasoning

Literary analysis is not a secondary demonstration chosen for convenience. It is a demanding test of whether a symbolic system can represent interpretation without pretending that every well-formed question has one correct deductive answer. A novel can be analysed through narratology, rhetoric, historical context, psychoanalytic theory, formalism, postcolonial theory, or a custom framework defined by the researcher. Different readings may be defensible because they select different evidence and operate under different theoretical commitments.

A conventional benchmark built around one gold answer is poorly suited to this domain. The system should instead be evaluated on whether it grounds observations in the text, applies a declared theory coherently, distinguishes observation from interpretation, preserves incompatible but defensible branches, and refrains from upgrading an interpretation into proof.

The domain is useful precisely because it exposes the limitations of superficial explainability. A model can cite a passage and still misrepresent it. It can mention a theory while using only generic language associated with that theory. A trustworthy analysis requires a traceable relation among the text, identified phenomena, the theoretical operations applied to them, and the resulting argument. The relation can be inspected even when experts disagree about the conclusion.

Theory packages can provide reviewed concepts, relations, interpretive procedures, and evidential expectations. Some operations may be deterministic, such as locating repeated phrases or shifts in named speakers. Others may be heuristic, such as identifying irony or unreliable narration. The package should state this difference. A symbolic label does not turn a heuristic judgment into an objective fact.

Plurality can be made productive. Competing analyses can share low-level textual observations while diverging at a theory-specific transformation. A comparison can reveal where they agree, where they invoke different evidence, and which claims would disappear if a theoretical assumption were removed. Such comparisons are more informative than asking several language models to vote on an interpretation.

Literary analysis also stresses long-range language understanding. Relevant evidence can be separated by hundreds of pages, and the significance of an image may depend on its recurrence, transformation, or relation to the narrative voice. A successful system must retrieve selectively without losing the textual sequence and must represent uncertainty when references or motifs are ambiguous.

The transferable value is substantial. Science also contains theory-dependent observation, contested interpretation, and underdetermination. A system that can preserve plural literary analyses while enforcing source grounding may be better prepared to represent competing scientific explanations without collapsing them into one synthetic answer. Literary analysis therefore tests epistemic discipline under ambiguity, not merely stylistic generation.

2.3 Specification generation from theories and document corpora

Organizations repeatedly convert prose into executable obligations. Research protocols become validation rules. Regulations become compliance checks. Architectural decisions become invariants. Scientific theories become simulation requirements or admissible transformations. Project documentation becomes tests and monitoring conditions. The conversion is expensive because domain meaning and implementation meaning do not align automatically.

Large language models can draft specifications quickly, but the central risk is not poor grammar. It is semantic drift. A generated specification may omit an exception, strengthen a recommendation into an obligation, merge requirements with different scopes, invent a missing threshold, or silently choose one interpretation of an ambiguous term. The output can look more precise than its source while being less faithful.

A trustworthy process keeps the original theory or document set canonical. Candidate requirements receive identities linked to source spans and versions. Their normative strength, actors, preconditions, obligations, prohibitions, exceptions, temporal scope, measurable outcomes, and unresolved terms are represented explicitly. When the source is vague, the specification should preserve the gap as a clarification obligation rather than fill it with plausible detail.

The resulting object is more than a polished requirements document. It can generate downstream tests, proof obligations, static-analysis rules, monitoring queries, or implementation tasks. These executable artefacts remain derived from the accepted requirement and cannot redefine it. If implementation experience reveals inconsistency or infeasibility, the evidence returns to a new requirement version.

This environment provides a direct measure of symbolic learning. A circuit or template learned from one set of documents is valuable only if it transfers to unseen documents without silently changing meaning. Evaluation can measure source coverage, unsupported additions, missed exceptions, contradiction rates, executable-test yield, correction effort, transfer across paraphrases, and localization of regressions after source edits.

The same approach applies to scientific theories. A theory can be partially compiled into concepts, relations, constraints, and procedures. New papers can then be checked for consistency with the formalized portion, and unresolved consequences can generate candidate experiments. Partial formalization is expected. The research question is whether the executable portion remains faithful and useful, not whether every theoretical insight can be reduced to code.

Specification generation is thus the bridge between document interpretation and persistent symbolic competence. Successful conversions can be promoted into reusable artefacts. Over time, recurring document structures and domain procedures become part of a domain-specific symbolic language model rather than being rediscovered through prompts for every task.

2.4 Symbolic language models inside agentic systems

Contemporary agents often use a large language model as planner, reasoner, tool selector, memory interpreter, error judge, and final writer. This concentration of roles is convenient because one interface can handle many tasks. It is also difficult to audit because the same component decides what the request means, what action to take, whether the action succeeded, and how to explain the outcome.

The research programme proposes an incremental migration rather than a wholesale replacement. Existing agents can be instrumented to classify model calls by function. Recurrent calls used for retrieval, exact transformation, policy checking, solver orchestration, or validation can be compared with explicit symbolic alternatives. Once a circuit is validated for a functional role, the surrounding agent can invoke it through a stable contract.

Replacement is therefore functional, not token-level. A symbolic component need not generate the same words as a neural model. It replaces a role when the agent can submit the same class of tasks and receive semantically useful outputs with stronger guarantees or lower recurring cost. Exact policy checking may become fully symbolic. Open-ended interpretation or rhetorical realization may remain neural.

This framing avoids an unproductive opposition between symbolic and neural AI. Neural models are valuable for language interpretation, candidate synthesis, analogy, and communication. The question is where they should have authority. A model may propose a task interpretation or a new command implementation, while deterministic validators decide whether the proposal satisfies a reviewed interface and test suite.

The migration can be measured. A domain-specific system can record what fraction of successful tasks depend on runtime LLM calls, what roles those calls serve, which calls produce unverified assertions, and how performance changes as reviewed circuits accumulate. A mature symbolic language model may still contain neural components, but its stable domain competence is increasingly represented in inspectable artefacts.

Different domains can share the same runtime and constructor while loading different knowledge, theories, capabilities, and policies. A scientific-review model, a regulatory model, and a literary-analysis model need not pretend to possess identical notions of evidence. Generality can reside in common execution mechanisms and in transferable subcircuits rather than in one universal knowledge base.

The use case is successful when symbolic execution improves the surrounding agent's controllability without destroying its linguistic usefulness. The relevant comparison is not an immature prototype against a frontier model asked to do everything. It is a controlled replacement study in which one recurring role is migrated at a time and evaluated for accuracy, provenance, latency, cost, robustness, and human correction effort.

Chapter 3 - Human-Readable Formalization and Executable Language

3.1 Controlled natural languages and executable specifications

The aspiration to make natural-language-like text executable has a long history. It emerges from a genuine tension. Ordinary language is accessible but ambiguous. Formal languages are precise but often inaccessible to domain experts. Controlled natural languages attempt to occupy the space between them by restricting vocabulary, syntax, or semantics while retaining enough natural form to remain readable. The classical survey literature emphasizes that CNLs are engineered languages, not naturally evolved sublanguages, and that they vary widely in purpose and degree of formality. [R40]

Attempto Controlled English is a foundational example. ACE constrains English so that sentences can be translated into discourse representation structures and Prolog. The resulting knowledge base can be queried and executed for simulation, prototyping, and validation. The importance of ACE is not that it solved unrestricted English. It demonstrated that a human-readable textual view can have a precise mapping to formal execution. [R10][R41]

Common Logic Controlled English and related languages pursue English-like access to first-order or Common Logic representations. The Object Management Group's SBVR standard takes a business-oriented route by separating vocabularies, facts, and rules and allowing structured natural-language notations to express machine-processable semantics. These approaches show that formalization becomes more usable when domain concepts and rules are exposed in terms recognizable to stakeholders. [R42][R43]

Grammatical Framework addresses a complementary problem. GF is a programming language and formalism for natural-language grammars with typed abstract syntax and reversible mappings to concrete languages. A shared abstract representation can be parsed from and linearized into multiple languages. This is particularly relevant when the executable representation should not be permanently tied to English surface forms. [R44]

Recent work connects controlled language to modern declarative solvers. CNL2ASP translates a controlled English specification into Answer Set Programming, including disjunction, choice rules, aggregates, and optimization constructs. NL2ASP adds a neural stage that translates freer natural language into the controlled form before deterministic conversion to ASP. These systems provide a concrete precedent for a hybrid pipeline in which a neural model proposes a normalized representation and a formal compiler owns the executable semantics. [R45][R46]

Other traditions use natural-language-like text as a programming or testing interface. Inform 7 is a natural-language-based programming language for interactive fiction. Gherkin gives plain-text scenarios enough structure for Cucumber to treat them as executable specifications connected to step implementations. Their success shows that human-readable executable text can be practical when the domain, grammar, and operational binding are carefully constrained. [R47][R48]

The approaches differ in where they place semantic authority. ACE aims for a controlled language with a formal interpretation. GF separates abstract and concrete syntax. Gherkin structures examples whose operational meaning is supplied by step definitions. Inform 7 is a full domain-specific programming language with an English-like surface. SBVR represents domain vocabularies and declarative business rules. CNL2ASP lowers a controlled specification to a particular reasoning formalism.

These distinctions matter for SOP Lang. It would be misleading to describe SOP Lang simply as another controlled English. Its declarations are readable and commands may adopt natural-language-like argument grammars, but the stable language does not attempt to define the full semantics of every phrase. It is closer to an extensible executable intermediate representation whose

commands own domain-specific microgrammars. A natural-language interface may compile into SOP Lang, and a controlled language may be implemented on top of it, without requiring the SOP kernel itself to become a universal grammar of English.

Approach	Primary contribution and limitation relative to the programme
Attempto Controlled English	Demonstrates a readable language with formal translation and execution; it is tied to a deliberately controlled English fragment and a particular semantic route.
Common Logic Controlled English	Offers English-like access to expressive logic; it does not by itself orchestrate heterogeneous runtime methods under budgets.
SBVR Structured English	Separates vocabulary and declarative business rules for stakeholder-readable formalization; operational execution usually requires additional mappings.
Grammatical Framework	Provides typed abstract syntax and reversible multilingual grammars; it is a language-construction framework rather than a general reasoning runtime.
CNL2ASP and NL2ASP	Show a practical neural-to-controlled-to-formal compilation pipeline; the executable target remains Answer Set Programming.
Gherkin and Cucumber	Connect structured plain text to executable tests through step definitions; the semantics reside substantially in the implementation bindings.
Inform 7	Shows that natural-language-oriented programming can support a rich domain; its success depends on a constrained semantic world and specialized compiler.

3.2 From executable prose to a dynamic circuit intermediate representation

The phrase executable natural language can describe several substantially different ambitions. It may mean a controlled language whose sentences map directly to logic. It may mean a natural-language-like programming language. It may mean a requirements notation whose scenarios invoke code. It may also mean a pipeline in which unrestricted language is interpreted into an executable representation and the interpretation remains visible for confirmation.

The SOP Lang programme adopts the last and most modular interpretation. Ordinary language is not assumed to be executable by itself. A language interface constructs semantic candidates, a controlled language or domain parser may normalize them, and SOP Lang represents the executable

circuit selected for the task. The circuit can invoke logic engines, databases, statistical procedures, neural models, proof assistants, or external tools. The representation is unified at the level of values, dependencies, contracts, and receipts rather than at the level of one universal logic.

Human readability is a design constraint, not a proof of naturalness. SOP declarations should be legible enough that researchers and engineers can inspect what was executed. Commands can use terms from the domain and can parse multi-line argument regions that resemble small declarative languages. Yet the system must not rely on a reader's intuition where formal behaviour matters. Each command implementation or profile needs a documented parser and contract.

This separation prevents the core language from growing into an unstable accumulation of domain keywords. A statistical command may understand terms for confidence intervals and baselines. A regulatory command may understand temporal scope and exceptions. A literary command may understand theory and textual evidence. These concepts are important, but they do not all belong in the tokenizer of the common language.

The unifying function of SOP Lang is therefore analogous to a compiler intermediate representation. Front ends may include controlled languages, parser combinators, neural semantic parsers, document compilers, or direct human authoring. Back ends may include Prolog, Soufflé, Z3, Lean, probabilistic logic, temporal engines, databases, cryptographic proof systems, or ordinary code. The IR preserves a common account of which operations produced which values and how the active circuit was constructed.

The word circuit adds an important requirement beyond readable orchestration. The active programme must be recoverable as a dependency structure whose values have stable identities. This structure enables topological scheduling, incremental recomputation, caching, provenance, validation, and alternative branches. A generic workflow can imitate these features, but they become research-relevant only when treated as language semantics and tested across implementations.

Dynamic construction distinguishes the programme from most executable-specification systems. The entire procedure need not be known before execution begins. A template may become applicable when new values appear. A failed method may activate a diagnostic circuit. A counterexample may force reinterpretation. The system therefore executes a sequence of bounded circuit versions rather than one immutable workflow.

The promise is substantial, but so is the uncertainty. It is not known whether useful language competence can be represented by a manageable library of reusable circuits, whether a deterministic constructor can select them quickly at scale, or whether the cost of maintaining explicit semantics will outweigh the benefits outside narrow domains. The prototypes make these questions experimentally accessible. They do not answer them in advance.

Positioning. SOP Lang is best understood as the executable, human-inspectable intermediate form beneath natural-language interfaces, not as a claim that unrestricted prose has acquired a single formal semantics.

Part II - Foundations for Dynamic Symbolic Execution

The second part separates the classical concepts that are often conflated, surveys the systems from which the programme can borrow, and then presents SOP Lang and the dynamic constructor at the appropriate technical level.

Chapter 4 - From Graphs to Executable Circuits

4.1 Graph, knowledge graph, dependency graph, dataflow, workflow, and circuit

The word graph identifies a mathematical shape, not a computational meaning. A graph contains vertices and edges, possibly directed, typed, labelled, weighted, or higher-arity. The same shape can describe social relations, roads, causal hypotheses, data dependencies, or executable operations. Calling a structure a graph says almost nothing about what an edge authorizes a machine to do.

A knowledge graph is primarily a representation of entities, concepts, and relations. An edge may state that a city is the capital of a country or that a paper cites another paper. The graph can be queried and reasoned over, but its relational edges are normally statements about a domain rather than instructions that one value must be computed before another.

A dependency graph represents reliance. An object may depend on a source, a software package, an assumption, or a previous result. Dependency graphs are important for invalidation because a change can be propagated to the objects that rely on it. They still need not describe an executable operation.

A dataflow graph assigns operations to nodes and values to edges or named connections. An operation becomes eligible when its required inputs are available. This model supports parallelism, incremental evaluation, and explicit def-use relations. Computational graphs in machine learning are dataflow graphs specialized for numerical operations and differentiation. They make the forward computation and the derivatives needed for training explicit.

A control-flow graph represents possible paths of execution rather than the values that flow between operations. A branch, loop, or exception changes which block executes next. Real programmes often require both control and data dependencies. SSA-based compiler IRs make data dependencies explicit while retaining control-flow structure.

A workflow graph coordinates tasks and services. It is an important engineering abstraction, but the values between tasks may be opaque files or JSON objects, and nodes may mutate external state. The graph tells an orchestrator what to run without necessarily providing a formal account of the semantic status of each intermediate result.

A circuit, as used in this programme, is an executable dependency structure with stable values and composable operations. The term emphasizes that the graph is not merely descriptive: it can be scheduled, run, inspected, rewritten, cached, and validated. The concept remains broader than an electronic or cryptographic circuit because nodes may invoke complex algorithms or external engines.

The distinction is not intended to introduce terminology for its own sake. It establishes falsifiable obligations. If an implementation calls arbitrary stateful tools and reconstructs provenance only after the fact, it is a workflow even if its documentation calls it a semantic circuit. If values have stable producers, dependencies are explicit, effects are declared, and subcircuits compose through contracts, then compiler and formal-method techniques can be applied meaningfully.

The same persistent system may contain several graph projections. A knowledge graph represents domain relations. A circuit graph represents active computation. A provenance graph records derivation. A plan graph records chosen procedures. A cache graph identifies invalidation dependencies. The projections can share identities without being collapsed into one undifferentiated graph.

Structure	Operational meaning
Graph	A mathematical carrier whose edge meaning is supplied by an application.
Knowledge graph	Represents entities, concepts, and domain relations; edges are primarily statements about the represented world.
Dependency graph	Represents that one object relies on another and supports impact analysis or invalidation.
Dataflow graph	Represents operations and the values they produce or consume; dependencies constrain execution.
Control-flow graph	Represents possible execution paths and branch or loop structure.
Computational graph	A dataflow graph specialized for executable mathematical operations, often supporting differentiation.
Workflow graph	Coordinates tasks and external effects; semantic typing and faithful provenance may be application conventions.
Dynamic semantic circuit	An executable, inspectable, versioned dependency structure whose values and operations carry semantic contracts and can be extended during bounded reasoning.

4.2 Lessons from compiler IRs, computational graphs, and cryptographic circuits

Compiler theory provides the strongest technical analogy for SOP Lang because compilers have spent decades solving the problem of transforming complex programmes while preserving explicit semantics. LLVM IR uses SSA-based values and type safety as a common representation across compilation stages. [R1] MLIR extends the idea by allowing multiple dialects and abstraction levels to coexist, with transformation and dialect-conversion infrastructure for progressively lowering high-level operations toward more concrete forms. [R2] These systems show that the choice of intermediate representation determines which analyses and optimizations become cheap enough to use routinely.

SOP Lang’s `@var command ... declaration` can be viewed as a high-level three-address-like operation with arbitrarily rich command semantics. `$var` corresponds naturally to an SSA operand. A result variable can be consumed by multiple later commands without implying mutation. The analogy becomes particularly useful for provenance and incremental recomputation: if a source variable changes, the runtime can invalidate descendants in the dependency graph rather than rerunning unrelated operations.

Control flow is more subtle. SOP Lang is primarily dataflow-oriented. Conditional execution can be represented by commands whose outputs encode alternatives, guards, or activation states. An implementation may also provide specialized control commands. The important point is to avoid

hiding mutable control state outside the graph. If two branches create competing semantic interpretations, both should remain explicit variables until a later selection operation resolves them.

MLIR's dialect concept offers a route for command ecosystems. A medical SOP profile may register commands and types that are meaningless in a compiler or legal domain. A proof profile may understand theorem objects and proof obligations. The global runtime only needs the common contract surface. This supports a multi-level approach in which a high-level command such as `check_contraindication` can later be lowered into retrieval, temporal checks, probabilistic evaluation, and SMT constraints depending on available backends.

Lowering is particularly important for language reasoning because the appropriate level of formalization may not be known initially. A sentence can first become a generic `Claim` or `Event`. A domain circuit may specialize it into a pharmacological interaction or legal obligation. Only then may a solver-specific lowering produce a Datalog relation or SMT formula. If the solver finds the formalization unsatisfactory, the system may need to return to the higher level and generate an alternative interpretation.

Compiler optimizations also translate into research hypotheses. Dead-code elimination becomes elimination of circuit branches that cannot contribute to the required outputs or audit artefacts. Common-subexpression elimination becomes reuse of identical semantic transformations over the same versioned inputs. Constant propagation becomes specialization of templates when contextual properties are already known. Partial evaluation becomes precomputing fixed portions of a reusable circuit before binding task-specific variables.

The analogy must preserve observability. A compiler can eliminate a computation if it has no effect on programme semantics. In a reasoning circuit, an intermediate value may be needed only for audit. The optimization policy must therefore treat provenance, explanations, and required traces as observable outputs. A branch that does not affect the final answer can still be essential if it constitutes evidence that a contradiction was checked.

E-graphs and equality saturation address a different optimization problem: how to represent many equivalent programmes without choosing one too early. The `egg` library demonstrates a practical implementation of equality saturation, while `egglog` combines Datalog-like fixed-point reasoning with equality saturation. [R16][R17] SOP Lang can use these ideas at two levels. A compiler for SOP circuits can canonicalize equivalent command compositions, and the construction runtime can maintain compact equivalence classes for alternative plans whose semantics are known to be the same.

The limitations are important. Natural-language interpretations are often not provably equivalent. Two circuits may simply be competing hypotheses. They must not be merged merely because their outputs look similar on a small sample. Equality saturation is therefore most appropriate after the circuit has entered a formalized subdomain where rewrite equivalences are trusted.

Strategy-guided equality saturation is particularly relevant to the combinatorial problem. Full equality saturation can exhaust memory as rewrite spaces grow, which has motivated guided and synthesized strategies. `EggMind` explicitly represents strategies and uses tractability guidance to avoid e-graph explosion. [R22] The parallel with SOP is strong: a large library of circuit templates creates a rewrite and composition space that must be guided. A SOP strategy artefact can play the same role as an `EqSat` strategy, limiting which templates and rewrites become eligible under a given goal and budget.

Compiler implementation practice also suggests a separation between source IR and runtime caches. SOP source should remain readable and versionable, while a build step creates indexes, type summaries, dependency signatures, cost profiles, and binary representations. The runtime

should load these summaries first and only materialize full circuit bodies when selected. This is the same broad principle by which compilers avoid reparsing or reanalyzing unnecessary components.

The deepest lesson from compiler theory is that representation and optimization co-evolve. A language can be Turing-complete yet unusable for optimization if it hides dependencies. SOP Lang should be judged by the analyses it makes cheap. If \$ and ~ allow the runtime to construct precise dependency and provenance graphs, then time-bounded construction, incremental recomputation, caching, and parallelism become plausible. If command effects remain opaque, compiler analogies will remain superficial.

Compiler concept	SOP Lang analogue
SSA value	A @var result with one semantic producer in a circuit version.
SSA operand	A \$var dependency that supplies a produced value to another command.
IR operation	A command instance together with its ordered argument stream and declared contract.
Dialect	A command/type family owned by a domain or backend profile without expanding the universal SOP grammar.
Lowering	Transforming a high-level semantic command or value into more concrete commands suitable for Datalog, SMT, proof checking, or another backend.
Optimization pass	A circuit rewrite that preserves required semantics and observability while reducing cost.
E-graph	A compact structure for equivalent circuit expressions where equivalence is formally justified.
Strategy DSL	A meta-level artefact that limits and orders transformations to prevent search-space explosion.

Compiler theory offers more than an analogy to SSA. A modern compiler separates parsing, semantic analysis, intermediate representations, optimization, lowering, code generation, and runtime execution because no single representation is ideal for all phases. A high-level AST preserves source structure but is inconvenient for low-level dataflow analysis. A low-level machine IR exposes registers and control flow but has already discarded domain concepts. The success of LLVM and MLIR rests on choosing intermediate structures whose invariants make transformations tractable. SOP Lang faces the same representation problem, except that its source may be natural language and its backends may have incompatible logics rather than incompatible instruction sets.

The \$ relation gives SOP Lang an immediate def-use graph. Once a runtime resolves \$x to the declaration that produces x, every consumer can be indexed without interpreting the semantics of the command body. This enables topological scheduling for pure acyclic fragments, dead-result analysis, cache invalidation, and parallel execution. ~x is not a normal SSA operand because it refers to the variable object rather than only its value. It is closer to an explicit handle into compiler metadata or IR operation state. This extension is deliberate: reasoning systems need to inspect derivations and alternatives as data.

Classical SSA also depends on control-flow concepts such as dominance and phi placement. SOP Lang should not import these mechanically. Many SOP circuits are naturally acyclic dataflow graphs, and loops may be encapsulated within commands or expressed through epoch growth and fixed-point templates. If an implementation later introduces explicit branch regions or loop regions, compiler SSA algorithms may become directly relevant. Until then, claiming full SSA semantics would create confusion. The defensible statement is that SOP Lang uses SSA-like single-producer values as a foundational invariant and borrows additional SSA machinery only where the runtime representation actually contains the corresponding control structure.

MLIR provides an especially strong precedent for command-owned semantics. MLIR has a common infrastructure but operations belong to dialects that define types, attributes, verification rules, and transformations. [R2] SOP commands play a comparable role. The core parser need not know what theory means to a z3 command or what relation means to a KB command. A domain package can define descriptors, validation, lowering, and optimizations for its commands. This suggests that the path to a standardized SOP ecosystem is not a large universal grammar but a registry of analyzable command families with explicit contracts.

Lowering is central to this vision. A high-level command can initially express an operation in terms meaningful to the problem domain, such as `check_policy_consistency`. A lowering pass can replace that command with a subcircuit that extracts constraints, normalizes units, invokes Datalog for relation closure, sends arithmetic constraints to Z3, and reconciles results. The high-level command remains useful for planning; the lower-level graph is useful for execution and explanation. Different implementations may lower the same high-level operation differently depending on available backends and guarantees.

This transformation illustrates why circuit versioning matters. The source-level `@result` and the lowered `@result1` should not be silently conflated. The runtime can record that `result1` implements or refines the semantic obligation represented by `result`. If a bug occurs, the trace can distinguish incorrect high-level planning from incorrect lowering. This is exactly the kind of layered diagnostic infrastructure that mature compilers provide and that current agentic tool workflows usually lack.

E-graphs and equality saturation contribute a complementary idea: defer commitment among expressions that are known to be equivalent. [R16] Egglog demonstrates that this can be combined with Datalog-like relational reasoning. [R17] For SOP Lang this is valuable for optimization of formally equivalent circuit fragments, canonicalization of learned circuits, and backend-specific lowering. It is dangerous for semantic ambiguity. Two interpretations of a sentence are not an e-class merely because both are plausible; they are alternatives until an equivalence relation has been justified. E-graph compaction should therefore operate only under trusted rewrite rules or empirically validated equivalence scopes.

Compiler property	Research question for SOP Lang
Def-use chains	Can \$ dependencies provide enough static structure to make incremental recomputation and provenance cheap at runtime scale?
Purity and effects	Which commands can be safely reordered, cached, or parallelized, and how should external effects be declared?
Multi-level IR	Which semantic objects should remain high level instead of being prematurely lowered into logic or backend-specific syntax?
Dialect verification	How much contract information can command families expose without bloating the universal SOP syntax?
Lowering correctness	How can the system verify that a backend-specific circuit preserves the intended high-level semantic obligation?

Compiler property	Research question for SOP Lang
Optimization	Which rewrites preserve not only final values but also required provenance, epistemic status, and observability?
E-graph equivalence	Where can equivalence be proven strongly enough to compact alternatives without erasing genuine semantic uncertainty?
Incremental compilation	Can changes to KB shards, commands, and templates invalidate only the affected descriptors and caches rather than rebuilding the entire symbolic model?

The broader lesson is that an IR becomes useful only when its invariants support algorithms. The fact that SOP Lang can express arbitrary computation through define is not itself an advantage. The research value comes from identifying a restricted, observable profile in which dependency extraction, effect analysis, template indexing, rewriting, and caching are sufficiently reliable to support large-scale reasoning. If most practical commands defeat static descriptors and hide relevant state, the compiler analogy fails and SOP Lang collapses back into an orchestration notation.

Cryptography provides a second, more distant but still valuable circuit tradition. Secure computation and zero-knowledge systems make dependencies explicit because the security proof depends on the exact computation being performed. ABY, for example, combines arithmetic sharing, boolean sharing, and Yao garbled circuits and provides conversions so that different parts of one computation can use different secure protocols. [R19] The lesson for SOP Lang is architectural rather than cryptographic: a single high-level computation can be partitioned across heterogeneous execution substrates while a common circuit records how outputs depend on inputs.

This resembles the role of SOP commands as backends. A relational closure may be efficient in Souffle, a numerical constraint in Z3, and an uncertain relation in ProbLog. The outer SOP circuit need not force one mechanism to handle all tasks. Instead it carries values across backend boundaries through explicit conversion commands. Such conversions should be visible because they can lose information or weaken guarantees.

Arithmetic circuits used in zero-knowledge proof systems make an even stronger point about explicit constraints. Circom allows programmers to define arithmetic circuits whose constraints have a restricted quadratic form, and the generated constraint system determines what the proof actually attests. [R20] Halo2 uses a PLONKish arithmetization with gates, columns, cells, and lookup mechanisms. [R21] In both cases, a programme is not trusted merely because it computes the expected answer on examples. The constraint system must capture the intended relation between inputs, internal witness values, and outputs.

A SOP circuit is not a zero-knowledge circuit. Its commands can be arbitrarily high level, dynamic, stateful, or externally implemented. Nevertheless, the concept of a witness is useful for reasoning about explanation. A final answer can be accompanied by the set of semantic variables, source references, solver outputs, and command applications that justify it. This is a semantic witness in an informal architectural sense, not a cryptographic witness. The distinction must be explicit to avoid overstating guarantees.

The cryptographic literature also provides a warning. A circuit can be under-constrained: it may accept witnesses that the designer did not intend. The analogous error in symbolic language reasoning occurs when an answer merely contains a citation but the citation does not support the

claim, or when a validator checks syntax without checking semantic correspondence. Explicit graphs do not create correctness automatically. The constraints and validators must encode the intended property.

SOP Lang can make such obligations first-class. A command can produce `@candidateAnswer`. Another command can consume `~candidateAnswer` and verify that every claim has an evidence path. A third can check consistency of the evidence. The final output command can refuse to expose the answer if the validation variables do not satisfy policy. The validation circuit becomes part of the programme rather than an informal post-processing step.

Some stable SOP subcircuits may eventually be compiled into cryptographic circuits. Consider a deterministic eligibility rule over structured private data. Once the language-understanding stage has produced verified structured variables, the policy-checking subcircuit could be lowered into an arithmetic constraint system and executed or proved without revealing all private inputs. This path is realistic only for restricted deterministic fragments. Dynamic retrieval, LLM calls, and arbitrary code-defined commands are not naturally arithmetizable.

Even when no zero-knowledge proof is generated, cryptographic thinking improves interface design. A command should specify what it assumes, what it guarantees, what data it reveals, and what information its output logically implies. Security labels can travel with SOP variables, and the scheduler can reject a backend that would receive data outside its authorization domain. A `~var` reference may require stronger permission than `$var` because lineage can reveal sensitive sources.

Mixed-protocol secure computation also suggests cost-aware partitioning. The cheapest backend depends on the operation. A symbolic language model should similarly avoid universal execution. The circuit constructor can maintain cost profiles for commands and conversions, then choose a composition that satisfies both semantic and resource constraints. The comparison therefore reinforces the central research objective: the circuit is valuable when it allows heterogeneous execution to be selected and audited coherently.

Cryptographic circuits are generally static once generated, while SOP circuits are deliberately dynamic. The connection is best understood in stages. The construction runtime first synthesizes or instantiates a SOP circuit. A stable deterministic subgraph can then be frozen, optimized, verified, or even compiled into a stronger formal representation. Dynamic structure and strong verification are therefore not opposites; they can apply at different phases of the same pipeline.

This staged view also clarifies reproducibility. A user request may produce circuit version `C0`, which is expanded into `C1` after missing evidence is detected. Once `C1` satisfies the acceptance policy, its exact source, command versions, knowledge versions, and traces can be retained. Reproducing the result means rerunning the frozen circuit against the same dependencies, not asking a planner to rediscover an equivalent plan from scratch.

The larger conclusion is that the circuit concept matters because it turns reasoning into an object that can be transformed, costed, verified, and archived. Cryptography demonstrates the extreme end of this principle. SOP Lang explores whether a looser but semantically explicit form can bring some of the same discipline to language-centric reasoning.

Cryptographic concept	SOP Lang research analogy
Circuit wire	A versioned semantic value flowing between command instances.
Gate	A command instance or stabilized subcircuit.
Mixed protocol	A circuit in which different reasoning backends execute different subproblems.
Conversion	An explicit command that translates one representation or guarantee class into another.

Cryptographic concept	SOP Lang research analogy
Witness	The structured derivation and provenance supporting an answer; only a metaphor unless compiled into a formal proof system.
Constraint	An invariant or validator that must hold before a SOP result is accepted.
Under-constrained circuit	A warning that superficial validation can accept semantically invalid results even when the implementation behaves as specified.
Arithmetization	A possible later lowering for restricted deterministic SOP subcircuits that need cryptographic proof.

The cryptographic comparison becomes clearer if circuit construction and circuit evaluation are separated. In secure computation and zero-knowledge systems, the function to be evaluated is generally compiled into a restricted circuit or constraint system before the sensitive execution begins. The circuit structure determines which relations between inputs, witness values, and outputs are actually enforced. ABY shows that one high-level computation can be split across arithmetic, Boolean, and Yao representations, with explicit conversions between them. [R19] Circom and Halo2 show that the proof is only as meaningful as the constraints generated by the circuit. [R20][R21]

SOP Lang lives one level earlier in the pipeline. Its active circuit may still contain dynamic retrieval, alternative reasoning, arbitrary command implementations, and neural calls. It is therefore not suitable as a cryptographic constraint system in general. But the architecture can progressively freeze portions of the graph. Once a semantic subproblem has been reduced to a deterministic finite relation over structured values, that subcircuit may be lowered again into an arithmetic circuit, SMT problem, proof obligation, or another strongly verifiable form.

This staged view is important for scientific claims. “The SOP circuit is verifiable” can mean several different things. The runtime may verify structural well-formedness, type compatibility, source provenance, or deterministic replay. A Z3 backend may verify satisfiability of generated constraints. A Lean backend may check a formal proof. A ZK circuit may prove a restricted computation without revealing its private witness. These guarantees should remain distinct in the value metadata and should not be collapsed into a generic verified flag.

The notion of an under-constrained circuit is particularly instructive. A cryptographic circuit can be implemented correctly and still fail to enforce the property the designer intended. The symbolic analogue is a validator that checks that an answer has a citation but never checks that the cited span entails the claim. Another analogue is a lowering command that omits an exception clause from a policy before invoking Z3. The backend can then return a mathematically correct answer for the wrong problem. Explicit structure improves auditability, but it cannot replace specification quality.

The semantic-witness analogy can nevertheless be made precise enough to guide implementation. A final SOP answer can reference the active variables on which it depends, the source spans that introduced factual premises, the command versions that transformed them, and any backend certificates produced along the way. A validator receiving ~answer can traverse this structure. It can reject answers whose claims contain no supporting path, whose critical path crosses an untrusted command, or whose evidence is stale. This is not a cryptographic proof, but it is a machine-checkable derivation record.

Verification layer

What can actually be claimed

SOP structural validation

The declarations, references, command contracts, and required metadata satisfy the runtime profile.

Verification layer	What can actually be claimed
Deterministic replay	The same frozen circuit and versioned dependencies reproduce the same declared-deterministic results.
Provenance validation	The answer has a dependency path to identified sources and transformations; this does not establish source truth.
SMT validation	The formal constraints have the reported satisfiability/model property; correctness is conditional on lowering.
Proof assistant	A formal theorem has a kernel-checked proof; semantic adequacy of the theorem statement remains a separate obligation.
Cryptographic proof	A specific lowered relation was evaluated or witnessed according to the cryptographic scheme without revealing protected inputs as defined by that scheme.

This layered notion of verification also suggests a useful optimisation target. Strong verification is expensive and unnecessary for every operation. The constructor can select a cheap heuristic circuit for early exploration, a deterministic symbolic circuit for stable reasoning, and a formal or cryptographic circuit only for the subset of claims that require stronger guarantees. Alternative commands and later meta-evaluation provide a natural SOP representation for this escalation, provided the time-budget semantics are explicit.

The cryptographic literature thus reinforces rather than proves the SOP hypothesis. It shows that explicit computational structure makes strong verification and heterogeneous execution possible. SOP Lang asks whether enough of that discipline can be retained while the graph remains high-level, dynamically constructed, and language-aware. The answer is not known; prototypes can demonstrate provenance and replay long before they demonstrate cryptographic compilation, and the document should preserve that distinction.

These precedents justify a disciplined circuit representation but do not prove that language can be compiled reliably. Compiler IRs begin from programs whose semantics are much more precise than ordinary prose. Cryptographic circuits assume that the function to be proved has already been specified. Computational graphs expose numerical dependencies but do not solve reference, pragmatics, theory choice, or epistemic authority. The research challenge lies precisely in constructing the executable object while preserving uncertainty about its interpretation.

The transferable lesson is modularity under invariants. An IR becomes useful when front ends and back ends can evolve independently while agreeing on a small set of properties. For SOP Lang, those properties include declaration identity, value production, dependency resolution, effect boundaries, versioning, and receipts. More ambitious type systems and proof obligations can be introduced by implementation profiles, but the common substrate must remain testable across them.

Chapter 5 - Symbolic, Neuro-Symbolic, and Formal Systems

5.1 Logic programming, rewriting, cognitive architectures, and bounded reasoning

The strongest argument for the feasibility of SOP Lang circuits is not that they replace mature symbolic technologies, but that their command abstraction can embed those technologies while preserving a common dependency and provenance model. Each system in the state of the art solves a different subproblem well. SOP Lang becomes interesting if it can compose them without forcing their internal semantics into one universal logic.

Prolog is the natural starting point. A Prolog engine receives facts, rules, and a query, then uses unification and goal-directed search to discover proofs or substitutions. SWI-Prolog additionally provides tabled execution, including mechanisms that memoize calls and help recursive programs terminate or avoid redundant work. [R4] In SOP Lang, Prolog is best treated as a backend command or subcircuit rather than as the global execution model. A command can receive a selected set of facts and a query, construct a Prolog programme, execute it, and return a proof object or answer linked to the input variables. The SOP planner retains responsibility for deciding when Prolog is the appropriate engine and which subset of the KB should be exposed to it.

This division solves one of the most persistent problems in symbolic AI: the distinction between inference inside a formalism and selection of the formalism itself. Prolog is excellent once the problem is encoded in Prolog. It does not by itself determine whether an ambiguous sentence should become a Prolog predicate, a temporal state transition, or an SMT constraint. SOP Lang aims to make that selection an explicit higher-level circuit.

Datalog and Souffle occupy a complementary position. Datalog's relational, fixed-point semantics makes it attractive for recursive queries and large deductive stores, while Souffle adds a typed language, components, functors, and a compilation-oriented implementation designed for large static analyses. [R3] A SOP knowledge compiler can translate declaration-oriented KB shards into Souffle relations, compute closures, and materialize only the derived results relevant to active goals. Semi-naive evaluation suggests the correct activation principle for SOP templates: propagate only newly available information instead of repeatedly scanning all known facts.

Maude offers a different conceptual tool. It is a reflective language and system based on equational and rewriting logic, and its strategy language separates the rules describing possible state transitions from control over how those rules are applied. [R5] This is directly relevant to dynamic SOP construction. SOP circuits can be treated as rewriteable terms; a construction strategy can transform a circuit after observing a failed or incomplete trace. Maude may therefore serve both as a backend for domain rewriting and as inspiration for a formal semantics of circuit rewrites.

The comparison also reveals a difference. A Maude specification emphasizes transformation of a state by rewrite rules. SOP Lang emphasizes explicit production and consumption of named values. The two can coexist: a Maude command may encapsulate a complex rewrite process and return a new SOP value, while the outer circuit records where that value came from and how it contributes to later results.

OpenCog Hyperon and MeTTa are the closest systems conceptually to a general symbolic cognitive substrate. MeTTa programmes operate over Atomspace metagraphs and combine pattern matching, variables, types, substitutions, and rewriting. Official project materials describe programmes themselves as graph structures over the Atomspace and emphasize reflective metagraph rewriting. [R6] SOP Lang shares the desire for extensible meta-level computation but chooses a narrower execution representation. The circuit has explicit producer-consumer dependencies and a deliberate

separation between value reference and variable reference. This may sacrifice some uniformity in exchange for scheduling, caching, and auditability.

The relationship should be studied experimentally rather than framed as competition. A MeTTa engine can itself be exposed as a SOP command. Conversely, SOP circuit traces could potentially be represented in a metagraph for higher-order analysis. The research question is whether the more constrained SOP dataflow representation provides enough generality for language reasoning while making circuit selection and incremental execution more tractable.

NARS contributes another missing piece: reasoning under insufficient knowledge and resources. The system is explicitly founded on the assumption that knowledge and computational resources are insufficient, and it incorporates prioritization and revision into the reasoning process rather than treating resource cost as external. [R9] This perspective aligns strongly with the SOP research problem. A time-budgeted constructor cannot pursue every derivation. It must rank tasks, preserve partial results, and stop with an explicit state of uncertainty when the budget expires. NARS should therefore be studied less as a backend to imitate and more as a source of principles for budget-aware control.

Attempto Controlled English demonstrates that a constrained natural language can serve simultaneously as human-facing text and formal knowledge representation. ACE translates controlled sentences into discourse representation structures and can support reasoning over the result. [R10] SOP Lang can use a similar controlled-language layer as a front end or validation boundary. A neural parser can propose a controlled reformulation, and an explicit circuit can reason over the validated form. The limitation is equally instructive: unrestricted language cannot be made unambiguous merely by introducing a formal backend.

The combined lesson is that the symbolic landscape already contains strong local solutions. SOP Lang should not reproduce unification, fixed-point evaluation, rewrite logic, resource-sensitive truth revision, or controlled-language parsing inside one monolithic runtime. It should provide the circuit layer that makes these capabilities selectable and composable. A successful implementation will therefore look more like a compiler and runtime for heterogeneous reasoning than like a new theorem prover.

System	How it maps into SOP Lang research
SWI-Prolog	A goal-directed logic backend command. Tabling provides concrete techniques for memoized recursive reasoning.
Datalog / Souffle	A scalable relational backend and a model for delta-driven fixed-point activation over large knowledge stores.
Maude	A rewriting backend and a theoretical analogue for transforming circuit states and expressing control strategies separately from rules.
MeTTa / Hyperon	A reflective metagraph substrate whose general meta-programming goals overlap with SOP Lang, but whose execution representation is less constrained by explicit SSA-like dataflow.
NARS	A source of principles for reasoning when knowledge and time are insufficient, particularly priority and explicit resource sensitivity.
Attempto ACE	A controlled-language front end that can establish a verifiable boundary between human-readable statements and formal representations.

The comparison with classical symbolic systems becomes more informative when it is made at the level of operational semantics rather than labels. Prolog, for example, does not merely provide “logic reasoning.” A Prolog engine takes a goal, unifies it with rule heads, recursively creates subgoals, and backtracks when a choice fails. Its execution structure is therefore a dynamically explored proof tree. Tabling changes that structure by memoizing subgoals and answers so that recursive reasoning can converge and repeated work can be avoided. [R4] A SOP command that invokes Prolog can

encapsulate this entire internal search while the outer SOP circuit records why the proof was requested, which facts and rules were supplied, and how the returned answers contribute to subsequent computation.

This separation is useful because SOP Lang should not attempt to reimplement Prolog's internal unification and search by expanding every proof step into generic SOP declarations. Doing so would discard decades of solver engineering and create enormous circuits. Conversely, treating Prolog as an opaque tool call would lose the ability to reason about its inputs and guarantee boundary. The productive boundary is a typed command interface that can expose proof objects or answer substitutions when available, while leaving the efficient resolution machinery inside the backend.

Datalog represents a different operational point. Its classical bottom-up evaluation repeatedly derives relational tuples until a fixed point is reached. Semi-naïve evaluation processes deltas so that newly derived tuples are combined with existing relations without recomputing the entire closure. Souffle compiles this style of relational reasoning into highly optimized analyses. [R3] The relevance to SOP Lang is deeper than "Datalog is a possible command." Delta evaluation offers a concrete model for template and knowledge activation: new @ values should trigger only indexes and templates that could become newly applicable because of that delta, not a rescan of the whole library.

Production systems contribute a related idea through discrimination networks such as RETE. Rather than testing every rule from scratch after each working-memory change, the matcher stores partial matches and propagates only relevant changes. This is almost exactly the systems problem faced by template circuits. The difference is that a production rule usually changes working memory or activates an action, whereas a SOP template materializes an explicit fragment of a new executable graph. The research opportunity is to reuse compiled matching techniques while preserving SOP's explicit value identities and epoch structure.

Maude and rewriting logic shift attention from proof search to transformation. A system state is represented algebraically, and rewrite rules describe allowed transitions. Strategies can control which rewrites are attempted. [R5] SOP template instantiation and epoch revision have an obvious rewriting interpretation: the current circuit plus runtime state is transformed into a new circuit. Yet the objects are not identical. Maude's primary semantics is rewriting terms or states; SOP Lang is interested in retaining a dataflow def-use graph in each resulting version. Rewriting logic may therefore be a theoretical foundation for the transformations between epochs without replacing the SOP representation executed within an epoch.

MeTTa and Hyperon provide the closest comparison to the ambition of a general reflective symbolic substrate. Their metagraph approach treats knowledge, patterns, and programmes in a highly uniform representation and supports reflective rewriting. [R6] SOP Lang deliberately chooses more operational constraint. A @ declaration creates a producer, \$ creates a value dependency, and ~ exposes meta-level access without dissolving the distinction between value and variable identity. This constraint may make some forms of reflection less elegant than in a general metagraph, but it may make scheduling, caching, incremental recomputation, and provenance easier to reason about. The empirical question is whether this loss of representational uniformity buys enough tractability.

NARS is relevant for a different reason. Its central premise is that reasoning occurs with insufficient knowledge and insufficient resources. [R9] A symbolic language model that attempts exhaustive closure before answering is not useful in interactive settings. SOP Lang makes the resource question syntactically concrete because competing commands and template instantiations can be represented explicitly, but it still needs a theory of priority. NARS suggests that priority, confidence, and resource allocation are not secondary runtime concerns; they are part of the reasoning behaviour itself.

Controlled natural languages such as Attempto Controlled English address a boundary problem that none of these solvers solve: how to obtain a formal representation from language in the first place. [R10] ACE constrains the source language so that it can be translated into a discourse

representation with a well-defined logical interpretation. SOP Lang can use such a system as a front-end command or as a validation target. More generally, a neural parser can propose a controlled paraphrase, and a symbolic validator can check whether the rest of the circuit reasons over that paraphrase rather than over an untraceable hidden interpretation.

Classical mechanism	What SOP Lang should borrow rather than reimplement superficially
Prolog resolution	Goal-directed unification, recursive subgoal construction, backtracking, and proof/answer substitution objects inside a backend command.
Tabling / SLG	Memoized subgoals, termination support, and dependency-aware invalidation for repeated or recursive symbolic requests.
Datalog fixed point	Monotone relational closure and delta-driven evaluation over large fact sets.
RETE-style rule matching	Compiled partial-match networks for sparse template activation after state changes.
Rewriting logic	A formal account of circuit/state transformation and strategy-controlled rewriting between epochs.
Reflective metagraphs	Ideas for treating programmes and knowledge as manipulable data, while testing whether SOP's stricter def-use discipline is operationally advantageous.
Resource-bounded reasoning	Priority, stopping, and explicit incompleteness when exhaustive reasoning is not affordable.
Controlled natural language	A verifiable boundary between language interpretation and formal reasoning, useful for measuring grounding errors separately from inference errors.

The scientific comparison therefore changes the research strategy. We do not need to demonstrate that SOP Lang can imitate each mature engine internally. We need to demonstrate that the language and runtime can compose them without erasing their guarantees, and that the overhead of crossing representation boundaries remains acceptable. The more difficult contribution is the orchestration semantics: deciding when to create a Prolog goal, when to request a Datalog closure, when a rewrite should create a new epoch, and when the current evidence is too weak to justify any of them.

This is also where prototypes should be instrumented. For each backend invocation, the trace should record the semantic object that was lowered, the backend representation produced, the result returned, and the conversion back into a SOP value. Errors can then be assigned to interpretation, lowering, solver behaviour, or result integration. Without this layer-by-layer attribution, a hybrid system can score well on a benchmark while hiding where its apparent reasoning ability actually comes from.

The survey reveals a recurring division of labour. Prolog and deductive databases are strong once a problem has been represented in predicates and rules. Maude is strong once change can be represented as rewriting. Cognitive architectures are strong when the research question concerns memory and control within a fixed architecture. NARS foregrounds insufficient knowledge and resources. Controlled languages make formal representation accessible by restricting expression.

SOP Lang does not make these systems obsolete. It needs to preserve enough structure to invoke them without erasing their distinct semantics. A Prolog proof, a Datalog fixpoint, a Maude rewrite trace, and a NARS judgment should not all be flattened into the same undifferentiated text value. The circuit layer provides composition; the backend retains responsibility for the meaning of its result.

5.2 Probabilistic, differentiable, solver-backed, and graph-based approaches

ProbLog extends Prolog with probabilistic facts and a distribution semantics over possible worlds. Its implementation also supports knowledge from Prolog/Datalog facts and external stores. [R7] In SOP Lang, ProbLog is a natural command backend when uncertainty is part of the formal model rather than merely a confidence score attached to a neural output. A circuit can retrieve the relevant uncertain facts, lower them into a ProbLog model, and return a probability together with the evidence variables from which that model was constructed.

DeepProbLog adds neural predicates to probabilistic logic programming, demonstrating that learned perceptual components can participate in symbolic derivations and that learning can cross the neural-symbolic boundary. [R13] This is highly relevant to a symbolic language model because language interpretation will often remain neural even if reasoning becomes more explicit. SOP Lang does not need to differentiate through the entire circuit to benefit from DeepProbLog; it can treat a DeepProbLog computation as one command whose result has a clearly identified probabilistic status. A future differentiable SOP profile could go further, but such a feature should not be assumed in the core design.

PyReason provides open-world temporal reasoning based on generalized annotated logic and produces explainable inference traces over graph structures. [R8] It is particularly suitable as a backend for event sequences, evolving states, and temporal relationships. A SOP temporal template can therefore choose between a native lightweight state-transition circuit and a PyReason command depending on complexity and budget. The result remains a SOP variable whose provenance points to the temporal facts supplied to the backend.

Scallop represents another important direction. It combines a Datalog-like relational language with provenance semirings and differentiable reasoning, allowing neural outputs and logical structure to be trained together. [R14] The key lesson for SOP Lang is the centrality of provenance. In Scallop, provenance is not an afterthought; it determines how derivations combine and how gradients or probabilities flow. SOP variables likewise need provenance as a structural property if the system is to distinguish a genuinely evidence-backed conclusion from a fluent post-hoc explanation.

Logic Tensor Networks interpret first-order logic through a many-valued differentiable semantics over tensors, allowing logical knowledge and data-driven learning to coexist. [R15] This is conceptually different from SOP's explicit circuit semantics. LTN seeks a unified differentiable representation, whereas SOP Lang seeks a heterogeneous orchestration layer in which some commands may be differentiable and others exact. The two goals are compatible. An LTN command can evaluate a family of fuzzy constraints, while a later SOP command decides whether that result is adequate for the requested epistemic policy.

Z3 represents the formal end of the spectrum. It is an SMT solver that combines specialized algorithms for background theories such as arithmetic, arrays, and bit-vectors. [R11] SOP Lang should use Z3 only after a circuit has constructed a suitable constraint set. The solver can answer satisfiability questions or provide models, but it cannot verify that the natural-language statement was

translated correctly. The circuit trace must therefore preserve the translation step as a distinct source of potential error.

Lean provides an even stronger verification boundary for explicit proofs. Its foundation in dependent type theory allows statements and proof objects to be checked by a comparatively small trusted kernel. [R12] A SOP circuit can generate proof obligations, call Lean, and receive a checked result. Again, the proof certifies the formal theorem, not the fidelity of the original language interpretation. SOP's role is to maintain the bridge from source text through formalization to verified object so that the boundary of the guarantee is visible.

ONNX is not a symbolic-reasoning system, yet its role as an extensible computational-graph specification is instructive. ONNX explicitly defines a graph model, data types, and operators for portable neural computation. [R18] It demonstrates the engineering value of a stable graph IR that decouples model producers from execution backends. SOP Lang seeks a similar decoupling for semantic and reasoning operations, but its values are not restricted to tensors and its commands can represent symbolic solvers, databases, or code-defined primitives.

The comparison with these systems clarifies why SOP Lang should not claim a universal internal semantics. A probability returned by ProbLog, a fuzzy truth degree from LTN, a model from Z3, and a checked theorem from Lean are not interchangeable evidence. They can all be SOP values, but their types and epistemic statuses must remain distinct. A powerful IR unifies composition without erasing semantic differences.

This suggests that the command registry should expose richer result contracts than conventional programming types. A result can be tagged as probabilistic, deductive, formally checked, neural, heuristic, or source-observed. The planner and validator can then require specific guarantees. A low-risk conversational task may accept a neural candidate; a scientific validation task may require a formal or independently corroborated branch.

The state of the art therefore supports the feasibility of the backend layer. Most required reasoning modes already have mature or at least research-grade implementations. What remains uncertain is whether a deterministic SOP construction runtime can choose among them efficiently and whether language interpretation can generate sufficiently stable semantic variables for the symbolic circuit to operate on. These are precisely the questions that an experimental programme should isolate.

System	SOP Lang role and guarantee boundary
ProbLog	Backend for explicitly probabilistic logical models; the SOP circuit remains responsible for constructing the relevant model.
DeepProbLog	Backend in which neural predicates participate inside probabilistic logic; useful when perception and symbolic inference must be coupled.
PyReason	Open-world temporal backend with inference traces; suited to event/state reasoning.
Scallop	Neuro-symbolic Datalog and provenance framework; provides concrete ideas for derivation tracking and differentiable reasoning.
Logic Tensor Networks	Differentiable many-valued first-order reasoning; a possible backend for soft logical constraints.
Z3	SMT backend for precisely formalized constraints; guarantees apply to the constraint system, not automatically to the source-language translation.
Lean	Proof-checking backend with a small trusted kernel; useful for critical formal subproblems.
ONNX	Evidence that a stable graph IR can decouple front ends from heterogeneous execution backends; the analogy is structural rather than semantic.

Probabilistic logic programming shows that “symbolic” does not imply that every value must be Boolean. ProbLog associates probabilities with facts and computes query probabilities from the possible worlds or derivations induced by the programme. [R7] In SOP Lang the important lesson is that uncertainty must have a declared semantics. A probability returned by ProbLog, a neural confidence score, a fuzzy degree of membership, and a heuristic planner score are all numbers between zero and one in many implementations, but they mean different things. A circuit that collapses them into one generic confidence field becomes semantically unsound even if its syntax remains clean.

DeepProbLog demonstrates a more integrated arrangement in which neural predicates participate inside probabilistic logic. [R13] Scallop similarly combines deductive relations, provenance, probability, and differentiable execution. [R14] Logic Tensor Networks interpret logical formulae in differentiable many-valued semantics. [R15] These systems are valuable precedents because they show several legitimate places at which learning can enter symbolic inference. SOP Lang should not choose one universal placement. A command may use a neural component internally; another may receive a neural candidate as an explicit `@` variable; a third may call a differentiable symbolic backend. What must remain visible is the guarantee boundary.

PyReason contributes temporal and open-world reasoning. [R8] A language task often contains two orders that must not be confused: the execution order of commands and the time order of the represented events. A SOP circuit can process an event today that describes an event from last year. Temporal relations therefore belong to semantic values or a temporal backend, not to the topological order of the active dataflow graph. This distinction, made explicit in the earlier circuit survey, is essential if SOP Lang is to handle documents, narratives, scientific observations, or regulations with validity intervals.

SMT solvers such as Z3 and proof assistants such as Lean occupy a stronger guarantee regime. Z3 can decide or search for models of precisely formulated constraints over supported theories. [R11] Lean can check proof terms against a small trusted kernel. [R12] Neither system validates the translation from natural language into its formal input. A SOP trace should therefore represent at least two distinct obligations: whether the backend result is correct for the formal problem, and whether the formal problem faithfully represents the semantic value that was intended. This prevents the common category error of treating “verified by Z3” as equivalent to “the original sentence has been verified.”

Graph-based machine-learning frameworks provide a different precedent. ONNX defines an interoperable computational graph in which nodes are operators and edges carry typed values. [R18] This makes models portable across frameworks and execution providers. SOP Lang seeks a related decoupling but at a higher semantic level. Its active graph can contain a command whose implementation is an ONNX model, another whose implementation is Z3, and another that performs indexed retrieval. The common circuit does not require those operations to share one numerical semantics; it requires their interfaces and dependencies to be explicit enough to compose.

Knowledge graphs should be kept conceptually separate from these computational graphs. A knowledge graph is primarily a representation of entities and relations. Graph neural networks may compute over such graphs, but the graph of knowledge is still not identical to the graph of computation. In a SOP system, a knowledge graph can remain persistent and huge while a retrieve command materializes a small `@subgraph` value. Subsequent commands may derive a circuit from that subgraph. Confusing the two graphs would make it tempting to load or execute the entire knowledge graph merely because it is represented as nodes and edges.

Where uncertainty or formal structure lives	SOP Lang consequence
ProbLog-style probabilistic facts	The probability is part of a formal probabilistic model; the outer SOP value should preserve that semantics and the model version.
Neural predicate inside logic	The neural component is inside the backend guarantee boundary; provenance should still record model identity and inputs.
Neural parser before symbolic reasoning	The parser output is an explicit candidate semantic value whose ambiguity and grounding error can be measured separately.
Differentiable logic	Soft satisfaction is a distinct epistemic type, not a proof. The circuit can use it for ranking or approximate constraints without relabelling it as deductive truth.
SMT result	Strong guarantee about the supplied constraint system, conditional on the correctness of lowering from SOP semantic values.
Proof-assistant result	Kernel-checked proof of a formal theorem, still conditional on the correspondence between theorem and source-language claim.
Knowledge graph	Persistent world/domain structure queried by commands; not itself the active execution graph.
Computational graph	Executable value transformation graph; conceptually close to a SOP circuit but usually specialized to numerical operators and static model execution.

The review therefore supports a layered rather than competitive interpretation. Probabilistic logic, temporal logic, SMT, proof checking, neural inference, and graph computation are alternative semantic substrates. SOP Lang is being investigated as the layer that exposes why a particular substrate was selected and how its result enters a larger chain of reasoning. The architecture succeeds only if it preserves enough information across these boundaries to make heterogeneous composition more reliable than simply letting a general LLM call tools opportunistically.

This point also identifies a likely long-term outcome. A symbolic language model may be domain-specific even if the SOP runtime is general. A biomedical profile may use a fine-tuned neural parser, biomedical ontologies, probabilistic evidence commands, and specialized temporal templates. A software-verification profile may use parsers, Datalog, SMT, and proof checking with little probabilistic reasoning. The common model is not a universal ontology; it is the circuit discipline, variable semantics, template mechanism, and budgeted constructor that allow each profile to expose its reasoning structure in the same executable language.

Neuro-symbolic systems demonstrate that perception and representation learning can be connected to explicit inference. They also show the danger of treating all forms of uncertainty as

interchangeable. A probability produced by a probabilistic programme, a fuzzy truth value, a neural confidence score, and a search priority are different objects. A circuit intermediate representation should preserve these distinctions so that downstream commands cannot accidentally present a heuristic rank as a calibrated probability or a formal proof.

Formal solvers establish another boundary. Z3 can determine satisfiability relative to a formalization, and Lean can check a proof term relative to a theorem statement and trusted kernel. Their strength makes the correctness of translation more rather than less important. A symbolic language system must keep the original language, the formal statement, the assumptions, and the solver result connected so that a proof of the wrong theorem cannot masquerade as an answer to the original question.

Graph-based optimisation also contributes partial mechanisms. E-graphs can compactly retain equivalent expressions and postpone extraction. This is attractive for circuit variants, but semantic equivalence in language is context-dependent. A rewrite valid for an algebraic expression may be universally sound, while a paraphrase equivalence may depend on jurisdiction, theory, or discourse context. The equivalence relation itself must therefore be typed and scoped.

Chapter 6 - SOP Lang as a Circuit Intermediate Representation

6.1 The minimal declaration model and command-owned grammars

SOP Lang begins from one declaration shape. A declaration opens with a produced variable and a command expression. The rest of the declaration is an argument region that may continue across any number of physical lines until the next declaration begins. The core parser identifies declaration boundaries, atomic tokens, quoted literals, and explicit references. It does not impose one global function-call grammar on every command.

This decision is central to extensibility. A retrieval command may use words such as from, about, and limit. A temporal command may use during and until. A scientific-review command may use a vocabulary for methods, evidence policies, and reporting status. A definition command may interpret formal parameters followed by a code body. These terms belong to the command grammar unless repeated experiments justify promoting a concept into the shared kernel.

The command-owned grammar is not an invitation to ambiguity. Each command implementation or implementation profile must define how it parses its argument stream, what values it accepts, what it produces, what effects it may perform, and how failures are represented. The common runtime preserves the token sequence and resolves references; the command supplies domain semantics.

A minimal declaration can be written as follows.

```
@events extract_events
$document
preserve-spans

@timeline order_events
$events
method "partial-order"
```

The first declaration creates a value named events. The command receives the value of document and an atomic token that its own parser interprets as an annotation. The second declaration consumes events and supplies a quoted literal to the order_events grammar. The indentation improves readability but does not determine dependency; the explicit references do.

This syntax is deliberately smaller than the semantic system built around it. Types, provenance policies, security labels, cost models, and authority status can initially be represented through command contracts, package metadata, or domain commands. Independent runtimes can therefore share a minimal parser while experimenting with stronger profiles.

A language intended to become an intermediate representation faces a tension between minimality and analyzability. If the global grammar is too rich, every new domain feature forces changes to the parser, tooling, and runtime. If the grammar is too permissive, static analysis becomes impossible because each command is effectively arbitrary text. SOP Lang deliberately places the extension point at the command boundary. The kernel recognizes enough structure to build the dependency graph, while command implementations are free to impose stricter schemas on their own arguments.

The declaration boundary is the first invariant. A declaration starts when the parser encounters a variable declaration token in declaration position, followed by a command expression. The declaration continues through subsequent physical lines until the next declaration starts. Quoted literals protect content that would otherwise be tokenized. \$name and ~name are recognized regardless of the command grammar because the runtime must resolve them before or during command invocation. Every other token can remain an atom until the command parser interprets it.

This representation is intentionally closer to an extensible assembly language than to a high-level programming language. The kernel does not need a universal notion of named arguments, object literals, list syntax, or annotations. A command that needs a structured sublanguage can define one.

The advantage is evolutionary: new command families can be introduced by coding agents without destabilizing the core syntax. The disadvantage is that malformed command-specific arguments cannot always be diagnosed by the global parser. The command registry must therefore expose schemas or validators when static diagnostics are important.

The same architecture appears in mature compiler infrastructures in a more formalized form. MLIR operations belong to dialects that define operations, types, and attributes, while the common IR provides generic structure and transformation infrastructure. [R2] SOP Lang's command-owned grammar is less formal than MLIR dialect definitions, but the research analogy is useful. A future implementation can allow commands to publish a machine-readable schema describing expected positional references, accepted keywords, result type, determinism, effects, and estimated cost. The global language remains unchanged while the runtime gains enough metadata for planning and validation.

A command invocation should be conceptually separated into two stages. The SOP kernel first constructs the ordered argument stream, resolving or preserving references according to the \$ and ~ markers. The command then parses that stream according to its own rules. This division makes it possible for the runtime to understand dependencies without understanding domain semantics. It can know that a declaration depends on variables claims and entities even if it does not know what the tokens where, source, or confidence mean to the command.

The distinction between literals and references also clarifies knowledge flow. A quoted literal such as "key" is embedded directly in the circuit version. A \$key reference points to a value whose origin can be elsewhere in the circuit or in a loaded knowledge shard. If the literal changes, the circuit source changes. If the referenced value changes through a new version or a different input binding, the same circuit structure can operate on the new value. This is the basis for template instantiation.

~var should be reserved for cases where variable identity matters. A validation command may need to know whether an answer was produced by a verified solver or by a neural model. A cache command may inspect the dependency hash. A selection command may compare multiple alternatives stored under one variable. A provenance command may traverse source spans and transformations. Passing all of this metadata inside ordinary semantic values would pollute domain logic and make reuse harder. The meta-reference provides a separate channel.

This separate channel also introduces security and capability questions. A variable reference can expose more information than its value. A command authorized to consume \$patientSummary may not be authorized to inspect source identifiers or hidden alternative branches through ~patientSummary. A serious SOP runtime therefore needs permissions at the command and reference level. The minimal syntax does not solve authorization, but it makes the distinction visible enough for a policy layer to enforce it.

The research profile should treat command parsing as deterministic for a fixed implementation version. If the command invokes a stochastic model internally, that nondeterminism belongs to the command's declared behaviour and must be represented in its trace. This is important for the overall objective of deterministic circuit construction. The planner can deterministically choose the same command and argument stream for the same state and budget even if a deliberately stochastic backend produces variable outputs.

The same principle applies to effects. A command that only transforms its inputs can be cached aggressively. A command that reads an external database must bind the database version or query timestamp into its dependency identity. A command that changes an external system cannot be treated as a pure node. The runtime can still represent the effect, but the contract must declare it so that replay and speculative parallelism remain safe.

This is where the simplicity of the syntax must not be mistaken for semantic informality. The global grammar can remain small while command contracts become rigorous. The research programme should explicitly separate the language kernel from the command contract system. The former enables extensibility; the latter determines whether large-scale planning, caching, and validation are feasible.

The tokens `from`, `about`, `domain`, `limit`, `granularity`, `open_world`, and `theory` are not global SOP Lang keywords in this example. They belong to the parsers of `retrieve`, `temporal_reason`, and `smt_check`. Another implementation can choose stricter positional forms or publish schemas that allow editors to validate these keywords. The point is to avoid turning every domain convention into a language feature.

A command definition follows the same principle. The syntax `@newCommand define ...` does not require the core parser to understand JavaScript. The `define` command receives its argument block and interprets the first atoms as formal parameter names and the remaining lines as source code according to the selected implementation profile.

The names after `define` are formal parameters, not circuit references; they are interpreted by the `define` command. The subsequent JavaScript lines are also part of the same declaration because no new `@variable` command declaration has begun. A restricted implementation can reject unsafe syntax, require pure functions, or compile the code into a sandbox. A non-JavaScript implementation can provide a different `define` command or omit code-defined commands entirely. The general SOP model does not depend on JavaScript; it depends on the ability to add command implementations through a command-owned definition mechanism.

This design is particularly important for coding agents. An agent that discovers a missing primitive can generate a command definition and tests without modifying the parser. An agent that discovers a higher-level pattern can generate a template circuit using existing commands. The separation between primitive acquisition and circuit acquisition becomes explicit and measurable.

Kernel responsibility	What remains outside the universal SOP Lang grammar
Detect declaration boundaries	The semantic meaning of a specific command and the validity of its domain-specific keywords.
Tokenize bare atoms and quoted literals	Whether a bare atom is a keyword, operator, type name, field name, or ordinary symbol.
Recognize <code>\$var</code> and <code>~var</code>	The internal structure of the referenced value and the metadata exposed by the variable reference.
Preserve argument order and physical-line content	Any nested command-specific mini-language, source-code grammar, query grammar, or annotation scheme.
Dispatch to a command implementation	The command's algorithms, external engines, security policy, determinism, and effect semantics.
Build the dependency graph	Domain interpretation and reasoning strategy, except insofar as commands publish contracts that the planner can inspect.

The intentionally small syntax deserves a more formal reading. A declaration begins with `@` followed by the name of the produced variable and a command expression. The argument region continues across as many physical lines as necessary until the next declaration begins. At the generic language level the argument region can be represented as an ordered stream composed of atomic tokens, quoted literals, `$` references, and `~` references. The command owns the interpretation of that stream. This means that SOP Lang can remain syntactically small while commands introduce domain-specific vocabulary without forcing every word such as `from`, `where`, `theory`, `policy`, `source`, `confidence`, or `timeout` to become a globally reserved keyword.

The command should normally receive both a normalized token view and enough raw-source information to implement its grammar reproducibly. A database command may parse relational operators. A Datalog command may interpret a block as rules. A document command may interpret keywords such as section and span. A define command may hand the remainder of the declaration to a JavaScript parser. These are command grammars, not additional layers of SOP Lang syntax. Implementations are free to constrain the accepted command grammar for safety or analyzability, but those constraints belong to a command profile or runtime profile rather than to the universal declaration mechanism.

In this example from, about, domain, limit, and theory have no universal meaning to the SOP parser. The retrieve and z3 commands assign meaning to them. The kernel only needs to preserve ordering, literal boundaries, and reference identity. This division is similar to an extensible IR whose operation families own their attributes, but it is even looser: a command may choose to parse a compact domain language from the argument stream. The benefit is evolvability; the cost is that tooling cannot infer all semantics from syntax alone. That trade-off is a central research question rather than an accidental implementation detail.

\$var and ~var should be interpreted as different operations, not merely alternative notation. \$var dereferences the current semantic value of a variable and therefore induces an ordinary data dependency. ~var passes a handle to the variable as a runtime object. The receiving command can inspect metadata that is not itself the semantic value: producer identity, command version, provenance graph, branch evaluations, cache status, timing, or validation records. A runtime can expose only a restricted immutable view through ~var so that meta-reasoning does not become uncontrolled mutation.

This distinction becomes especially important when alternatives are evaluated. If @answer is produced by direct_reason | prolog_reason | temporal_reason, \$answer may denote the policy-selected value or another value-level representation defined by the implementation. ~answer can expose the complete alternative object, including branch-specific results and unexecuted branches. A later evaluator can therefore reason about computation itself without requiring the semantic result to contain all execution metadata.

Command definition through @newCommand define extends the same mechanism to implementation growth. The formal parameters belong to the define command grammar and should not be written as \$ references, because they are names bound by the new command implementation rather than wires from the surrounding circuit. The remainder can be source code in JavaScript or another implementation language supported by the profile. Once successfully compiled, versioned, tested, and registered, the new command becomes indistinguishable at the SOP level from an older native command.

This bootstrap mechanism is powerful enough to become dangerous. If define grants unrestricted filesystem, network, reflection, or process access, the symbolic IR loses most of its analyzability and security. Production profiles should therefore treat command definitions as capability-bearing artefacts. A coding agent may author JavaScript, but promotion can require static checks, sandbox execution, tests, effect declarations, deterministic-mode tests, and an explicit version identifier. The language remains open to new primitives without making every request an opportunity to execute arbitrary untrusted code.

Kernel fact

A declaration has one @ variable name

What the research runtime still has to define

Whether names are globally unique, scoped by file/package, or versioned by epoch; the prototype convention can evolve as long as def-use identity remains unambiguous.

Kernel fact	What the research runtime still has to define
A command receives an argument stream	The command-specific grammar, typing rules, effect policy, diagnostics, and static descriptor used by planners.
\$var yields a value dependency	The exact value model, serialization, lazy loading, type compatibility, and cache identity.
~var yields a variable reference	Which metadata is visible, which actions are permitted, and how reference inspection interacts with security and privacy.
define can create commands	Sandboxing, implementation languages, compilation, versioning, regression tests, and promotion policy.
The next @ declaration ends the previous argument block	Profiles may add stricter lexical constraints, but they should avoid creating a second global language layered over SOP Lang.

The scientific value of this minimality must be evaluated empirically. If command-owned parsing causes every tool to become an incompatible mini-language, composition will be harder and static analysis weaker. If, conversely, the kernel absorbs every annotation discovered by prototypes, SOP Lang may become a large syntax whose evolution is slow and whose domains are coupled. A credible standardization path is therefore incremental: preserve the minimal token and reference model, observe recurring command contracts across prototypes, and promote only those concepts whose cross-domain utility can be demonstrated.

6.2 Values, references, alternatives, subcircuits, templates, and definitions

The produced name is an SSA-like value identity. A later declaration that consumes the value creates a def-use dependency on its producer. Revising a state should create a new value rather than silently mutate the old one. This discipline preserves history and supports incremental recomputation.

SOP Lang distinguishes receiving a value from receiving a reference to the runtime variable or artefact. The ordinary value reference is written with a dollar sign. A tilde reference asks the command to receive the variable identity or handle so that it can inspect producer information, alternatives, provenance, cache status, validation state, or other metadata defined by the profile. The distinction belongs here, in the technical model, because it matters only after values and runtime identities have been introduced.

```
@candidate infer_claim
$evidence
```

```
@assessment inspect_candidate
~candidate
policy "source-and-method"
```

The second command is not merely given the claim payload. It is given a handle through which the implementation can inspect how the candidate was produced. Receiving a handle should not imply authority to mutate the value. A revision produces a new declaration and a new identity.

An implementation may allow several command alternatives in one command expression, separated by a vertical bar. Alternatives can be executed in parallel, ranked and tried sequentially, or treated as

named strategies behind one responsibility. The shared semantic requirement should be that alternative outcomes remain distinguishable and auditable. A majority vote does not convert an uncertain candidate into truth.

Subcircuits provide hierarchical composition. A package can expose an input-output contract and be invoked as a higher-level operation. Its internal declarations remain inspectable for audit and optimization, while callers depend on a stable interface. This is the mechanism by which a large model avoids exposing every primitive to the top-level constructor.

Template circuits are stored circuits that can be instantiated over a current state. They have applicability descriptors that the constructor can index cheaply. When required value types, relations, policies, or goals are present, the template contributes new declarations and therefore new variables. The descriptor should be simpler than the circuit body; otherwise the runtime would need to execute every template to decide whether it is relevant.

Command definition allows the system to represent newly learned computational primitives. A typical profile can treat a declaration whose command is `define` as a proposal containing formal parameters and code. Representation is not installation. The code remains an untrusted artefact until it has been statically checked, tested, sandboxed, assigned a content identity, and promoted into a reviewed command registry.

```
@candidateAnswer direct_reason | temporal_reason | prolog_reason
  $question
  $context

@newTemporalCommand define
  state event
  ""
  // JavaScript body in an implementation profile
  return applyEvent(state, event);
  ""
```

The relationship between SOP Lang variables and Static Single Assignment should be stated carefully. SSA is a compiler representation in which each SSA value has one defining instruction. LLVM IR is explicitly SSA-based, and MLIR also uses SSA values as a central organizing principle. [R1][R2] SOP Lang can adopt the same invariant at the level of circuit values: within a circuit version or epoch, each `@name` should have one producer. A new state, belief, or interpretation is represented by a new variable rather than by silently replacing the value of an existing variable.

The benefit is not merely aesthetic. Suppose a text describes the transfer of an object from Ana to Mihai and later from Mihai to Elena. An imperative representation might update one mutable holder field. The final result would be easy to obtain but the intermediate epistemic history would be hidden inside mutation. A SOP circuit can instead construct `@state0`, `@state1`, and `@state2`. The path from the final state to the event that produced it becomes part of the graph, and the runtime can recompute only the descendants of a changed event.

The analogy to SSA is not exact because SOP Lang variables can represent rich semantic objects and because a runtime may maintain external mutable resources. The research invariant should therefore be framed as value-version immutability, not as a claim that all memory everywhere is in SSA form. A command that reads or writes external state must declare the relevant effect and version. Within the circuit graph, however, the value produced under a variable identity should remain stable for the lifetime of that circuit version.

This invariant interacts productively with `~var`. Meta-reasoning can examine the variable object without mutating its semantic payload. If a validation command determines that `@candidate` is weak, it produces `@assessment`. If a revision command improves the candidate, it produces `@revised`. The original candidate remains available for comparison and audit. The circuit therefore records epistemic revision rather than erasing it.

Alternative commands extend this model from a single producer implementation to a controlled family of producers associated with one declared variable. A profile may interpret `command1 | command2 | command3` as an OR-node whose branches share the same input dependencies. To preserve auditability, the runtime should record each branch as a sub-result with its own command identity, cost, status, provenance, and value. The output variable can then hold an `AlternativeSet` or equivalent structured value. An explicit selection command converts that set into a selected result when the policy demands one.

This is a useful place for `~var. $candidate` can expose the semantic contents of the alternative set, while `~candidate` can expose the branch-level execution metadata. A meta-rational evaluator may prefer a slightly slower branch because its conclusion is formally verified or because its sources are stronger. Another policy may choose the first result that meets a latency threshold. The selection policy becomes a circuit operation rather than hidden scheduler behaviour.

The ability to preserve alternatives is related to, but distinct from, e-graphs. An e-graph compactly represents many equivalent expressions and delays the choice of a concrete representative until extraction. The egg library and later egglog work show the practical power of maintaining equivalence classes and performing rewrite-driven optimization. [R16][R17] SOP alternative variables are more general and less structured: branches may be different methods rather than provably equivalent expressions. Nevertheless, e-graph techniques suggest ways to merge equivalent branches, canonicalize subcircuits, and avoid repeatedly exploring semantically redundant alternatives.

Subcircuits provide hierarchical abstraction. A SOP fragment that accepts semantic events and produces a temporal state can be packaged behind a command contract. To an outer circuit it appears as one command. Internally it remains a graph whose trace can be opened when needed. This mirrors how compiler IR regions, functions, and dialect operations hide internal detail while preserving analyzable interfaces. It also makes the planner tractable because it can select a coarse capability before expanding its implementation.

Template circuits are the central mechanism for dynamic growth. A template is not merely a textual substitution. It is a circuit fragment with matching conditions over the current state, available variable kinds, goals, command capabilities, or knowledge tags. When the matching conditions become satisfied, the runtime can instantiate a fresh copy whose formal inputs are bound to concrete variables. The instantiated declarations receive fresh variable identities, preserving the SSA-like invariant.

A transfer template illustrates the idea. The exact syntax for declaring a template need not become universal SOP grammar; a template command, package metadata, or repository convention can own that representation. What matters is the semantic model. The template can state that if a current world state, a transfer event, and a relevant object are available, then a new state can be produced by a particular subcircuit. When a new transfer event enters the state, the template index can activate the matching template without rescanning unrelated templates.

This behaviour resembles production-system matching and semi-naive Datalog evaluation. In a production system, new facts activate rules whose conditions match. In Datalog, semi-naive evaluation propagates only the delta of newly derived facts to avoid rederiving the entire fixed point. Souffle demonstrates how typed Datalog and compilation can make large recursive analyses practical. [R3] The SOP research programme can borrow the same principle: template matching should be delta-driven, indexed, and incremental.

The key difference is that a SOP template can instantiate arbitrary heterogeneous command graphs rather than one uniform logical rule language. This generality increases expressive power but also increases search risk. A production rule has a relatively predictable shape. A template circuit can

introduce many variables, invoke external engines, and spawn alternatives. The runtime therefore needs explicit cost estimates, expansion limits, and dominance checks before instantiation.

Template instantiation also provides a concrete definition of dynamic learning. A newly learned reasoning pattern can be stored as a template rather than immediately fused into a global rule engine. It remains versioned, testable, and selectively activated. If the pattern later causes regressions, the system can disable or revise that template without retraining the entire model.

This suggests a layered notion of competence. Primitive commands define atomic capabilities. Stable subcircuits package local skills. Templates define when those skills should be introduced into a task. Meta-circuits evaluate alternatives and traces. The circuit-construction runtime decides which templates deserve activation under the current budget. A symbolic language model is therefore the persistent ecosystem of these layers plus the selection machinery that assembles a small task-specific programme from them.

Mechanism	Research interpretation in SOP Lang
SSA-like binding	Each variable in a circuit version has one semantic producer; revisions create fresh variables rather than overwriting prior results.
Alternative declaration	One declaration may launch several command implementations while preserving branch-specific results and traces under a common variable.
Explicit selection	A later command evaluates or selects alternatives rather than allowing the runtime to discard them implicitly.
Subcircuit	A SOP graph with a stable contract that can be exposed as one command to outer circuits.
Template circuit	A reusable graph fragment whose requirements are matched against runtime state and whose instantiation creates fresh concrete variables.
Epoch	A stable circuit version that can be executed and traced before a planner constructs a revised or expanded version.
Delta activation	Only new or changed state elements trigger matching work, avoiding full rescans of the template and knowledge space.

The relationship to Static Single Assignment is useful only if its limits are stated precisely. Classical SSA is normally defined over a control-flow graph. Each SSA name has one static definition, and phi functions reconcile values that reach a control-flow merge. SOP Lang does not need to reproduce the entire classical construction. Its valuable invariant is narrower: within a concrete circuit version, an @ variable has one declaration that is treated as its semantic producer. A change of state should therefore create a new variable instead of mutating the old one invisibly. This is enough to obtain direct def-use chains, versioned state, selective invalidation, and provenance.

An alternative command declaration does not have to violate this invariant. The declaration itself can remain the single producer of a runtime variable whose internal result contains several branch evaluations. In other words, `direct_reason` | `prolog_reason` | `temporal_reason` is not three unrelated assignments to the same variable. It is one explicitly alternative computation whose branches retain identity. A later command can inspect `~candidate` and select, compare, aggregate, or trigger additional work. This is closer to an AND/OR computation object than to an imperative variable that is repeatedly overwritten.

The analogy with a classical phi node should therefore be used cautiously. A phi node selects the value corresponding to the control-flow predecessor that actually executed. An SOP meta-selection command may compare several branches that all executed, distinguish failed from unknown from timed out, keep multiple surviving values, or request a new epoch. It is a semantic reconciliation operation, not merely an SSA merge. Treating it as ordinary phi would hide precisely the information that the symbolic runtime needs to preserve.

Subcircuits add hierarchical abstraction. A set of declarations can expose a stable input-output contract and be treated by an outer circuit as one command. Internally it remains inspectable and traceable. This is analogous to a function call, a compiler region, or a hierarchical hardware module, but the intended guarantee is stronger than ordinary software encapsulation: the outer runtime should still be able to recover the provenance and guarantee class of the result when policy requires it. Encapsulation reduces planning complexity without requiring epistemic opacity.

Template circuits introduce a qualitatively different capability. A template is not merely a function awaiting explicit invocation. It is a reusable graph pattern associated with requirements over the current runtime state or active goals. When those requirements match, the template can be instantiated with fresh variable names and appended to the current circuit. The source template remains persistent; the instantiated declarations belong to a concrete epoch. This resembles a production rule, graph-rewriting rule, macro expansion, and partial program synthesis at the same time, but none of those analogies is exact.

The production-rule analogy is useful because a template can be activated by state. The graph-rewriting analogy is useful because activation changes the executable graph itself. The macro analogy is useful because a parameterized fragment becomes concrete source. The synthesis analogy is useful because the runtime is constructing a programme that did not exist in full before the request. The key difference from a generic rule engine is that the instantiated fragment creates explicit @ variables and \$ or ~ dependencies that become part of a traceable dataflow circuit.

Classical construction	Closest SOP Lang interpretation and important difference
SSA definition	One @ variable declaration is the semantic producer of that variable within a circuit epoch. SOP retains only the part of SSA that is useful for explicit value identity and def-use reasoning.
Phi node	A possible analogy for reconciliation, but SOP alternatives may preserve many branch outcomes and meta-information instead of selecting only the executed predecessor.
Function/module	A subcircuit can be exposed as a command with a stable contract while retaining an inspectable internal trace.
Production rule	A template can match state and instantiate computation, but its consequence is a new explicit SOP graph rather than only a mutation of working memory.
Graph rewrite	Template instantiation or epoch revision changes the executable graph. The research problem is to constrain rewrites so that construction remains deterministic and bounded.

Classical construction	Closest SOP Lang interpretation and important difference
Macro or partial evaluation	A parameterized circuit can be specialized using current variables and literals, but specialization must preserve provenance and runtime contracts.
AND/OR graph	Required inputs of a template form AND dependencies; alternative commands or alternative templates form OR choices. This interpretation becomes central to budgeted construction.

Template matching is also the point at which combinatorial explosion first becomes visible. If every new @ variable is compared with every template, dynamic composition will fail at scale even if execution of each command is cheap. The template model must therefore be designed together with descriptors and indexes. A template should expose enough summary information for the runtime to determine whether loading and parsing its full body is worthwhile. The source form can remain expressive; the compiled descriptor must be intentionally restrictive and cheap.

Another open issue is recursion. A template can instantiate a circuit that produces variables which make the same or another template applicable. This is desirable for recursive reasoning but can create infinite growth. Prototypes therefore need explicit fixpoint, depth, novelty, and budget semantics. One promising rule is that a deterministic epoch may continue only when an instantiation creates a new signature, a stronger epistemic value, or a newly satisfiable goal. If no such progress occurs, repeated activation should stop even if syntactic matching remains possible.

These distinctions make the SOP model less mysterious. The persistent model contains source templates and reusable subcircuits. A request produces a small concrete circuit. The runtime executes that circuit, observes the variables created, and may instantiate additional templates into a new or expanded epoch. Learning modifies the persistent library only after validation. The hard research question is not whether this can be implemented; prototypes already establish that it can. The hard question is whether the amount of active graph growth can remain small enough to compete with direct neural inference while preserving the explicit structure that motivates the approach.

6.3 Knowledge as declarations, circuits, and indexed external stores

A symbolic language model requires a persistent knowledge substrate, but storing knowledge as executable code creates severe scalability and governance problems. Millions or billions of facts should not become millions or billions of active command instances in memory. At the same time, separating knowledge into an unrelated database format weakens the unifying benefit of SOP Lang. The most promising design is to use SOP Lang as the persistent representation while distinguishing between inert declarations, executable circuits, and templates that are activated only when needed.

The first representation style treats facts and entities as SOP declarations interpreted by data-oriented commands. An entity command can parse identifiers and attributes. A relation command can parse subject, predicate, object, source, validity interval, or confidence. These declarations are syntactically SOP Lang, but the loader does not need to execute each one as a computational node. It can compile or index them into compact storage. The variable name becomes a stable identifier by which other declarations can refer to the fact or entity.

This approach has several advantages. Provenance can be expressed directly in the same language, domain-specific record formats can be owned by commands, and knowledge shards remain human-inspectable. A coding agent that learns a new domain fact can emit a declaration rather than modifying procedural code. The main danger is confusing representation with activation. A billion SOP declarations cannot be parsed into a billion in-memory variable objects at startup. The persistent form must support prebuilt indexes, lazy block loading, and compact binary caches.

A small knowledge fragment can look like the following. The tokens type, name, source, and valid_from are conventions of the entity and relation commands, not global grammar.

In persistent storage, the loader can index fact_001 under the relation command, the predicate token owns, the referenced variable identifiers, the source, and any domain tags published by the command parser. A query about ownership then jumps directly to relevant shards. The textual SOP form remains the source of truth, while binary indexes and compact caches act as rebuildable accelerators.

The second representation style stores knowledge as circuits. Instead of recording only that a relation exists, a knowledge unit can describe how a state is transformed or how a conclusion is derived. A transfer of possession can be represented as a state-transition circuit. A scientific claim can be represented together with the evidence-processing path that supports it. A regulation can include a template that becomes executable when the relevant facts are present. This style preserves procedural and justificatory knowledge that is difficult to encode as isolated triples.

Circuit knowledge has a higher activation cost and should therefore be used selectively. A raw database fact does not need an executable graph if its only role is retrieval. A reusable reasoning procedure benefits from being a circuit because its dependencies, alternatives, and validations matter. The knowledge layer should not force every noun or datum into the same representation class.

A hybrid architecture therefore appears strongest. Persistent SOP packages can contain declaration blocks, circuit blocks, template descriptors, command definitions, indexes, and validation metadata. Declaration blocks compile into compact fact stores. Circuit blocks load as executable graphs only when selected. Template descriptors remain in a small index that allows the construction runtime to decide when their bodies should be loaded. Command implementations are cached by package and version rather than duplicated with each knowledge item.

This resembles database query planning and compiled analysis systems more than a conventional rule engine. The runtime should never scan the filesystem to discover every fact at startup. Package manifests and indexes must identify shards by command family, ontology, key ranges, predicates, source, time interval, and other command-published fields. The index itself should be versioned so that startup can map it directly rather than rebuild it from millions of source files.

The same design supports many KBs. Each KB can have an independent namespace, manifest, index, and cache. A circuit variable can refer to a fact from another KB through an imported or resolved identifier. The runtime loads only the shards needed by the active circuit. Memory pressure can trigger eviction of inactive shards while stable indexes remain mapped or cached. This is critical if the long-term objective is knowledge at the scale of billions of facts.

SOP Lang also offers a natural place for source lineage. A fact declaration can refer to a source variable representing a document, dataset, experiment, or observation. A derived declaration can consume ~fact_001 when it needs the full provenance object rather than only the relation value. Because the reference mechanism is shared with executable circuits, the boundary between static knowledge and dynamic reasoning does not require a separate provenance language.

The danger is overloading variable identity. A persistent fact identifier must remain stable across package reloads, while an execution variable may be ephemeral and versioned per circuit instance.

Implementations should therefore distinguish persistent symbol identifiers from runtime variable instances even if both appear through SOP names. Namespace-qualified identifiers, package versions, or canonical IDs can provide this distinction without changing the surface syntax.

Knowledge consistency is another open problem. An open-world KB may contain incompatible claims from different sources. The storage layer should not collapse them into one truth value. Separate declarations can preserve source-specific claims, while reasoning circuits decide how to aggregate them for a particular task. This is where the distinction between \$var and ~var again matters: ordinary reasoning may consume the claim value, while a credibility or conflict-resolution circuit inspects source metadata through the reference.

A declaration-oriented KB also creates opportunities for Datalog-style acceleration. Relations extracted from SOP declarations can be exported or compiled into Souffle for recursive closure. [R3] The results can return as SOP variables with derived provenance. The KB therefore remains SOP-centric without requiring the SOP runtime to reimplement every database optimization.

The research programme should compare all three storage styles experimentally. A declaration-only KB may provide the best scale but force too much procedural logic into commands. A circuit-heavy KB may provide strong provenance but incur unacceptable loading and matching costs. The hybrid design is the leading hypothesis because it makes representation uniform while keeping execution selective. Its success depends on a compiler-like pipeline that separates canonical source, indexes, compact caches, and active runtime objects.

KB representation	Expected strengths and risks
SOP declarations as data	Compact, indexable, streamable, and suitable for very large fact collections. Risk: procedural relationships may be hidden inside generic commands.
SOP circuits as knowledge	Preserves derivation, state transitions, and reusable reasoning structure. Risk: expensive if treated as active graphs at KB scale.
Template circuits	Encode reusable conditional knowledge that becomes executable only when a matching state or goal appears. Risk: uncontrolled matching can create combinatorial growth.
Hybrid package	Keeps one textual ecosystem while compiling declarations into stores, indexing templates, and loading executable circuits lazily. Risk: requires a sophisticated build and cache pipeline.

The phrase knowledge base can conceal several fundamentally different objects. A persistent knowledge graph stores entities and relations. A deductive database stores extensional facts together with intensional rules. A rule base stores executable or declarative implications. A document store preserves source material from which facts may be extracted. A SOP Lang knowledge package may need to include all of these, but they should not be collapsed into one gigantic active circuit. The central architectural distinction is between canonical knowledge representation on disk and the small set of variables materialized into the active circuit for a particular request.

The most direct SOP representation treats factual knowledge as declarations whose commands construct data rather than perform expensive reasoning. A fact command can interpret its arguments as a relation record; an entity command can create an entity descriptor; a source command can attach provenance. The text remains auditable and versionable. Crucially, storing one billion such declarations does not imply creating one billion runtime nodes at startup. A build step can compile the files into relation indexes, symbol dictionaries, posting lists, compact segments, and manifests. Only facts retrieved for a task need to be materialized as runtime variables.

This representation behaves differently from a knowledge graph even when it encodes the same relation. The persistent relation can be compiled into a graph or relational store for efficient querying. When retrieved, however, the SOP declaration gives the runtime a value identity and a path back to

the canonical source. SOP Lang is therefore an authoring and interchange representation; the physical query engine does not have to preserve the textual layout in memory.

A second option represents knowledge procedurally as circuits. A rule about transitive closure, state transition, causal propagation, or document interpretation can be stored as a reusable circuit or template. This is appropriate when the knowledge is inherently about how to derive something, not merely about what is true. The danger is treating every fact as a tiny executable circuit. At very large scale that mixes storage with control, increases activation cost, and makes simple relation lookup unnecessarily expensive.

The hybrid model is therefore the strongest current research hypothesis. Ground facts, entity descriptions, source metadata, and relatively static relations remain declarative and are compiled into indexed stores. Rules whose semantics fit a mature engine may be compiled into that engine, for example Datalog. Reusable procedures and cross-engine strategies become SOP circuits. Conditional knowledge that should become active only when its prerequisites appear becomes template circuits. The single SOP ecosystem remains visible to authors and coding agents, while runtime representations are specialized for scale.

Knowledge form	Detailed SOP Lang treatment
Ground fact or relation	Canonical SOP declaration compiled into a relation-oriented index. It becomes an active @ value only when retrieved or explicitly referenced by an active circuit.
Entity and ontology metadata	Declarations compiled into symbol/type indexes and selectively loaded. Stable integer IDs may replace long textual names in runtime caches while source SOP remains canonical.
Deductive rule	Either a declarative rule consumed by a Datalog/Prolog command or a template circuit if activation must construct a richer heterogeneous subgraph.
Procedure or reasoning skill	Reusable SOP circuit exposed behind a command-like contract; suitable for hierarchical planning and reuse.
Conditional strategy	Template circuit whose descriptor is indexed by the goals, value kinds, domains, and capabilities that can activate it.
Source document	Stored as document content with indexes; SOP declarations point to spans and versions rather than duplicating the entire document in each fact.
Compiled index/cache	Operational product rebuilt from canonical SOP packages. It may be binary and highly compact because auditability is preserved by the source package and build manifest.

This design also clarifies the relation between SOP variables and knowledge identifiers. A persistent entity identifier is not automatically the same thing as an active @ variable. The entity Paris may exist in the KB under a stable ID. When a request retrieves Paris, the circuit may create @parisContext as a task-local semantic value that contains or references the persistent ID together with the relevant source set, ontology version, and request-specific interpretation. Keeping these namespaces distinct avoids accidental assumptions that every persistent concept must remain resident as an active circuit object.

The approach creates concrete systems questions. Index construction must be incremental enough that updating one shard does not require scanning an entire KB. Manifests must allow thousands or millions of source files to be located without directory traversal. Cache keys must include the versions that affect meaning. Large values should be represented by handles so that a \$ reference does not imply copying megabytes of data. The runtime must release inactive shards after their last dependent variable is no longer needed. These are not peripheral storage optimizations; if they fail, the symbolic model cannot scale beyond toy knowledge bases.

A further research question is whether declarative SOP should standardize a small vocabulary for facts or leave all factual grammars to commands. A universal triple syntax would simplify tooling but would be too weak for n-ary relations, temporal intervals, provenance, quantitative observations, and domain-specific structures. A command-owned representation can express these naturally but risks fragmentation. The likely path is a compact set of shared data constructors plus domain packages, evaluated empirically against the cost of translation between representations.

A symbolic language model needs a credible account of learning. If every command and circuit is handcrafted, the system is an expert platform rather than a model that improves through experience. SOP Lang makes several forms of learning explicit because the learned artefacts can be persisted in the language itself. The key research question is how these artefacts are proposed, tested, generalized, and promoted without turning online interaction into uncontrolled self-modification.

The most elementary acquisition is a new knowledge declaration. A document parser can create entity and relation declarations linked to source spans. This is analogous to updating an external memory rather than retraining a model. The new knowledge becomes queryable immediately if the index is updated incrementally. The risk is semantic drift: the parser may create a relation under the wrong ontology. Promotion should therefore be conditioned on validation or stored with an uncertain status that downstream circuits can inspect.

The next level is a new command. A coding agent may discover that a recurring transformation is inefficient to express as a large circuit and propose a new primitive. The define command provides a language-internal route for such acquisition. In a JavaScript profile, the agent can generate the formal parameter list and source body, along with tests and declared properties such as purity, determinism, effects, and cost class. The runtime need not trust the command merely because it compiles. A command registry can maintain candidate, validated, deprecated, and trusted states.

The next level is a new reusable circuit. Suppose a set of benchmark tasks repeatedly requires event extraction, reference resolution, temporal ordering, state updates, and answer validation. A coding agent can compose existing commands into a circuit and test it against held-out examples. If successful, the circuit can be packaged behind a stable command interface. This is a form of program learning in which the learned object is inspectable and executable.

Template learning adds a control dimension. It is not enough to know how to execute a temporal circuit; the model must learn when that circuit is relevant. A template associates a circuit shape with match conditions over goals and state. The matching conditions themselves can be generated and simplified from successful traces. If every question containing the word before activates the temporal template, the rule may be too superficial. If activation depends on semantic event types, requested output type, and available temporal relations, the template is more likely to transfer.

Strategy learning sits above individual templates. The system may learn that direct deterministic reasoning should be attempted before expensive retrieval, or that an ambiguous formalization should be preserved as alternatives until a validator can separate them. These strategies can themselves be represented as meta-circuits. A strategy is therefore not hidden policy code; it can be versioned, benchmarked, and compared.

Recent work on equality saturation provides an instructive analogue. EggMind represents EqSat strategies explicitly in a DSL and uses an LLM-guided process to synthesize reusable strategies while controlling e-graph explosion. [R22] The domain is compiler optimization rather than language reasoning, but the methodological lesson is direct: explicit strategy artefacts create a surface on which automated search can operate, and tractability feedback can guide synthesis. SOP Lang can apply the same principle to circuit construction strategies.

Learning should be separated into proposal and acceptance. A neural model or coding agent can propose a command, circuit, template, or strategy. Deterministic tests, formal checks where possible, benchmark evaluation, and regression analysis decide whether the artefact becomes active in the persistent model. This separation allows the system to exploit generative creativity without granting generative components authority over the model state.

The promotion process can also be domain-specific. A text-normalization command may require only deterministic unit tests. A medical reasoning template may require source provenance, domain validation, and calibration. A proof-generating command can be checked by Lean or another formal backend. The SOP representation can store the validation metadata alongside the artefact without introducing special global syntax because a registry command can own that metadata grammar.

The accumulation of artefacts creates a new scaling problem: model bloat. If every successful task adds a nearly identical circuit, the library becomes harder to search than the original problem. Generalization therefore becomes an explicit maintenance operation. The system can compare graphs, factor common subcircuits, canonicalize argument patterns, and merge templates whose behaviour is equivalent over a validation set. E-graphs may help for formally equivalent subexpressions, while clustering and symbolic anti-unification can identify structural similarity in more general circuits.

For a symbolic language model, the analogue of fine-tuning may therefore be a coordinated update of several layers. Domain-specific concepts enter the KB; new parsers or neural front ends improve interpretation; stable reasoning patterns become circuits; activation conditions become templates; and selection costs are recalibrated from runtime evidence. None of these mechanisms requires changing the universal SOP grammar.

The possibility of domain-specific models should be treated as a likely outcome rather than a failure. A legal model may have different command families, ontologies, and acceptance policies from a scientific model. The kernel, graph semantics, template mechanism, and construction runtime can remain common. This is similar to compiler infrastructures in which different dialects and lowering pipelines share common IR machinery without pretending that every source language has the same semantics.

Dynamic online learning should be conservative. A temporary circuit synthesized to solve one request can exist only in the current session or epoch. Promotion to the persistent library should occur only through a later validated development step; the runtime handling a user request should not silently rewrite global knowledge. In a research prototype, promotion can be an explicit command executed after tests. This preserves reproducibility and makes regressions attributable to concrete artefacts.

Learned artefact	How it changes the symbolic model
Knowledge declaration	Adds a persistent fact, entity, relation, source, or observation without changing the reasoning machinery.
Command implementation	Adds a new primitive operation that can be invoked by future circuits and templates.
Reusable circuit	Packages a successful composition of commands behind a stable interface.
Template circuit	Adds a rule for when a circuit pattern should be instantiated over a current state or goal.
Meta-circuit or strategy	Changes how alternatives are generated, ordered, evaluated, repaired, or terminated under a budget.
Index or cost model	Improves the runtime's ability to find relevant knowledge and circuits without changing their semantics.
Domain profile	Combines ontologies, commands, templates, policies, and optional neural components into a specialized symbolic language model.

Learning in this programme should be understood as controlled modification of a persistent executable ecosystem. A new fact changes the knowledge state. A new command changes the primitive operation set. A new reusable circuit changes the composition library. A new template changes the conditions under which computation can be expanded. A new strategy changes the search policy over those artefacts. These changes have very different failure modes and therefore should not share one undifferentiated promotion mechanism.

Command learning through define is the most invasive case because it introduces new executable semantics. A coding agent can generate a JavaScript implementation, but success on one task is not enough. The command needs a stable formal-parameter contract, effect declaration, test corpus, deterministic-mode behaviour where applicable, resource bounds, serialization behaviour, and a version identity. If the command wraps an external solver, the wrapper must distinguish guarantees supplied by the solver from assumptions introduced by translation into the solver language.

Circuit learning is less invasive because it composes existing commands. Its main risks are overfitting and unnecessary duplication. A circuit that solves one benchmark instance may simply encode superficial regularities. Promotion should therefore depend on transfer tests and perturbations. A promoted circuit should also be compared structurally with existing circuits so that the library does not grow into thousands of near-duplicates that make selection more difficult than synthesis.

Template learning adds another dimension: applicability. A template contains not only a useful subcircuit but also a claim about when instantiating that subcircuit is worthwhile. A coding agent can easily make this claim too broad. The result is activation storms. Template validation must therefore measure false activation as well as task accuracy. A high-quality template is not merely one that can solve a task when invoked; it is one whose descriptor and matching conditions allow the runtime to find it sparsely and predictably.

The current prototype history is valuable precisely because it has exposed these distinctions. Earlier versions of the broader circuit idea experimented with more syntax, more implicit state, and different ways to bind parameters. Subsequent prototypes have progressively concentrated the model around SOP declarations, explicit produced variables, positional or command-parsed argument streams, and dynamic loading. This evolution should be documented as normal research convergence, not hidden as if the language had been fixed from the beginning. A mature research programme should expect further simplifications if benchmarks show that a construct can be represented more cleanly by commands or templates.

Artefact maturity	Evidence required before persistent promotion
Knowledge declaration	Source provenance, schema/ontology compatibility, conflict policy, and reproducible indexing.
Command defined by code	Sandbox and capability review, unit/property tests, resource limits, deterministic profile where meaningful, and explicit versioning.
Reusable circuit	Held-out task success, perturbation stability, regression tests, cost profile, and comparison with existing circuits.
Template circuit	All circuit tests plus applicability precision, activation cost, termination behaviour, and descriptor quality.
Construction strategy	Benchmark-wide improvement under fixed budgets without hidden nondeterminism or catastrophic regressions.
Domain model	Evidence that domain-specific packages reuse the common SOP runtime while gaining measurable advantage from specialization.

This model of learning creates a useful distinction between online adaptation and persistent learning. During a request, the runtime may create temporary variables, instantiate templates, synthesize a temporary subcircuit, or even use a coding agent in a permissive research profile. None of those artefacts must automatically change the persistent model. They can be stored as candidates with traces. Promotion becomes a separate reproducible experiment. This avoids the classic problem in self-modifying systems where one anomalous interaction silently changes future global behaviour.

The strongest long-term hypothesis is that repeated neural improvisation can gradually be converted into tested symbolic artefacts. A novel task may initially require an LLM to propose a representation or circuit. If the same structural problem recurs, coding agents can extract a reusable command or circuit and validate it. Later requests can use the stabilized artefact directly. Whether this transition happens often enough to matter is unknown and should be measured by the declining or non-declining fraction of reasoning steps that require opaque neural calls as the library matures.

The emerging position is hybrid. SOP Lang should be able to represent small declarative knowledge, rules, templates, and references in a common inspectable form. Large corpora and relation stores should remain in indexed, inert formats whose identities and schemas are exposed to the circuit. Executable command bodies belong in reviewed packages, not in arbitrary knowledge records loaded at request time.

This division limits the attack surface and supports scale. A persistent symbolic model can contain many small SOP source files while startup loads an immutable catalogue rather than scanning them. Request-time execution materializes only the relevant shards and values. The active circuit remains small even when the persistent model is large.

Knowledge acquisition can use coding agents, parsers, or human authors. Their outputs enter a candidate area. Compilation and validation produce receipts. Promotion creates a new immutable

generation. The runtime binds each request to explicit catalogue and command generations, making historical execution replayable.

Chapter 7 - Constructing Dynamic Circuits Under Explicit Resource Budgets

7.1 Deterministic agenda construction and incremental activation

The viability of the entire programme depends on one algorithmic problem: given an input, a current state, a potentially enormous knowledge base, and a large library of commands and templates, how can the system construct a useful SOP circuit without enumerating an exponential number of compositions? Expressiveness is easy; selective activation is hard. The research runtime must behave like a compiler, deductive database, planner, and cache manager simultaneously, but under a user-specified deadline.

The leading design should be deterministic for fixed inputs, persistent model state, runtime version, and budget. Determinism makes benchmark comparisons meaningful and allows regressions to be localized. It does not require that every command be deterministic internally. A command that invokes a stochastic LLM can declare this property, and its random seed or trace can be recorded. The circuit constructor itself should nevertheless choose candidates through stable rules and stable tie-breaking.

The runtime state can be described abstractly as a variable directory, a set of unresolved goals, an agenda of candidate expansions, a template index, a command registry, a knowledge-index handle, a cache, and a remaining resource budget. The constructor alternates between executing ready declarations and discovering templates that can produce missing capabilities or outputs. Unlike a classical forward-chaining system, it is guided by explicit goals. Unlike a purely backward planner, it reacts incrementally to new variables produced during execution.

A useful priority relation can be lexicographic rather than a fragile learned scalar. Candidate expansions can first be ordered by goal relevance, then by the number of unsatisfied inputs, then by expected runtime cost, then by risk or guarantee class, and finally by a stable identifier. Domain profiles can refine the ordering, but deterministic tie-breaking remains explicit. Learned estimates may supply expected cost or success probability without controlling the entire semantics.

Template discovery should be index-driven. Every template publishes a compact descriptor that can be loaded without loading the body. The descriptor identifies the kind of output or capability it can produce, the kinds of inputs it expects, domain tags, effects, cost class, and optionally required backend capabilities. The runtime first searches the descriptor index using the current goal and available variable signatures. Only the highest-priority matching templates have their bodies loaded.

Matching should be incremental. When @events becomes available, the runtime should query only templates whose descriptors mention compatible event inputs or goals affected by events. This mirrors semi-naïve Datalog evaluation, which propagates new tuples rather than recomputing the entire closure. [R3] The same principle appears in production-system matching: match networks retain partial results so that a new fact triggers local work rather than a full rule scan. The exact data structure is an implementation question; the research invariant is that activation should be compiled and delta-driven.

Goal-directed narrowing provides the complementary direction. If the requested output is a VerifiedAnswer, the constructor can search backwards through templates capable of producing that type or guarantee. Their required inputs become subgoals. This prevents the forward engine from activating every template merely because its inputs happen to be available. The two directions meet when a backward-required input is already present or becomes produced by a forward activation.

The planner should distinguish type compatibility from semantic usefulness. A template may technically accept an EventSet but belong to a domain unrelated to the active task. Descriptor lookup

therefore needs several forms of evidence: output kind, input kind, domain, ontology version, required knowledge namespaces, accepted epistemic status, and expected cost. The descriptor must remain compact enough to index. If selecting a template requires parsing its full body, the architecture has already lost one of its central scalability advantages.

Alternative commands should be budgeted explicitly. A declaration with several commands separated by `|` can launch all branches only if the budget and branch policy allow it. Otherwise the runtime can schedule them sequentially in deterministic priority order or run a bounded number in parallel. The alternative variable records branches that were not executed because of budget, distinguishing not evaluated from failed, timed out, returned unknown, and produced a value.

Meta-commands can inspect alternative variables through `~var` and decide whether more computation is justified. Suppose a fast native circuit returns a low-confidence answer and a formal backend remains unexecuted. The evaluator can compare the remaining budget with the expected cost of the formal branch. This creates an anytime architecture: the system can return the strongest answer justified within the available budget, while preserving the fact that stronger checks were not performed.

The budget should be a first-class runtime parameter. It can be wall-clock time, CPU time, number of node executions, memory allocation, external-call quota, or a composite policy. For an interactive symbolic language model, wall-clock deadline is likely the primary parameter, but internal quotas prevent one slow command from monopolizing the request. Commands should publish cost estimates and support cancellation or smaller modes where feasible.

A useful construction policy can be described without introducing a new SOP grammar. The planner itself may be implemented as ordinary commands operating on `~` references to goals, variables, and candidate template descriptors. The conceptual agenda is nevertheless easier to state in algorithmic terms. The runtime normalizes the goal, retrieves compatible descriptors, orders them deterministically, instantiates only a small prefix, executes ready nodes, indexes the new variables, and repeats until a terminal condition is reached.

Construction phase	Deterministic research rule
Goal analysis	Normalize requested outputs and acceptance guarantees into goal signatures.
Descriptor lookup	Use indexes to retrieve only commands and templates capable of contributing to the active goals.
Compatibility filtering	Reject candidates whose required inputs, effects, security policy, ontology, or backend capabilities cannot be satisfied.
Priority ordering	Use a stable lexicographic cost-and-relevance tuple with deterministic tie-breaking.
Template instantiation	Load and instantiate only selected template bodies, creating fresh variables for the current circuit epoch.
Ready-node execution	Execute declarations whose \$ dependencies are available; expose <code>~</code> references according to permission and metadata policy.
Delta activation	Index newly produced variables and trigger only matching templates and pending subgoals.
Alternative management	Preserve branch status and cost under explicit alternative variables; do not collapse branches invisibly.
Validation	Test the current candidate against the requested provenance, security, consistency, and guarantee policy.
Termination	Stop on acceptance, budget exhaustion, fixed point, or explicit unresolved ambiguity and return the corresponding status.

The constructor can stop for several reasons. It may produce an output that satisfies the acceptance policy. It may exhaust the budget. It may reach a fixed point in which no new eligible template or command can advance the goals. It may detect a contradiction that requires user clarification. These terminal states should be explicit SOP values or result statuses, not exceptions hidden in infrastructure.

A deterministic constructor can still be adaptive. Historical measurements can update command cost estimates, template success rates, and cache policies. The update procedure can itself be deterministic given the historical dataset. The important distinction is between using learned estimates to rank explicit candidates and letting an opaque model directly generate an unconstrained execution graph whose legality and cost are known only after the fact.

Construction can also proceed in epochs. An initial circuit may contain only language interpretation and a direct reasoning attempt. Its execution trace can show that an input is missing or a branch is ambiguous. The planner then creates a second circuit version that adds retrieval or validation. The first graph remains available. This allows the runtime to separate construction decisions from execution effects and to test whether a rewrite actually improved the result.

The token when in this example belongs to the retrieve command's grammar or to a constrained profile for that command. The SOP kernel still only sees an argument atom followed by a ~quality reference. This illustrates an important architectural principle: control hints can be implemented as command semantics without forcing every control construct into the language core.

This construction model reframes reasoning as controlled graph growth. The system is not searching through arbitrary text continuations. It is searching through executable symbolic structures whose interfaces are partially known before execution. Types, capabilities, and dependency signatures prune the space much more aggressively than token-level generation, but the space can still explode. The next section therefore focuses on the mechanisms needed to keep growth bounded.

The construction problem can be formulated more sharply as search over an implicit AND/OR hypergraph. An active goal such as `VerifiedAnswer` is an OR node because several commands or templates may be capable of producing it. A selected template introduces an AND requirement because all of its required inputs or subgoals must become available before the relevant command can run. Template circuits may themselves create additional goals, and alternative command declarations expose local OR structure directly in SOP syntax. The full graph is never materialized because its size can be enormous; the runtime expands only a budgeted frontier.

This formulation connects SOP construction to classical planning and heuristic search without requiring a new planning language. A descriptor for a command or template can declare producible value kinds, required input kinds, domain tags, effects, lower-bound cost, and guarantee class. Starting from the desired output, a backward search retrieves descriptors that could produce it and creates subgoals for missing prerequisites. At the same time, forward delta activation reacts to concrete `@` values that have already been produced and discovers templates whose prerequisites are newly satisfiable. The runtime is therefore naturally bidirectional: backward reasoning establishes relevance, while forward activation exploits available evidence.

Several deterministic search families are plausible. A best-first agenda can order partial circuit states by a stable tuple consisting of unmet goals, guarantee deficit, estimated remaining cost, already incurred cost, and a deterministic tie-breaker. An A*-like variant can use lower-bound cost estimates where those estimates are meaningful. An AO*-like interpretation is natural for AND/OR dependencies. A bounded beam can cap memory but risks discarding the only successful plan. The programme should not choose one algorithm by analogy alone; the constructor itself is a benchmark target, and different task families may justify different deterministic policies under a common budget interface.

The main source of tractability should be indexing rather than heuristic cleverness. At build time, every command and template can be compiled into descriptors. Output capabilities can map to posting lists of producers. Required input kinds can map to compatibility indexes. Domain and ontology identifiers can eliminate semantically irrelevant candidates. Structural hashes can identify duplicates. Cost summaries can exclude candidates that cannot possibly fit the remaining deadline. This is the equivalent of compiling a rule matcher or query planner before requests arrive.

At runtime, the state should be delta-driven. When a new variable appears, the constructor updates only indexes associated with its type, capabilities, ontology, or explicit metadata. Templates with no relationship to the delta are untouched. This follows the same principle as semi-naive Datalog and RETE-style matching but applies it to programme construction. The prototype challenge is to define a compact descriptor rich enough to avoid false activations without requiring the runtime to parse every template body.

Execution and construction should interleave lazily. It is wasteful to synthesize a complete deep circuit before any command runs because early results may resolve ambiguity, satisfy goals, or reveal that a branch is impossible. Conversely, immediately executing every applicable command can create uncontrolled forward chaining. A reasonable discipline is to expand only the highest-priority frontier needed for the current goals, execute newly ready nodes, incorporate the resulting variables, and reconsider the frontier. This creates an anytime planning loop whose state remains explicit.

The example is deliberately schematic. The tokens `budget` and `remaining_budget` belong to command grammars or a constrained runtime profile, not necessarily to universal SOP syntax. The important idea is that the candidate variable is a first-class object. A meta-command can inspect which alternatives finished, which guarantees they provide, and whether the remaining budget justifies another template expansion. If a stronger branch was never run because of the deadline, the final answer can state that fact instead of treating absence as failure.

Constructor concept	Operational interpretation
Goal	A required output value, property, or guarantee that is not yet satisfied by current variables.
Producer descriptor	Compact indexable summary of a command or template that can contribute to a goal without loading its full body.
AND dependency	A template or command requires several inputs/subgoals before it can produce the desired value.
OR dependency	Several commands, templates, or alternative branches can satisfy the same goal.
Backward narrowing	Search from desired outputs to producers and prerequisites so irrelevant forward rules are never considered.
Forward delta activation	React only to newly available variables and newly satisfied conditions.
Agenda	Deterministically ordered set of candidate expansions and executable declarations.

Constructor concept	Operational interpretation
Budget	Explicit deadline or composite resource policy that limits both construction and execution.
Terminal status	Accepted result, justified partial result, ambiguity requiring clarification, fixed point without solution, or budget exhaustion.

The constructor should be deterministic for a fixed model snapshot and budget even if some commands are stochastic. This means that descriptor lookup, candidate ordering, pruning, and tie-breaking are stable. A stochastic command can record its seed or run identity, and an alternative object can preserve variability. Deterministic construction is scientifically valuable because it allows a regression to be attributed to a model change rather than to hidden planner randomness.

What remains unknown is whether descriptors and local indexes can prune the search enough for open-ended language. The risk is that semantic applicability cannot be summarized cheaply, so the planner must inspect many templates or invoke an LLM just to decide which symbolic operation to run. One of the central falsification criteria should therefore be the fraction of the template library examined per request as the library grows. If that fraction approaches a constant large percentage instead of shrinking through sparse activation, the architecture will not scale as intended.

The constructor is best understood as a bounded planner over capabilities rather than as a free-form programme generator. The goal and current values define an initial frontier. Indexes retrieve a small set of candidate producers. Exact applicability checks eliminate candidates that violate types, policies, or missing prerequisites. Execution produces a delta of new values, and only that delta is used to activate the next round.

Determinism does not require one fixed thread schedule. It requires a canonical allocation policy, stable branch identities, explicit budgets, and a canonical receipt. Independent operations can run concurrently as long as scheduler accidents do not change which outcomes are admissible or how exhaustion is reported.

A constructor may use historical statistics to order candidates, including a learned ranker, without delegating semantic authority to that ranker. Ranking changes the order in which the finite budget is spent. Validation remains the responsibility of the invoked method and authority gates. Coverage tests must reveal when ranking systematically hides a valid route.

7.2 Scaling persistent symbolic models without combinatorial explosion

Combinatorial explosion can enter the system through several channels: too many knowledge facts, too many applicable templates, too many alternative circuits, repeated recursive reasoning, and semantic ambiguity that persists across many steps. These problems require different controls. A single global beam width is insufficient because it may suppress useful reasoning while leaving the expensive source of explosion untouched.

Knowledge scale should be handled primarily by indexing and physical organization, not by planner heuristics. Facts should live in shards that can be addressed by compact indexes built during KB compilation. A request about one concept should not require scanning thousands of files. Index roots can map domain tokens, variable identifiers, relation keys, source ranges, and type summaries to shard offsets. The runtime opens only the required fragments and releases them when no active circuit references them. Memory-mapped index structures and immutable segment files are attractive because they allow many KBs to coexist without loading their contents.

The textual representation can still consist of many small SOP files for modularity, audit, and version control, but the runtime should never discover them by directory traversal at request time. A manifest generated during KB build can map logical identifiers to files or packed segments. A second-level cache can retain hot parsed blocks or compact binary forms. The source remains auditable; the cache provides operational scale.

The persistent index should be considered a compilation product rather than primary knowledge. If it is deleted, it can be rebuilt from canonical SOP sources. This allows aggressive representation choices without sacrificing inspectability. The index can use integer symbol IDs, compressed posting lists, relation-specific layouts, bloom filters, or other structures that would be unpleasant as authoring formats.

Template scale requires signature indexing. Every template should publish a summary that is much smaller than its body. The summary can include output capability, required input categories, domain, minimum guarantees, estimated cost, and a canonical structural hash. Indexing these descriptors makes it possible to retrieve a small candidate set for a given goal. If descriptors are not expressive enough to prune the library, the template model itself needs to evolve rather than relying on brute force.

Subsumption can reduce duplicate templates. If one template is a strict specialization of another but does not provide better cost or guarantees for any indexed context, it can be removed or demoted. Structural canonicalization can identify templates that differ only in variable names. A validation corpus can detect behaviourally similar templates even when syntactic comparison is insufficient, though such equivalence remains empirical unless formally proven.

Alternative-circuit scale requires dominance pruning. Suppose two branches produce the same output kind from the same inputs, but one is slower, uses weaker sources, and provides no stronger guarantee. The dominated branch should not be scheduled. A branch can also be pruned when its lower-bound cost exceeds the remaining budget or when its unresolved inputs cannot be satisfied by any indexed template.

Memoization is essential. SWI-Prolog tabling shows the general benefit of remembering previously solved calls in recursive reasoning. [R4] SOP Lang can memoize command invocations by a key derived from command version, argument values or variable identities, relevant context versions, and declared effects. Template instantiations can also be memoized if the same bindings recur. Cache correctness depends on precise effect declarations; a hidden dependency on current time or external state invalidates naive reuse.

Fixpoint detection prevents repeated expansion. If an epoch executes and produces no new variable signatures, no stronger guarantees, and no new eligible templates for unresolved goals, further deterministic construction under the same state is pointless. The runtime should stop rather than cycle. If a command produces monotonically improving lattice values, a convergence criterion can replace exact equality.

E-graph-inspired compaction can help when many circuit expressions are known to be equivalent. [R16][R17] Instead of storing every rewritable form, the runtime can store one equivalence class and defer extraction until cost information is available. This should be limited to formal fragments with trusted rewrite rules. For competing semantic interpretations, the correct structure is an alternative set, not an equivalence class.

Time budgeting should influence search before expensive work begins. A candidate template can expose a lower-bound cost. If its minimum cost exceeds the remaining deadline, it should not be instantiated. External backends can offer modes with different costs, such as a fast heuristic parser and a slower formal check. The planner can treat these as separate command alternatives with known guarantee levels.

The runtime should also distinguish construction cost from execution cost. A sophisticated planner that spends hundreds of milliseconds finding a ten-millisecond circuit may be inferior to a simpler direct circuit under an interactive budget. The benchmark must therefore measure end-to-end latency, including index lookup, template matching, circuit loading, execution, and validation.

The billion-fact target creates a deeper architectural issue: most knowledge should remain inactive. A symbolic language model cannot behave like a neural model in which every parameter participates indirectly in every forward pass. Its advantage is precisely sparse activation. The research metric should therefore include the number of KB shards loaded, facts materialized, templates examined, and variables created per request. These quantities expose whether the architecture is genuinely sparse or merely hiding exhaustive work behind a database call.

Multiple KBs should be treated as first-class namespaces with separate trust and policy domains. A circuit may query a public scientific KB, a private organizational KB, and a task-local scratch KB. The planner should not merge them into one global store. Their descriptors, indexes, and security labels can be consulted independently, and ~var references can preserve origin.

At extreme scale, command implementations should be able to operate on iterators or handles rather than materializing full value sets. \$facts may represent a lazy relation view whose command contract allows streaming. This is another reason not to equate a SOP variable with an in-memory object. A variable is a semantic value identity; the physical representation may be a compact handle to data on disk or in a remote store.

The active circuit itself should also be bounded. Variables no longer reachable from unresolved goals, output obligations, validation obligations, or retained trace roots can be released from hot memory. Their identities and serialized values can remain in a trace store if audit requires them. This makes the graph dynamically loadable without changing the semantic history.

Explosion source	Primary control mechanism
Billions of facts	Prebuilt indexes, immutable shards, lazy loading, compact handles, and eviction of inactive data.
Thousands or millions of templates	Descriptor indexes, goal-directed lookup, type/capability filtering, subsumption, and lazy body loading.
Repeated recursive reasoning	Memoization, tabling-inspired caches, and explicit fixpoint detection.
Many competing plans	Deterministic cost ordering, bounded alternative sets, dominance pruning, and lower-bound budget checks.
Equivalent circuit rewrites	Canonicalization and e-graph-style compaction only where equivalence is trusted.
Ambiguous language interpretations	First-class alternatives that remain distinct until evidence or policy justifies selection.
Expensive external backends	Cost-profiled command alternatives, cancellation, and budget-aware scheduling.
Large active state	Reachability-based eviction of variables and KB shards no longer required by active goals, outputs, or audit policy.

The scalability programme therefore depends on cooperation between language semantics, storage layout, indexing, and planning. No single clever search algorithm can compensate for a KB that must be scanned or a template library with no descriptors. The most important engineering objective is to make irrelevant knowledge and circuits cheap to ignore.

Scalability has at least four independent dimensions that should be measured separately: the number of persistent knowledge items, the number of reusable commands and templates, the number of active variables in one request, and the number of alternative partial circuits considered by

the constructor. Optimizing one dimension does not automatically solve the others. A Datalog backend may handle billions of tuples while the SOP planner still collapses under ten thousand templates; conversely, an excellent planner may be useless if every KB lookup requires loading gigabytes of source.

Persistent knowledge scale is primarily a database and storage-layout problem. Canonical SOP source can remain split into many reviewable packages and files, but runtime discovery must rely on compiled manifests and indexes. File-system traversal is not a viable query algorithm. Logical identifiers should map directly to shards or packed segments. Symbol names can be interned into compact numeric IDs. Relation-specific postings can allow retrieval by subject, relation, object, type, source, or domain. Bloom filters or summaries can prevent unnecessary shard loads. These representations are caches or build products; the authoritative semantic source can remain SOP text.

Template-library scale is closer to compiler and planning infrastructure. Template bodies should be cold until their descriptors survive goal and compatibility filtering. Descriptors can be stored in compact tables that fit in memory even when the bodies do not. A descriptor may expose only information needed for planning, such as produced capabilities, prerequisite classes, domain, effects, estimated cost, and structural fingerprint. If a template requires a sophisticated semantic model merely to determine applicability, that model can be represented as a second-stage matcher invoked only after coarse indexes have reduced the candidate set.

Active-circuit scale is controlled by hierarchical abstraction and garbage collection. A subcircuit that has produced a stable result can often be collapsed to an interface-level object while retaining its trace on disk. Large retrieved objects can be referenced by handles. Variables whose consumers have completed and whose traces are persisted need not keep their full payloads in RAM. This is analogous to spilling compiler values or evicting database pages, except that provenance must remain recoverable. A memory-sensitive runtime therefore needs separate policies for semantic value retention, trace retention, and persistent cache retention.

Alternative-search scale requires dominance and subsumption. Suppose two partial circuits can produce the same required guarantee from the same available inputs, but one has strictly greater lower-bound cost and no stronger provenance, security, or epistemic property. The more expensive partial circuit is dominated and can be pruned. If one template is a specialization of another with no measurable advantage in its specialization domain, the library can demote or merge it. These decisions must be conservative; an apparently redundant circuit may embody an exception that is not visible in a coarse descriptor.

Memoization and tabling address repeated subproblems. A cache key cannot simply hash the textual arguments. It must include command version, dereferenced values or stable variable identities, relevant ontology and KB versions, declared environmental dependencies, and effect policy. For pure deterministic commands, this can yield aggressive reuse. For commands whose meaning depends on current time, external services, or mutable state, the key must include those dependencies or caching must be disabled. Hidden effects are therefore not merely a software-engineering smell; they directly undermine symbolic scalability.

Recursion and repeated template activation require fixpoint semantics. The runtime should distinguish syntactic novelty from semantic progress. A new variable that is equivalent to an existing value may not justify further expansion. Lattice-based domains can define “stronger than” so that only genuinely improved knowledge propagates. Datalog provides a mature model for monotone closure; non-monotone reasoning requires explicit revision epochs so that withdrawn conclusions do not masquerade as monotonic additions.

Explosion dimension	What must be demonstrated experimentally
Knowledge volume	Query latency and memory should depend mainly on the retrieved shard set, not on total canonical KB size.
Template count	Descriptor lookup should examine a sparse candidate set whose relative size does not grow linearly with the whole library.
Active variables	Peak memory should be bounded by active reasoning state and handles, with old payloads evictable without losing provenance.
Alternative circuits	Dominance, lower-bound cost, and explicit branch budgets should prevent exponential frontier growth on benchmark families.
Recursive activation	Memoization and fixpoint detection should terminate repeated reasoning whenever the semantic state stops improving.
Equivalent plans	Canonicalization or trusted e-graph techniques should reduce duplicate work without merging unresolved semantic alternatives.
External calls	Command cost profiles, cancellation, and cache policy should prevent a few expensive tools from consuming the full request budget.
Updates	Changing one KB shard, command, or template should invalidate only dependent indexes and cached results rather than rebuilding the model globally.

The time budget should be treated as a semantic input to construction, not only as an operating-system timeout. A twenty-millisecond request and a ten-second request may legitimately produce different circuits and different guarantee levels. The short budget might select a cached or heuristic circuit and return a qualified answer. The longer budget can activate retrieval, run several alternatives, invoke a formal solver, and validate provenance. The trace should make these differences explicit so that “unknown under this budget” is distinguishable from “false” or “no proof exists.”

This also suggests an evaluation surface analogous to scaling laws, but for symbolic activation. Instead of measuring only accuracy against parameter count, the programme can measure accuracy and guarantee quality against time budget, active KB fraction, active template fraction, number of instantiated declarations, and number of neural calls. A promising symbolic model should improve as the persistent library grows while keeping the active fraction sparse. If adding knowledge or skills causes proportional increases in per-request search, the model is accumulating information without learning how to access it.

No single mechanism is likely to solve combinatorial growth. The research hypothesis is architectural cooperation: compile compact descriptors; index persistent knowledge; perform backward goal

narrowing; react to deltas; instantiate templates lazily; preserve alternatives only while they remain non-dominated; memoize repeated subproblems; stop at fixed points; and let an explicit budget decide how far stronger validation can proceed. Each mechanism is familiar in another field. What is unknown is whether their combination remains coherent when the objects being selected are open-ended SOP circuits generated around natural-language tasks.

The decisive scaling metric is activation efficiency. Increasing the persistent model by an order of magnitude should not increase startup work or candidate examination by the same factor. The catalogue must provide route-level summaries, type and relation indexes, dependency metadata, source locations, and materialization hints. Cache eviction should affect performance rather than logical identity.

Negative answers require special care. A constructive positive witness can terminate early. A claim that no solution exists may require complete exploration of a finite, well-defined frontier. If the frontier is truncated by time, the correct result is not false but incomplete or unknown. Count and universal queries have similar completeness obligations.

The constructor must also distinguish semantic expansion from language recovery. A failed parse may require a different interpretation strategy. A successfully parsed but unsupported goal may require new knowledge or a new capability. A solver timeout may require a weaker method. Mixing these failures into one generic retry loop is a source of combinatorial waste and poor explanations.

At scale, the research programme should compare several families of constructors under a common SOP contract: RETE-like discrimination networks, Datalog-style semi-naive activation, tabled goal-directed search, cost-based query planning, hierarchical capability routing, and learned ranking constrained by deterministic admissibility checks. The objective is not to declare one classical method universally superior but to determine which combination maintains coverage, bounded latency, and small active circuits across realistic workloads.

Chapter 8 - Provenance, Research Objects, and Trust

8.1 From rerunnable computation to semantically inspectable research

Executable science should be positioned as an extension of reproducible computation, not as a replacement for it. CWL, REANA, notebooks, containers, and executable research objects already address the practical problem of rerunning data analyses. Whole Tale explicitly defines an executable research object that combines data, code, environment, and narrative. RO-Crate provides a standards-based package and metadata layer. W3C PROV supplies an interoperable vocabulary for derivation. [R32][R33][R34][R49][R50][R51]

The proposed contribution is a semantic execution layer above and between these artefacts. A scientific claim can be linked to the workflow outputs, source data, statistical method, assumptions, and validation gates that support it. A question activates the relevant claim slice rather than rerunning every artefact in the research object. A correction invalidates only dependent slices.

The Open Research Knowledge Graph and nanopublications demonstrate complementary ways to decompose scholarly communication into machine-actionable contributions or small provenance-bearing assertions. SOP circuits can consume and produce such objects, but should not duplicate their publication and identifier functions. The circuit supplies executable dependency and validation structure; the scholarly infrastructure supplies stable public objects. [R52][R53]

This architecture permits counterfactual analysis. A reviewer can substitute a baseline, alter an exclusion criterion, remove a source, or instantiate an alternative theory and ask which claims change. Re-execution is therefore not limited to repeating the authors' path. It can explore the sensitivity of the argument while preserving the distinction between observed data and hypothetical intervention.

The boundary between formal and empirical evidence remains explicit. A theorem, a simulation, an observational dataset, a laboratory measurement, and an interpretive judgment have different validity conditions. A common circuit language should preserve their types and legitimate consumers rather than force them into one proof relation.

A scientific article in this model acquires an executable companion. The prose remains the medium for explanation, context, and concepts that cannot yet be formalized. The companion represents the formalizable portion and records where execution depends on an informal mapping. This boundary is part of the result.

Infrastructure layer	Role in an executable scientific object
Notebook or workflow	Executes computational methods and records data transformations.
Container or environment description	Preserves software dependencies and runtime context.
RO-Crate or Whole Tale	Packages artefacts, metadata, people, code, data, environment, and narrative.
W3C PROV	Exchanges provenance about entities, activities, and agents.

Infrastructure layer	Role in an executable scientific object
ORKG or nanopublication	Publishes machine-actionable scholarly contributions or small provenance-bearing assertions.
SOP circuit companion	Connects questions and claims to the bounded semantic procedures, evidence, assumptions, and validation gates required to evaluate them.

8.2 Authority, uncertainty, abstention, and governance

Traceability is necessary and insufficient. A system can faithfully record the path from a false source to a false conclusion. Provenance answers where a value came from. Credibility, validity, and authority require additional policies and evidence. The circuit should therefore treat epistemic status as structured information rather than as one confidence number.

A result may be directly quoted, deductively derived, statistically estimated, heuristically classified, proposed by a language model, verified by a solver, contradicted, or left unresolved. These statuses are not a single linear scale. Formal verification of a theorem is different from empirical replication, and two independent sources are different from a mathematical proof.

Abstention is a core capability. A runtime should be able to report that the input is outside its language profile, that required knowledge is missing, that alternatives remain unresolved, that a search frontier was truncated, or that a validator rejected the candidate. A fluent fallback may provide contextual material, but it must not be presented as proof.

Governance requires separation of duties. A model or coding agent may propose a command, template, mapping, or claim. A compiler may check syntax and interfaces. Tests may validate behaviour within a scope. An independent verifier may examine evidence. A maintainer or institution may promote the artefact. These roles should be representable and enforceable rather than merely described in policy documents.

The programme contributes mechanisms to trustworthy AI, not a blanket guarantee. NIST's AI RMF treats trustworthiness as a family of context-dependent properties. ISO/IEC 42001 establishes an organizational AI management system, and ISO/IEC 42006 addresses credible audit and certification of such systems. OECD principles emphasize innovative and trustworthy AI that respects human rights and democratic values. SOP circuits can support transparency, accountability, traceability, reproducibility, and bounded operation, but they do not automatically establish fairness, safety, privacy, legal compliance, or social legitimacy. [R30][R62][R63][R64]

The EU regulatory context increases the practical value of explicit participation records and disclosure, but the architecture should not claim automatic compliance. Legal obligations depend on the system, role, and use case. What SOP can provide is a technical basis for recording which models, tools, knowledge versions, policies, and humans participated in a result and which operations had authority.

Red-team evaluation should target the boundaries. Untrusted documents can attempt to inject instructions. Restricted values can attempt to flow to unauthorized backends. A high-confidence proposal can attempt to bypass a failed verifier. A source correction can test invalidation. A historical receipt can test replay. Trust is demonstrated by resistance to these concrete failure modes, not by the presence of an explanation page.

Trust-relevant distinction	Why the distinction must remain machine-visible
Source versus derived value	Prevents retrieved or quoted material from being confused with a system conclusion.
Candidate versus promoted artefact	Prevents generated code, rules, or knowledge from acquiring runtime authority automatically.
Confidence versus authority	Allows confidence to guide search without authorizing truth.
Evidence versus proof	Prevents related context or citations from being presented as a completed derivation.
Completed versus truncated search	Makes budget exhaustion visible and blocks false negative conclusions.
Deterministic execution versus generative proposal	Allows users to understand which parts can be replayed exactly and which remain model-dependent.
Current generation versus historical receipt	Supports reproducibility after knowledge, commands, or policies evolve.

Part III - Experiments and a Programme of Research

The final part turns the architectural argument back toward the existing prototypes, identifies what they establish, and specifies the experiments required before the programme can support stronger claims.

Chapter 9 - Three Prototypes and What They Establish

9.1 SOPLangCircuits: persistent knowledge and bounded runtime selection

SOPLangCircuits is the experiment in the family that most directly asks whether the circuit metaphor can become an operational runtime rather than a notation. Its README describes a reference runtime that answers controlled natural-language requests by selecting and executing SOP Lang circuits while keeping knowledge, commands, and reasoning artefacts in SOP files. The runtime does not treat the knowledge directory as an in-memory programme that must be parsed on every request. It publishes immutable catalog generations and starts from the published catalog, so request latency is not coupled directly to the number of source files. Candidate circuits are identified, their inputs are matched, and branches are evaluated under an explicit time budget. Caches hold route shards, SOP source objects, compiled executables, and materialized knowledge packs within a configurable byte budget. [R27]

This experiment attacks one of the most serious objections to the circuit proposal: a symbolic system may be elegant at the level of ten or one hundred rules yet become unusable when the persistent model contains millions of concepts and circuits. A naïve implementation would repeatedly scan source files, construct enormous graphs, or test every template whenever a new value appears. SOPLangCircuits instead treats source SOP as a canonical representation from which request-oriented indexes and materializations are derived. That is analogous to the distinction between a database schema plus stored data and the query plan constructed for a single SQL request. The persistent model can be large because the active circuit is intentionally small.

Experimental mechanism	What it teaches the research programme
Immutable catalog generations	SOP source and runtime lookup structures should be separated. A request can bind to a content-defined catalog generation, making execution reproducible even while a newer knowledge generation is being prepared.
Route shards and materialized packs	The unit loaded at runtime should be an indexed fragment relevant to a goal, not an arbitrary source file and not the complete knowledge graph.
Primary time budget	Reasoning cost must be part of semantics. A symbolic system that can only answer after unbounded search is not a practical replacement for an LLM call.
Branch evaluation and backtracking	Alternative circuits are operational objects whose cost and outcome can be recorded. Alternatives are not merely prose suggestions from a planner.
Cache independence from logical results	Evicting regenerable artifacts must affect performance, not the truth of values already established in the active branch.

Experimental mechanism	What it teaches the research programme
Positive versus completeness-sensitive queries	A constructive witness may justify early success, while negative, count, and exhaustive answers require stronger saturation conditions. Budget exhaustion therefore has semantic consequences, not merely latency consequences.

The distinction between constructive and completeness-sensitive answers is particularly important. If a circuit is searching for one path from Alice to David in an ancestor relation, finding a valid path can be a complete positive witness. If the question is whether no such path exists, timing out before exploring the relevant frontier cannot justify a negative answer. Likewise, an instruction to count all matching entities requires evidence that the search space was not truncated. This forces the runtime to represent not only answer values but also completion state. A result may be TRUE with witness, FALSE after complete saturation, UNKNOWN because the budget expired, or PARTIAL because some but not all requested results were established. This is a deeper semantic discipline than a workflow engine that reports only success or failure of tasks.

The experiment also clarifies that knowledge representation and active circuit representation are related but not identical. SOPLangCircuits includes direct facts, generic rules, transitive relations, permissions, risk classification, and cross-KB references. A fact may exist persistently without being materialized as a request-local @ variable. Once selected, the runtime can create a value whose producer and source generation are explicit. This is the practical form of the distinction developed earlier between a knowledge graph and a dataflow circuit: the knowledge graph describes potentially relevant relationships in the world, whereas the active SOP circuit describes which of those relationships and which operations are currently participating in a computation.

One implementation-specific detail is useful precisely because it should not be mistaken for the final language definition. The current SOPLangCircuits profile uses a syntax of the form ~kb/path#name for a persistent exported value, while the broader SOP Lang model developed in this survey gives ~var the more general meaning of passing a reference or handle to a variable rather than its value. These interpretations can be reconciled by treating a persistent KB export as one kind of handle. The stable kernel should define the semantic category — value access through \$, handle or reference access through ~ — while individual profiles can define resolvable handle namespaces. This is preferable to freezing an experiment-specific path syntax into the universal grammar. [R27]

The example illustrates how the research language can generalize the runtime lesson without prescribing one selector. kb_select may return a compact set of handles or materialized values. The declaration for @riskPath exposes several alternatives through | in an implementation profile that supports alternative command evaluation. The verifier receives ~riskPath rather than merely \$riskPath so it can inspect which alternative produced which candidate, whether a branch exhausted its budget, and what evidence is attached. The command names and keywords remain command-owned; SOP Lang itself only needs the declaration boundary and reference markers.

What remains unknown is at least as important as what has worked. A catalog proves that source scanning can be avoided; it does not yet prove that candidate selection remains accurate when there are millions of templates, many ontologies, and partially overlapping domains. A time budget proves that search can be bounded; it does not determine the optimal allocation of that budget among retrieval, alternative circuits, verification, and explanation. Cooperative deadline checks in synchronous JavaScript demonstrate one mechanism, but a hardened runtime may require process isolation or preemptible workers for untrusted or expensive commands. The central scaling experiment must therefore measure not only median latency, but missed applicable circuits, false

candidate activations, completeness status, cache sensitivity, and degradation as the persistent catalogue grows by orders of magnitude.

A second open problem concerns the semantics of templates. If a newly materialized variable makes ten templates applicable, and each generated circuit creates values that activate ten more templates, the graph can grow exponentially even though lookup is indexed. The stable architecture needs novelty criteria, typed preconditions, goal relevance, dominance rules, bounded frontier widths, and explicit fixpoint semantics. SOPLangCircuits shows that runtime selection is implementable and that catalogues can make it cheap enough for controlled examples. It does not yet establish the general constructor. That boundary between proven mechanism and open research question is precisely why the repository is valuable as an experiment rather than as a final design.

The prototype's strongest contribution is operational rather than linguistic. It demonstrates that a SOP-based model can maintain many source files while starting from a published immutable catalogue, load only relevant materialized knowledge, and execute branches under an explicit time budget. Its distinction between conclusive positive witnesses and negative or complete queries that require untruncated saturation is especially important for trustworthy answers. [R27]

The prototype does not establish language-model-scale coverage. Its controlled input and example knowledge bases are deliberately limited. The open research question is whether the same indexing and activation principles remain effective when the catalogue contains much broader linguistic capabilities, overlapping ontologies, and many alternative circuits.

9.2 dynamicCircuits: coding agents as compilers of documents

dynamicCircuits tests a different and equally important hypothesis: even if SOP circuits can execute reliably, who writes them? Hand-authoring every circuit would reproduce the classical knowledge-engineering bottleneck. The experiment therefore places a coding agent in the slow path. Given only a knowledge-base directory, the tool treats source documents as material for knowledge learning; reviewed circuits occupy a trusted area, generated proposals are written separately as candidates, and reports document the process. Given a work directory, the agent first constructs SOP for the current request and facts and then a root circuit that composes the task-specific package with reviewed KB SOP. After the coding agent exits, a local compiler/runtime executes a fixed task entry point and writes the official result from public outputs and receipts. The coding agent may record provenance, but it cannot reinterpret or author the official runtime result. [R28]

This architecture is important because it breaks a common but unsafe equivalence in agentic systems: the component that proposes a programme need not be the component that certifies its result. An LLM or coding agent is excellent at searching a large design space, reading heterogeneous documents, inventing a useful decomposition, and writing new code. Those are precisely the activities for which deterministic symbolic search may initially be weak. Yet the same generative flexibility makes the agent a poor authority for deciding that its own programme has been executed correctly. dynamicCircuits turns the agent into a compiler or synthesis assistant and places the result boundary in the executor.

dynamicCircuits boundary

General research lesson

kb/input versus kb/circuits

Source documents are not automatically trusted executable knowledge. Reviewed SOP is a distinct promoted artefact class.

dynamicCircuits boundary	General research lesson
kb/candidates	Learning means proposal of inspectable symbolic artefacts, not silent modification of model weights or immediate mutation of runtime authority.
Task SOP versus reviewed KB SOP	A request may be interpreted locally without rewriting the persistent knowledge base. Task-specific uncertainty is therefore separated from reviewed domain rules.
Coding agent exits before official execution	Synthesis authority and answer authority are separable. The symbolic executor owns the official output.
Runtime receipts	The result is paired with machine-readable evidence about package identities, node execution, hashes, checks, child executions, and outcome.
Incremental reruns	If only SOP changes, re-execution can occur without repeating language synthesis; if inputs or reviewed knowledge change, the slower synthesis path is invoked again.

The candidate directory is more than an implementation convention. It suggests a stable learning model for symbolic language systems. In a neural model, a gradient update changes the internal parameters and the resulting capability is difficult to inspect as an isolated object. In the SOP approach, learning can produce a candidate command, template, mapping, or circuit with source provenance and tests. The candidate can fail compilation, fail regression tests, prove over-specialized on transfer data, or conflict with a reviewed rule. Passing automated checks still need not promote it. Promotion is an authority decision with its own receipt. This makes learning slower than an unconstrained in-context update, but it also makes the change legible.

The experiment also demonstrates a useful decomposition between semantic compilation and symbolic execution. Suppose a set of legal or scientific documents describes ten obligations. The coding agent may infer a task-specific representation and generate SOP that makes the relevant obligations explicit. The runtime then closes bindings from reviewed mandatory matchers and can discover obligations activated by values produced in earlier rounds. The repository evaluation includes selective discovery and multi-round branching closure. The conceptual lesson is that the agent does not need to anticipate every concrete activation. It can compile the local material into values and interfaces; reviewed matchers can then drive deterministic closure. [R28]

The example should not be read as a claim that the current dynamicCircuits syntax implements every command shown. It is the stable architectural pattern suggested by the experiment. A coding agent produces a task-local representation, reviewed templates operate over that representation, execution reaches a bounded closure, and an authority gate determines what kind of result may be emitted. The use of `~` is intentional: the template instantiator and authority gate often need handles to provenance-bearing variables and branches rather than only their plain values.

There is also a direct lesson for `@newCommand` define. SOP Lang permits an implementation to define a command through a declaration whose body is code, for example JavaScript. That is

valuable during development because coding agents can extend the primitive vocabulary when existing commands cannot express a newly discovered operation. Yet `dynamicCircuits` shows why command definition and command installation must be separated. A candidate package may contain a proposed define declaration, but a normal analysis request should not be able to inject arbitrary executable code into the trusted registry. Compilation, sandboxing, tests, review, signatures or content hashes, and explicit promotion must stand between a generated command body and deployment authority.

This reconciles extensibility with the eSLM security boundary discussed next. The language can remain capable of defining new commands without implying that every KB or user request has permission to install them. The semantic object “command definition” belongs to SOP Lang; the policy “which command definitions may execute in this deployment” belongs to a trust profile. This distinction is likely to be fundamental if SOP Lang is to serve research automation, where imported literature and third-party datasets must be treated as data even when they contain text that resembles executable instructions.

The principal unknown exposed by `dynamicCircuits` is the economics of symbolic learning. Coding agents can generate SOP, but how often do those circuits generalize beyond the exact documents that motivated them? How much human review is needed before promotion? Does a stronger coding model create systematically better reusable circuits, or merely more elaborate task-local programmes? How can a benchmark distinguish legitimate domain specialization from overfitting to a test corpus? The research programme therefore needs longitudinal measures of candidate acceptance rate, review effort, transfer performance, regression incidence, library duplication, and the fraction of later tasks solved by retrieved reviewed circuits without invoking the coding agent again.

`dynamicCircuits` makes a different boundary concrete. Documents are placed in learning or task workspaces, an external coding agent proposes SOP artefacts, and a deterministic CLI compiles and executes the official task package. Trusted knowledge remains read-only during analysis, and learning proposals are not promoted automatically. [R28]

This experiment shows how neural or agentic synthesis can be used without granting it answer authority. It also exposes the cost of compilation: candidate SOP must be reviewed, tested, and generalized. The research problem is not merely generating a circuit that solves one task, but determining when a generated circuit represents a reusable capability rather than an overfit script.

9.3 eSLM: bounded language, witnesses, abstention, and authority gates

eSLM approaches the common problem from the direction suggested by its name: what would have to be true for a deterministic symbolic system to behave as a language model over a useful subset of requests? The current repository describes a symbolic runtime with declarative knowledge packages. For supported input it constructs an explicit task, executes a bounded method, and returns semantic values together with a reviewable witness. It builds query-local context, but explicitly refuses to equate context with proof. If it cannot establish an answer it abstains, and some document-style requests may receive a clearly marked PARTIAL artefact whose factual sentences remain bound to supplied text or admitted KB claims. [R29]

The most important contribution of this experiment is not another reasoning algorithm. It is the vocabulary of trust boundaries. The deployment profile is deliberately austere: trusted dependency-free Node.js plus inert data, with no network, LLM, child process, eval, or executable KB payload. The operator CLI may optionally ask an external language agent for a translation or simplification proposal, but the proposal has no answer authority and must pass the unchanged local parser and semantic gates. Training is again separate: a coding agent may propose untrusted records or changes, while validation and promotion remain host operations. These boundaries make

the system less flexible at runtime, but they allow claims about what the runtime did to be substantially stronger. [R29]

eSLM concept	Stable interpretation for SOP Lang research
Processing node	One typed responsibility. In SOP Lang this may be represented by a command or subcircuit with a stable contract rather than a vaguely named agent.
Strategy	One reviewed implementation of a responsibility. Several strategies can correspond to alternative commands or template implementations without becoming different cognitive “nodes”.
Coordination node	A bounded allocator that funds and compares candidate strategies. It is closer to a meta-command or privileged scheduler than to ordinary domain knowledge.
Authority gate	A verifier of an invariant that cannot be outvoted by confidence scores. Some gates may be privileged runtime operations even if their invocation is visible in SOP.
Immutable packet plus receipt	The cross-component boundary carrying values and evidence. SOP @ variables and ~ handles can provide the language-level identity, while receipts record execution-level facts.
Abstention and PARTIAL	First-class outcomes when proof or coverage is insufficient. The absence of an established answer is not converted into a fluent guess.

The distinction between confidence and truth is especially important for alternative command execution. A syntax such as @x commandA | commandB | commandC does not mean that three algorithms vote democratically and the majority answer becomes true. Alternative branches may generate interpretation candidates, relevance evidence, or possible witnesses. A coordinator may use confidence to allocate time or rank those candidates. Only the authority gate appropriate to the semantic claim can authorize the final status. If the claim is mathematical, this may be a proof checker. If it is a factual extraction, it may be a source-entailment and provenance validator. If it is a scientific hypothesis, the strongest legitimate status may remain “supported by these observations under this model” rather than TRUE.

The processing-graph discovery work in eSLM provides another useful methodological lesson. The repository maintains a research ledger in which dataset-derived hypotheses about processing roles do not become design authority. Alternative algorithms that appear to serve the same stable responsibility are consolidated as strategies rather than multiplied into new nodes. Promotion requires a reviewed design update, implementation, genericity controls, ablation, protected transfer, regression, and a matching receipt. Several discovery cycles over distinct data projections did not automatically justify a proliferation of nodes. Negative results of this kind are valuable because they

counter a natural tendency of agent-generated architectures to create a new named component for every observed pattern. [R29]

This is directly applicable to SOP Lang template libraries. If every newly observed task pattern creates a new top-level command, the command namespace and constructor search space will explode. The preferred approach is to identify stable typed responsibilities and allow multiple implementations behind those responsibilities. A temporal ordering command might have a cheap local strategy, a Datalog strategy, and a richer solver-backed strategy. The runtime may expose them through | or through a strategy registry, but the semantic contract remains one responsibility. This is the symbolic analogue of abstraction rather than memorization.

eSLM also demonstrates a useful distinction between evidence retrieval and reasoning authority. Query-local context can be gathered before the final route is known, but retrieved context is not itself a proof. This matters for scientific search. A literature search may discover ten papers apparently related to a hypothesis. Their existence can justify relevance and further investigation, but it does not justify the hypothesis. A scientific circuit must transform sources into explicit claims, assess methods and applicability, resolve conflicts, and connect the accepted evidence to the conclusion. This prevents a retrieval-augmented language model pattern in which cited text is treated as if citation alone established correctness.

The open issues are substantial. eSLM itself identifies broader English composition, richer first-order reasoning, document ingestion, general AND/OR planning, mature trust and conflict policy, unified multi-method verification, cross-stage scheduling, and enforceable whole-process resource controls as ongoing work. Its language strategies demonstrate that symbolic parsing and controlled approximation can cover useful families, but not that unrestricted natural language can be handled competitively without neural assistance. Its strongest contribution to the unified architecture is therefore the control plane: typed strategies, explicit work budgets, immutable receipts, abstention, and authority boundaries. [R29]

Synthesis of the three experiments. SOPLangCircuits asks how a large persistent SOP model can be activated cheaply at runtime. dynamicCircuits asks how coding agents can construct and learn SOP artefacts without becoming answer authorities. eSLM asks how language, strategies, evidence, abstention, and verification can be organized so that symbolic execution remains trustworthy. The stable architecture should combine these responsibilities rather than merge the repositories mechanically.

eSLM studies the broadest system boundary. It combines a controlled English subset, declarative packages, exact retrieval, bounded symbolic methods, query-local context, structured failure, cited partial artefacts, and strict separation among deployment, operator, and training authority. Context is explicitly not proof, and an external language agent can propose normalization without obtaining answer authority. [R29]

The prototype demonstrates that a symbolic system can produce more than yes-or-no answers while retaining source bindings for factual sentences. It does not demonstrate unrestricted language understanding. Its own staged programme identifies broader composition, richer first-order reasoning, document ingestion, general AND/OR planning, trust and conflict policy, multi-method verification, and whole-process resource controls as open work.

Taken together, the three experiments establish feasibility of important mechanisms but not convergence. SOPLangCircuits emphasizes activation over persistent SOP knowledge. dynamicCircuits emphasizes document compilation and proposal boundaries. eSLM emphasizes language, bounded strategies, evidence status, and abstention. The stable architecture should unify contracts and evaluation while allowing the experiments to continue exploring different implementation choices.

Chapter 10 - A Unified Architecture for Dynamic Symbolic Systems

10.1 Slow acquisition and compilation, fast bounded execution

The clearest convergence from the experiments is a separation between two timescales. The slow timescale is allowed to be creative, expensive, and partly neural. The fast timescale must be bounded, deterministic relative to its declared inputs and versions, and conservative about authority. This distinction is more fundamental than the choice of solver or programming language because it determines where open-ended intelligence may be used without contaminating the reproducibility of ordinary inference.

At the slow timescale, a coding agent can read a book, regulation, scientific field manual, collection of papers, benchmark failures, or an existing software implementation. It can propose entities, domain constructors, parsers, command implementations, template circuits, mappings to external solvers, regression tests, and indexes. It may also identify that the current SOP vocabulary is inadequate. In that case it can propose `@newCommand` define with a code body or generate a package for a backend such as Datalog, Z3, or a domain library. None of these actions need be deterministic in the strict runtime sense because the output is not yet trusted. The artefact is a candidate subject to compilation, testing, review, and promotion.

At the fast timescale, the runtime receives a request, a budget, a set of accessible knowledge generations, and a registry of reviewed commands, strategies, templates, and policies. It should not search the entire semantic universe. It should derive a goal representation, query compiled descriptors, select a small frontier, materialize only required knowledge, instantiate templates, execute the resulting SOP circuit, and stop when a valid terminal status is obtained or the budget is exhausted. If an optional LLM front-end is used to suggest an interpretation, that suggestion becomes just another candidate value. It cannot silently alter the reviewed registry.

Slow acquisition / compilation

Fast deterministic inference

Reads broad document collections and development corpora

Reads only request-relevant source fragments and indexed artefacts selected for the active task.

May use coding agents and powerful LLMs

Uses reviewed local strategies by default; optional external models are proposal-only unless a deployment explicitly grants a narrower authority.

May synthesize new commands and templates

May instantiate reviewed templates but does not install arbitrary generated executable code.

Optimizes for discovery and generalization

Optimizes for bounded latency, replay, explicit coverage state, and result authority.

Produces candidates, tests, indexes, and design evidence

Produces active `@` values, traces, witnesses, receipts, and a final typed outcome.

Promotion is an explicit governance step

Execution consumes only promoted artefacts from declared generations.

This architecture resembles ahead-of-time and just-in-time compilation, but the analogy is incomplete. The slow phase is not merely translating known code; it is learning representations and

procedures from under-specified language. The fast phase is not merely running a fixed binary; it dynamically constructs a request-specific circuit. Nevertheless, the compiler analogy is useful because it forces a clean artefact boundary. The output of slow learning is not “knowledge in the model” in an opaque sense. It is a versioned set of commands, descriptors, tests, source mappings, and SOP packages that a deterministic runtime can later consume.

The two-timescale model also gives a credible path toward replacing LLM calls progressively rather than ideologically. Early in a domain, the system may depend heavily on a language model to interpret documents and synthesize task circuits. As reviewed circuits accumulate, recurring operations move into the symbolic library. The percentage of runtime work delegated to opaque generative models can then be measured and, where desirable, reduced. A mature domain-specific symbolic language model may therefore emerge through compilation of repeated agentic reasoning into reusable SOP structures rather than through a single training run.

The separation must remain visible to users and evaluators. If a result was produced entirely by reviewed deterministic circuits, its receipt should say so. If a language model proposed a mapping that was then verified, the receipt should identify that boundary. If a claim depends on a model output that could not be independently checked, the final epistemic type should preserve that dependence. Trust is not obtained by hiding neural components behind symbolic syntax.

The two timescales also suggest a concrete unit of symbolic learning: the compiled capability package. A package should contain source bindings, one or more SOP templates or command definitions, a descriptor that tells the constructor when the capability may be relevant, tests that establish the scope in which it has been validated, and a promotion receipt. This is more informative than storing only executable code. The constructor needs to know not merely that a command exists, but what semantic outputs it can produce, what inputs and authorities it requires, what effects it may have, how expensive it has historically been, and which failures invalidate its use. The slow phase is responsible for compiling these descriptors; the fast phase consumes them without rediscovering the capability from source text.

Compiled capability artefact	Role at fast runtime
SOP source or template	Human- and agent-readable canonical expression of the computation or reasoning pattern.
Applicability descriptor	Small indexable summary of required input types, goal types, domains, authority requirements, and other conditions used before loading the full circuit.
Command or backend binding	Reviewed implementation identity, including effect and determinism declarations where relevant.
Validation envelope	Benchmarks, regression cases, transfer tests, known limitations, and domain assumptions that constrain legitimate reuse.
Cost profile	Observed or conservatively estimated time, memory, external calls, and completeness behavior under named work profiles.

Compiled capability artefact	Role at fast runtime
Promotion receipt	Evidence that the artefact crossed the required review boundary and identifies the exact version admitted to the trusted registry.

This packaging model reconciles automatic learning with deterministic inference. A coding agent may generate many candidate packages, and a neural model may rank or refactor them during development. None of this needs to enter the critical request path. Once a package is promoted, request-time behavior is determined by its explicit descriptor and implementation identity. If the package is later improved, it receives a new identity and future requests may bind to a new registry generation while historical receipts still point to the previous one. The resulting discipline resembles software release engineering more than continual online weight updates, which is precisely why it is attractive for scientific and regulated environments.

The slow phase should also be allowed to discover that an apparent new capability is actually a specialization of an existing one. For example, repeated scientific-review circuits may differ only in the statistical test or source ontology they use. Promotion should prefer a generic template with typed parameters when transfer tests show that the abstraction is stable. Conversely, forcing premature generalization can produce a command whose contract is so broad that the constructor cannot select it precisely. The research problem is therefore not simply to accumulate circuits, but to learn the right granularity of reusable responsibilities.

The two timescales also define a deployment strategy. Development environments may use powerful coding agents, web access, large corpora, theorem provers, and expensive experiments. Production inference can remain dependency-light, bind to immutable artefact generations, and operate within explicit work profiles. This permits aggressive learning without making every request depend on an open-ended model.

The separation is not absolute. Runtime failures can produce learning packets, and newly promoted artefacts can change future inference. The boundary is one of authority and timing: request-time execution cannot silently install what it has just generated.

10.2 Stable contracts, implementation profiles, and artefact governance

The experiments support a deliberately small stable SOP kernel. A declaration begins with `@var` followed by a command expression and an argument region. The argument region may span arbitrarily many physical lines until the next declaration begins. Arguments are initially only lexical material: atomic tokens, quoted literals, `$` references that request the value of a variable, and `~` references that request a variable or artefact handle. The command owns the interpretation of the remaining argument stream. This command-owned grammar is not a convenience detail; it is the mechanism that prevents SOP Lang from accumulating a new universal keyword for every scientific discipline or backend.

Alternative command lists separated by `|` should be treated similarly: a useful general facility whose exact evaluation contract belongs to an implementation profile until experiments establish a stable semantics. One profile may execute alternatives in parallel subject to a shared budget. Another may rank them and execute sequentially. Another may use them only as candidate implementations selected by a strategy registry. What should be stable is that alternative outcomes remain distinct and inspectable. A downstream meta-command can compare them through a handle; the runtime must not silently collapse them into a single value by majority vote.

Template circuits provide the bridge from a persistent library to a dynamic active graph. A template is stored SOP plus an applicability descriptor. When the current state satisfies the descriptor, the

constructor instantiates the template over current variables and creates new declarations. The descriptor should be compiled into indexes and be deliberately simpler than the full command grammar. Otherwise template selection would require parsing and executing every template to decide whether it might be relevant, recreating the combinatorial problem the catalog was meant to solve.

Command definition through `@newCommand` define should remain part of the language model because the programme explicitly aims to learn new computational primitives. The stable security rule, however, should be that definition is not installation. The code body is an artefact that can be parsed, statically checked, tested, sandboxed, assigned a content identity, and promoted to a trusted command registry. A deployed KB cannot smuggle executable payloads into the runtime merely because SOP can represent them. eSLM demonstrates the value of inert deployment knowledge, while `dynamicCircuits` demonstrates the candidate-review boundary. [R28][R29]

Artefact state	Meaning in the unified lifecycle
Source	Original document, theory, dataset, ontology, codebase, or manually written SOP from which candidates may be derived.
Candidate	Generated declaration, command, template, mapping, test, or package with no automatic runtime authority.
Compiled	Candidate that has passed syntactic and structural compilation and has reproducible derived indexes or executables.
Validated	Compiled artefact that passes its required unit, regression, transfer, security, and semantic checks, but may still await governance review.
Promoted	Versioned artefact explicitly admitted to a trusted registry or knowledge generation for a specified domain and authority level.
Active	Request-local instantiation or materialization of a promoted artefact as <code>@</code> variables and executable circuit nodes.
Retired	Artefact no longer selected for new requests but retained for reproducibility of historical receipts and prior results.

The lifecycle is also where provenance standards can connect. A promoted artefact should record what source entities, compilation activities, agents, tests, and revisions produced it. W3C PROV already provides a domain-independent model for entities, activities, agents, derivation, generation, and usage. SOP receipts should therefore be exportable to PROV rather than inventing an incompatible provenance universe. Conversely, SOP needs more operational detail than PROV alone provides: variable identities, command versions, branch statuses, budget consumption, and verifier outcomes. The sensible relationship is profile and mapping, not replacement. [R32]

A similar principle applies to research object packaging. RO-Crate 1.3, released in June 2026, is a current Recommendation for packaging file-based research data with machine-readable metadata

and supports provenance and workflow/script descriptions. A scientific SOP package can live inside or alongside an RO-Crate, while the crate records datasets, software, authorship, licensing, and associated publications. SOP Lang then supplies the semantic execution layer that connects claims and methods. This avoids rebuilding solved packaging and provenance infrastructure inside the circuit runtime. [R33]

The unified design therefore needs a sharper distinction between kernel semantics and implementation profiles. The kernel should be small enough that independent runtimes can agree on what a declaration, value reference, handle reference, declaration boundary, and command result mean. Profiles may then constrain the command grammar, define persistent-handle namespaces, select concurrency semantics for `|`, restrict which effects are legal, or prescribe how template descriptors are encoded. This is analogous to a language specification combined with ABIs, dialects, and deployment profiles: interoperability depends on a stable core, while experimentation remains possible above it.

Candidate for stable kernel	Candidate for implementation profile
@var declaration boundary and single-producer identity	Whether names are lexical, module-qualified, content-addressed, or mapped through package namespaces.
\$var value access	Concrete serialization, lazy materialization rules, and whether values can be remote or streamed.
~var handle/reference access	Persistent URI forms, KB export syntax, branch handles, capability handles, and security policy attached to dereferencing.
Command-owned argument region	Allowed command-specific keywords, schemas, parsers, and restrictions on multiline bodies.
Explicit output identity	Backend-specific caching, storage representation, and inter-process transport.
Failure as an explicit execution outcome	Exact status vocabulary, retry policy, completeness certificate, and mapping to domain-specific epistemic states.
Circuit composition by references	Whether <code> </code> is parallel, ranked, fallback, multi-attempt, or strategy-registry syntax in a particular runtime.

This split prevents a recurring specification failure mode: every useful experimental feature becoming permanent syntax. A peer-review command may want keywords for evidence policy, statistical recomputation, or novelty search. Those are important concepts, but they do not justify extending the SOP tokenizer. The command can parse them from its own argument region. A cryptographic profile may require annotations about public and secret values, while a scientific profile may require provenance or authority labels. Unless multiple implementations show that a concept must participate in universal name resolution or circuit composition, it should remain command or profile semantics.

Unification should therefore begin with conformance tests rather than a merger of repositories. Independent runtimes need to agree on declaration boundaries, produced values, reference

resolution, alternative identity, subcircuit interfaces, version binding, error categories, and receipt structure. Storage engines, language front ends, solver adapters, and security profiles may differ.

A governed capability package should contain source bindings, SOP templates or command definitions, applicability descriptors, interface and effect contracts, tests, validation scope, and a promotion receipt. The package tells the constructor not only how to execute a capability but when it is relevant and what its result can authorize.

10.3 The constructor as the central unresolved scientific problem

Once the two timescales and artefact lifecycle are separated, the central unresolved scientific problem becomes sharper: given a request, current variables, available knowledge generations, and a hard work budget, how can the runtime construct a sufficiently good SOP circuit without enumerating the immense space of possible circuits? Expressiveness is not the issue. A system with arbitrary commands can express almost any computation. The research contribution depends on making selection tractable and predictable.

The constructor should be understood as an indexed AND/OR search over typed capabilities. A template with three required inputs creates an AND condition: all three must be available or producible. Several templates that can produce the same goal create OR alternatives. Command lists separated by | expose an explicit local OR. Subcircuits create hierarchy. Knowledge retrieval creates latent producers that are not materialized until requested. The naive expansion of this structure is exponential. Therefore the persistent model must compile information that allows most alternatives to be rejected before their bodies are loaded.

Constructor mechanism	Open research question
Goal signatures	How expressive can goal descriptors become before matching them is as expensive as running candidate circuits?
Typed preconditions	Can types, ontology relations, and epistemic requirements eliminate most impossible compositions without excluding useful cross-domain mappings?
Capability indexes	What index structures keep candidate lookup sublinear as templates and knowledge grow from thousands to millions?
Cost and success models	Can historical receipts predict expected latency, evidence quality, and completion probability without turning the constructor into another opaque learned controller?
Anytime scheduling	How should a finite budget be divided between cheap direct attempts, deeper search, evidence acquisition, independent verification, and alternative explanations?
Novelty and fixpoint tests	What formal condition prevents repeated template activation while still allowing legitimate recursion and multi-round closure?

Constructor mechanism	Open research question
Completeness semantics	When may the system issue a negative, count, or exhaustive result, and when must it return UNKNOWN or PARTIAL because the frontier was truncated?
Canonical parallelism	How can strategies run concurrently for latency while receipts and results remain deterministic with respect to a canonical policy rather than thread-completion order?

Classical systems provide partial answers. RETE compiles shared rule conditions so updates do not rematch every rule from scratch. Datalog uses semi-naive evaluation so only newly derived facts drive the next round. Prolog tabling avoids solving identical subgoals repeatedly. Query planners use statistics to rank access paths. E-graphs compactly represent equivalent expressions and defer selection. Compiler dataflow analysis uses monotone transfer functions and fixpoints. None of these mechanisms directly solves open-ended semantic circuit synthesis, but each offers a tested way to avoid redundant work. The research programme should therefore implement these mechanisms as competing constructor strategies and measure where each is effective instead of designing a wholly novel scheduler from first principles.

A particularly promising design is a layered deterministic agenda. The first layer identifies a request family and required output type. The second consults small route indexes keyed by type, domain, relation, and goal. The third loads only candidate template descriptors. The fourth performs exact applicability checks against current variables and policies. The fifth assigns work according to deterministic cost and success estimates. Execution creates new values that are inserted into a delta set, and only descriptors indexed by the new signatures are reconsidered. This is close to semi-naive production matching over a typed dataflow graph. Expensive semantic search or neural similarity can be used to propose additional candidates, but those proposals enter the same exact gate.

The budget should be multidimensional even if the user sees one simple time limit. Wall-clock time is intuitive, but memory, external calls, branch count, loaded knowledge bytes, proof obligations, and output-construction work can each become bottlenecks. A work profile can translate a user budget such as “200 ms”, “2 s”, or “deep” into deterministic sub-budgets. eSLM already treats exact work limits as part of a strategy contract; SOPLangCircuits treats timeBudgetMs as primary. The unified constructor should make resource exhaustion a typed outcome at every level rather than an exception that disappears from the final explanation. [R27][R29]

The constructor is also where meta-rationality belongs. Meta-rationality here does not mean a vague philosophical faculty. It means that the system can reason about which reasoning procedure is appropriate, what authority that procedure has, how much work it deserves, and whether the evidence it produced justifies further expansion. ~var is a crucial language mechanism because a meta-command can inspect the identity and history of a candidate without conflating that inspection with the candidate value itself. Alternative strategies may be compared, but only declared verifiers may change epistemic status.

Success on this problem would justify the phrase symbolic language model more strongly than adding more solvers. A model is useful when it selects relevant computation automatically across diverse inputs. The persistent SOP library may contain millions of facts and thousands of reasoning patterns, but the active circuit for a question should remain small. The target metric is therefore not maximum graph size; it is how little of the persistent symbolic model must be activated to solve a request reliably.

The sketch separates routing, exact applicability, execution, delta production, and the next constructor decision. It is deliberately not a recursive language construct built into SOP Lang. `constructor_step` is an ordinary or privileged command whose implementation can be replaced by competing deterministic algorithms. One implementation may use a RETE-like discrimination network, another may use Datalog-style delta propagation, another may use a cost-based query planner, and another may use a learned ranker whose suggestions are filtered by exact constraints. All can be compared under the same SOP-level task and receipt contract.

The critical benchmark is not whether one constructor can eventually discover a solution. It is the shape of degradation as the persistent symbolic model grows. Candidate lookup should remain bounded, active-circuit size should grow much more slowly than model size, and missed routes should be observable through coverage tests. A constructor that remains fast by silently discarding difficult domains is not a success. Conversely, a constructor that preserves theoretical completeness by exploring millions of irrelevant circuits is operationally unusable. The programme therefore needs Pareto measurements relating accuracy, coverage, completeness claims, latency, memory, and activated artefact count.

Constructor research should also distinguish deterministic policy from deterministic wall-clock execution order. Parallel hardware may execute independent branches in different orders. Reproducibility does not require forcing every branch onto one thread; it requires that the allocation policy, admissible outputs, and canonical receipt do not depend on scheduler accidents. A canonical agenda can assign branch identities before workers run. Workers may finish asynchronously, while the coordinator admits outputs according to deterministic rules or records that a deadline ended the episode before a named frontier was completed. This matters especially if alternatives separated by `|` are executed concurrently.

A further open question is whether success statistics from historical receipts can improve routing without reintroducing an opaque learned controller. A deterministic runtime may use a versioned cost table derived offline from previous executions. More ambitious profiles may use a learned model to rank candidate capabilities. The safe boundary is that ranking changes search order and resource allocation, not semantic validity. If the learned ranker never proposes the correct circuit, coverage suffers; if it proposes an invalid circuit, exact type, policy, and verifier gates must still prevent an unauthorized result. This creates a clean experimental distinction between heuristic relevance and truth authority.

The programme will not become a symbolic language model merely by accumulating more commands. Model-like behaviour appears when the system can select and compose relevant computation across diverse requests. The active circuit should remain a small fraction of the persistent capability library.

Constructor quality must be evaluated jointly with coverage. A system that is fast because it ignores difficult routes is not successful. A system that preserves theoretical completeness by exploring an exponential space is not usable. The research objective is a measured frontier among latency, active-circuit size, memory, external calls, answer quality, provenance completeness, and explicit abstention.

The constructor is also the point at which meta-rational control becomes concrete. It allocates work among methods, decides whether a result deserves validation, and determines when further search is not justified by the remaining budget. These policies can be compared experimentally rather than embedded in an opaque prompt.

Chapter 11 - Experimental Programme, Falsification, and Maturity

11.1 Hypotheses, baselines, and measurements

A research programme becomes scientifically useful only when its central claims can fail. The strongest version of the SOP Lang hypothesis is not that arbitrary computation can be encoded. That is trivial once commands can execute code. The meaningful claim is that a relatively compact and reusable library of commands, templates, and knowledge declarations can solve diverse language-reasoning tasks through deterministic sparse circuit construction, with advantages in provenance, stability, reuse, or cost.

The first hypothesis concerns representability. A broad family of reasoning tasks should be expressible with a stable kernel and without continually adding global syntax. New capabilities should enter primarily as commands and templates. If every new benchmark requires modifying the language parser or adding ad hoc runtime exceptions, the command-owned grammar model is too weak.

The second hypothesis concerns reuse. Circuits learned on one set of examples should transfer to structurally similar but lexically different examples. A possession-state circuit should not be tied to the names Ana, Mihai, and Elena. A contradiction-checking circuit should generalize across documents. Reuse can be measured by the fraction of successful execution graph structure that comes from existing validated circuits rather than being synthesized anew.

The third hypothesis concerns sparse construction. As the KB and circuit library grow, the number of descriptors examined, shards loaded, and templates instantiated per request should grow slowly relative to total model size. If a model with one million templates effectively scans hundreds of thousands of them for each question, it has failed to exploit symbolic sparsity.

The fourth hypothesis concerns deterministic bounded reasoning. Given the same input, state, budget, and deterministic command outputs, the constructor should produce the same circuit and result. Increasing the budget should generally preserve already justified results while permitting stronger validation or additional alternatives. This gives the system an anytime character without making the reasoning process irreproducible.

The fifth hypothesis concerns provenance. For every accepted claim, the runtime should be able to traverse the circuit backward to source declarations, external observations, or formally justified premises. Provenance coverage is not equivalent to truth, but incomplete coverage is a clear failure for tasks that require evidence-backed answers.

The sixth hypothesis concerns error localization. When a final answer is wrong, the trace should allow the evaluator to distinguish an interpretation error from a knowledge error, a planner-selection error, a backend-inference error, or a realization error. This decomposition is a practical advantage over end-to-end models only if the categories can be measured reliably.

The seventh hypothesis concerns learning. Adding validated circuits and templates should reduce the need for expensive synthesis or neural reasoning on recurring task families without causing unacceptable regressions elsewhere. This can be measured longitudinally as the persistent library grows.

Benchmarking should begin with controlled datasets in which world states and reasoning chains are known. bAbl-style tasks are useful for possession, location, and temporal state. CLUTRR provides relational generalization. ProofWriter-style tasks provide rule-based entailment. Lightweight arithmetic and strategy tasks can test the ability to select specialized solvers. The objective is not to optimize exclusively for these datasets but to use them as diagnostic laboratories where the correct semantic state can be inspected.

The baseline set must include both symbolic and neural systems. A hand-written Prolog or Datalog solution establishes the performance ceiling for a fully formalized task and exposes the cost of manual encoding. A small LLM establishes the performance of direct neural reasoning. A tool-using LLM baseline shows what can be achieved without a dedicated SOP circuit IR. A SOP implementation with templates but no dynamic construction isolates the value of the planner. A SOP implementation with dynamic construction but no persistent learned circuits isolates the value of reuse.

Metrics must be layered. Final-answer accuracy is necessary but insufficient. Semantic parsing accuracy should be measured before reasoning. Circuit validity should be measured separately from answer generation. Provenance coverage, number of active variables, templates examined, KB shards loaded, backend calls, construction latency, execution latency, and validation latency should all be recorded. A correct answer obtained through an incorrect semantic representation should be counted as a semantic failure even if the benchmark score happens to be correct.

Budget curves are especially important. The same test should be run under several deadlines. A useful system may provide a fast candidate in a few milliseconds, an evidence-backed answer under a larger budget, and a formal check under a still larger one. The circuit trace should show what additional computation the larger budget purchased. This is more informative than one average latency figure.

Robustness testing should vary wording while preserving underlying semantics. If a minor paraphrase causes the constructor to select a completely different and weaker circuit, the system remains brittle at the language front end. The test should also vary irrelevant details, entity names, and ordering of independent facts.

Regression testing is central to learning. Every promoted command, template, or strategy should run against the full retained suite of previously solved cases. The result should record not only answer regressions but changes in circuit cost and provenance. A new strategy that preserves accuracy but doubles the number of loaded KB shards may still be a regression.

The programme should also include adversarial tasks designed to expose false confidence. Inputs with insufficient information, contradictory sources, ambiguous pronouns, or solver-incompatible constraints should reward explicit refusal or conditional answers. A symbolic language model that always returns a single conclusion has failed one of the main motivations for explicit circuits.

Research hypothesis	Primary falsification signal
Stable SOP kernel	Frequent need to add global syntax or runtime exceptions for new domains.
Circuit reuse	Most successful graphs must be synthesized from scratch even on structurally repetitive tasks.
Sparse activation	Per-request examined templates or loaded KB facts grow close to total model size.
Deterministic bounded reasoning	Identical state and budget produce unstable construction choices without declared nondeterministic commands.
Provenance completeness	Accepted claims cannot be traced to source variables or justified derivations.
Error localization	Failures cannot be classified reliably into interpretation, knowledge, planning, inference, or realization errors.
Learning through artefacts	Adding validated circuits and templates does not reduce cost or neural dependence on recurring task families.
Domain portability	The same kernel cannot support multiple domain profiles without duplicating core runtime logic.

The experimental programme should distinguish prototype evidence from architectural evidence. Existing prototypes demonstrate that SOP declarations can be parsed and executed, that \$ dependencies can form runnable circuits, that commands can be generated or wrapped, that knowledge packages can be loaded selectively, and that coding agents can create useful symbolic artefacts. They do not yet demonstrate that a large persistent symbolic model can construct the right circuit under a tight budget across diverse natural-language tasks. The latter is the main scientific claim under investigation.

The concept should therefore be presented as evolving. Syntax that has survived several prototypes deserves stronger status than a newly proposed metadata convention. A runtime optimization demonstrated on one benchmark deserves less confidence than a storage/indexing technique known to scale independently. The survey can use mature external systems as feasibility anchors without implying that their properties automatically transfer to SOP Lang. This separation between inherited evidence and unvalidated integration is central to a credible research narrative.

Research area	Current status and uncertainty
Core SOP declaration model	Prototyped and conceptually stable: @ variables, command invocations, multiline arguments, \$ value references, and command-owned parsing form the current centre of the language.
~ variable references	Prototyped as a useful meta-level mechanism; the exact immutable metadata view, security boundary, and portability contract remain open.
Alternative commands with	A strong design for explicit competing computations; scheduling, result packaging, and deterministic budget semantics need benchmark-driven consolidation.
Subcircuits and hierarchical reuse	Feasible and conventional in implementation; the research issue is how much contract and provenance can be preserved while keeping planning summaries compact.
Template circuits	Central and plausible, but activation semantics, descriptors, recursion control, and false-match rates remain major research questions.
Command definition with define	Feasible in research prototypes; safe sandboxing, effect declarations, multi-language support, and promotion policy need maturation.
SOP-based knowledge packages	Multiple representations have been prototyped or designed. The hybrid fact/declaration plus indexed store plus circuit/template model is currently the strongest hypothesis, not a final standard.

Research area	Current status and uncertainty
Deterministic dynamic constructor	The principal unresolved algorithmic problem. Candidate mechanisms are known, but large-scale tractability has to be demonstrated.
Symbolic language model claim	A long-term research hypothesis. A likely intermediate result is a family of domain-specific or hybrid models sharing one SOP runtime.

Metrics should expose the internal boundary. End-to-end answer accuracy is necessary but insufficient. The benchmark harness should record whether language interpretation was correct, whether the intended facts were retrieved, whether the constructor found an appropriate circuit, whether the backend solved the formalized problem, whether the answer-generation layer preserved the result, and whether provenance remained complete. A system that returns the right answer after a wrong formalization should not receive the same research credit as a system whose representation and inference are both correct.

Construction quality needs its own metrics. The runtime should report the number of descriptors inspected, template bodies loaded, template instances created, active variables, branch alternatives retained, cache hits, KB shards loaded, and milliseconds spent in planning versus execution. These measurements reveal whether performance is coming from genuine sparse activation or from brute-force search hidden behind a fast benchmark.

Budget curves are more informative than a single latency point. For each task family, the system can be evaluated at several budgets. A good anytime constructor should produce an early valid or explicitly partial result and improve guarantee quality when additional time is available. The curve can be compared with a small LLM whose inference cost is comparatively stable. This may reveal domains in which symbolic construction is slower initially but cheaper after reusable circuits and caches mature.

The strongest test of the learning hypothesis is longitudinal. Start with a small command and template library, allow coding agents to propose validated artefacts over successive benchmark rounds, and measure whether future tasks require less construction, fewer LLM calls, and fewer newly authored commands. If the library grows but recurring tasks remain equally dependent on neural improvisation, the proposed symbolic training mechanism has failed to accumulate reusable competence in the intended sense.

The experimental programme should add language and governance baselines to the existing reasoning baselines. A controlled-language front end can be compared with direct human-authored SOP, neural normalization followed by deterministic parsing, and direct LLM tool orchestration. The experiment should measure not only final answers but semantic representation accuracy, correction effort, source coverage, unsupported additions, and stability under paraphrase.

Constructor experiments require synthetic scaling and realistic corpora. Synthetic models can control the number of templates, overlap, depth, and distractors. Realistic corpora reveal descriptor quality, ontology mismatch, and language ambiguity. Both are necessary because an index that performs well on artificial keys may fail on scientific terminology, while a real corpus alone cannot reveal the precise source of scaling failure.

Receipts should be first-class evaluation artefacts. For each result, evaluators should be able to determine the selected capabilities, loaded knowledge, branch frontier, budget consumption,

stopping reason, validators, and authority gate. A high answer score with incomplete receipts is not evidence for the trustworthy architecture described here.

11.2 Use-case experiments and credible research outcomes

The research should mature in stages that progressively remove hidden assistance. The earliest prototype can use manually written commands and templates over controlled text. Its purpose is to validate the variable semantics, \$ / ~ distinction, trace format, alternative representation, and deterministic scheduler. A system that cannot explain its own small circuits should not be scaled to millions of concepts.

The next stage should introduce persistent KB packages and prebuilt indexes. The experiment should intentionally use more knowledge than can be loaded into memory. The objective is to measure whether goal-directed retrieval activates only the required shards and whether circuit traces remain valid after eviction and reload. This stage determines whether SOP Lang is viable as a representation for knowledge as well as computation.

A third stage should add multiple symbolic backends. Prolog or Souffle can handle relational inference, Z3 can handle constraints, and a temporal engine such as PyReason can handle evolving states. The planner must choose between these commands based on task structure and budget. The benchmark should include tasks where selecting the wrong backend is expensive or incomplete, making selection quality observable.

A fourth stage should add neural language interpretation while keeping reasoning explicit. The neural component can translate unrestricted input into candidate semantic declarations or controlled language. Competing interpretations should become SOP alternatives rather than disappearing inside one text representation. This stage will expose the grounding problem: the symbolic circuit may be perfectly correct relative to the wrong interpretation.

A fifth stage should introduce coding-agent learning of commands and templates. The agent can inspect failure traces, propose a new artefact, create tests, and run the regression suite. Promotion remains explicit. The metric is not how much code the agent can generate; it is whether persistent competence improves and whether the library remains compact enough to search.

A sixth stage should study automatic strategy learning for the construction runtime. This is the most risky component because it can reintroduce opacity at the control level. Strategy artefacts should therefore remain explicit SOP meta-circuits or similarly inspectable objects. Learned heuristics may rank descriptors, but acceptance and budget semantics should remain deterministic.

Several failure modes are plausible. Natural-language grounding may dominate the error budget so completely that symbolic reasoning contributes little outside formal domains. Template matching may remain combinatorial despite indexes. The command-owned grammar may become too unconstrained for reliable static analysis. The KB source representation may be readable but too expensive to compile at scale. Coding agents may generate many brittle circuits rather than generalizable abstractions. These outcomes should narrow the claim rather than be hidden.

A particularly credible outcome is a family of domain-specific symbolic language models. A model for scientific literature can have commands for claims, evidence, experiments, statistics, and citations. A model for regulations can have temporal and jurisdictional operators. A software-verification model can lower aggressively to SMT and proof systems. Each profile can use a domain-tuned neural parser while sharing the SOP kernel, variable semantics, scheduler, template mechanism, and persistent circuit library.

This would still represent a meaningful advance. Current LLM deployments often repeat the same reasoning patterns in generated text or agent loops. If recurring patterns can become validated SOP circuits, the system can progressively replace expensive generative search with deterministic

execution. The fraction of requests resolved without an LLM becomes a measurable indicator of symbolic maturation.

Another credible outcome is an “LLVM-like” IR for research automation rather than a complete language model. In that scenario, SOP Lang would standardize the interface between language front ends, knowledge systems, solvers, and validators. Different groups could implement commands and templates without agreeing on one logic. Even if neural models remain indispensable, the circuit IR could improve observability and reuse.

A more ambitious outcome is a genuinely persistent symbolic model whose competence is largely encoded in circuits and construction strategies. The model would resemble neither a classical expert system nor a transformer. It would maintain a large but sparsely activated library of executable semantic structures and build a small problem-specific circuit for each request. Learning would reorganize this library and improve the constructor rather than primarily adjusting billions of weights.

The research should avoid claiming that such a system will achieve general intelligence. The scientifically useful claim is narrower: explicit dynamic circuits may provide a scalable representation for a significant subset of language-mediated reasoning. The subset may begin with domains where states, relations, constraints, and provenance are important. Its boundary can be discovered experimentally.

Success should also be judged by failure quality. A system that answers fewer questions than a frontier LLM but reliably marks ambiguity, preserves evidence, and exposes where it lacks a circuit may be more useful in scientific or regulated contexts. Conversely, a system that obtains good benchmark accuracy only by delegating most reasoning to an opaque LLM has not validated the core hypothesis.

The programme is therefore best understood as an attempt to mature a computational substrate. SOP Lang already supplies a compact syntax for declaring values and commands. The research question is whether strong semantics, scalable storage, deterministic construction, and learned reusable circuits can turn that syntax into a new class of symbolic language models. The state of the art makes the individual ingredients plausible. The experiments must determine whether their integration remains tractable.

Maturity stage	Principal research objective
Kernel prototype	Validate declaration boundaries, \$ / ~, SSA-like values, traces, alternatives, and deterministic scheduling on controlled tasks.
Persistent KB	Demonstrate indexed lazy loading and eviction with knowledge far larger than active memory.
Multi-backend reasoning	Demonstrate deterministic selection among relational, temporal, SMT, and other symbolic commands.
Neural front end	Preserve multiple semantic interpretations and measure grounding errors separately from symbolic inference.
Artefact learning	Use coding agents to propose commands, circuits, and templates while promotion remains benchmark- and regression-gated.
Strategy learning	Improve construction policies through explicit inspectable strategies while retaining deterministic budget semantics.
Domain models	Evaluate whether specialized symbolic language models share a common SOP runtime while differing in ontologies, commands, KBs, and neural parsers.

The roadmap should mature the system from representation to selection rather than attempting open-domain language first. The earliest experiments should freeze a small command vocabulary and ask whether SOP variables, alternatives, traces, and template instantiation behave deterministically and are easy to debug. The next experiments should deliberately make the

persistent KB much larger than memory so that indexing, sharding, handles, and eviction are stressed before semantic complexity is increased. Only after these layers are stable should the constructor be evaluated against thousands of reusable templates and increasingly free natural language.

A second strand should investigate representation pressure. Every time a prototype requires a new global keyword or an exception inside the kernel, the team should ask whether the requirement can instead be expressed as a command grammar, a descriptor field, a template convention, or a library circuit. Conversely, if the same semantic concept repeatedly appears across unrelated commands and cannot be analyzed without command-specific parsing, that is evidence that the kernel contract may be too weak. The language should evolve from repeated empirical pressure, not from attempts to predict every future feature.

A third strand should compare deterministic constructors. A simple indexed best-first planner should establish a baseline. More sophisticated AND/OR or cost-bounded planners can then be evaluated on exactly the same persistent model and budgets. Learned ranking can be introduced only after deterministic structural baselines are understood. This ordering matters because otherwise improved benchmark results may come from a neural selector whose behaviour hides whether SOP's explicit structure contributed to tractability.

A fourth strand should study domain specialization. It is entirely plausible that general natural-language interpretation remains neural while the symbolic core becomes increasingly useful after the domain is known. Medicine, regulation, scientific literature analysis, software verification, and security reasoning have different ontologies and solver requirements. The question is not whether they can share one complete KB, but whether they can share SOP syntax, circuit construction principles, runtime tracing, storage organization, and a substantial library of generic reasoning subcircuits.

Possible research outcome	Scientific interpretation
General symbolic language model succeeds	Sparse deterministic construction and reusable circuits scale across diverse language tasks with decreasing neural dependence.
Domain-specific symbolic models succeed	The common SOP runtime is general, while ontologies, neural parsers, templates, and commands specialize by domain. This remains a significant architectural result.
Hybrid orchestration IR succeeds	SOP becomes an auditable LLVM-like layer for heterogeneous symbolic/neural reasoning even if most language interpretation remains neural.
Only restricted formal domains succeed	The circuit model is valuable for executable science, verification, policy, or structured reasoning but not a broad language model.
Dynamic construction does not scale	Template selection or semantic matching remains too expensive as libraries grow; this falsifies the central scaling hypothesis and motivates more static or learned routing.

Possible research outcome	Scientific interpretation
SOP kernel proves too weak or too permissive	Repeated special cases or opaque commands show that the minimal language does not expose enough semantics for reliable optimization and verification.
Neural dependence does not decline	Symbolic artefact learning fails to replace recurring neural reasoning, limiting the model to a transparent orchestration layer rather than a symbolic alternative to LLM computation.

The document should therefore avoid presenting the present syntax as a completed language standard. The better framing is a maturing research substrate. Several prototypes provide enough experience to make the central abstractions concrete, and the reduction toward SOP Lang has already removed constructs that were less coherent in earlier designs. The next phase is not primarily syntax design. It is the empirical maturation of descriptors, template activation, knowledge organization, circuit learning, and deterministic construction under realistic resource constraints.

The criterion for success is not that a SOP programme looks elegant. The criterion is that explicit representation creates measurable advantages. Circuits should transfer between tasks. New knowledge should be activated sparsely. Provenance should survive backend boundaries. Failures should be localized. A larger persistent model should not imply proportional per-request search. Additional time should buy stronger verification in a predictable way. Coding-agent learning should create persistent artefacts that reduce future cost. These properties, if demonstrated together, would justify treating SOP Lang as more than a workflow notation and would support the stronger claim that it can become a computational foundation for symbolic language models.

The roadmap now has concrete experimental evidence behind it. The three repositories discussed in the next chapter should be interpreted as controlled architectural probes. SOPLangCircuits stresses persistent knowledge activation and runtime search; dynamicCircuits stresses the document-to-SOP learning workflow and the separation of coding-agent proposals from executor-owned results; eSLM stresses language handling, trust boundaries, finite strategies, abstention, and independent authority. None by itself is the target architecture. Their value is that they expose different failure modes early enough for the common model to evolve before its interfaces become rigid. [R27][R28][R29]

Accordingly, the research programme should resist premature convergence at the implementation level while seeking convergence at the contract level. The strongest candidates for stabilization are the SOP declaration model, single-producer value identities, the distinction between value access and variable or artefact handles, command-owned parsing, explicit resource budgets, versioned knowledge, replayable receipts, and a promotion boundary between generated candidates and trusted runtime artefacts. The exact scheduler, template descriptor language, branch semantics, knowledge indexes, command ABI, and boundary between privileged runtime gates and ordinary SOP commands remain open research questions.

Peer-review experiments should use papers with known computational and reporting faults, live-review assistance studies, and retrospective cases in which later corrections or retractions change the status of claims. The system should be evaluated as an assistant and verification substrate, not only as a review-text generator.

Literary-analysis experiments should compare theory-bound interpretations, source grounding, preservation of ambiguity, and sensitivity to theory changes. Expert evaluation is necessary, but it can be made more reliable by judging specific links among textual observations, theoretical operations, and claims rather than assigning one holistic score to an essay.

Specification experiments should compare direct LLM drafting, coding-agent SOP synthesis, synthesis constrained by reviewed theories and ontologies, and expert-authored specifications. Metrics should include source fidelity, exception coverage, executable-test yield, correction effort, and transfer to unseen documents.

Agentic replacement experiments should instrument an existing system, classify model calls by role, and replace one role at a time. This permits causal evidence about whether a SOP capability improves reliability or cost. It also reveals which roles resist symbolic compilation.

The credible outcomes form a spectrum. The strongest outcome would be a broadly capable symbolic language model that learns reusable circuits and resolves many ordinary requests with little runtime neural assistance. A more probable and still important outcome is a family of domain-specific symbolic models with shared infrastructure. Another valuable outcome is an IR and governance layer that makes hybrid agents more inspectable even when neural components remain dominant. A negative result showing that constructor search or semantic maintenance does not scale would also advance knowledge by defining the limits of explicit symbolic execution.

Research question	Evidence required for a credible answer
Can circuits transfer?	Performance on unseen tasks and domains with the same promoted capability, not merely replay of generated examples.
Can construction scale?	Sublinear or bounded candidate activation as the persistent model grows, with observable coverage and failure reasons.
Can neural authority be reduced?	Role-by-role replacement studies showing which model calls become deterministic or verifiable without unacceptable loss.
Does provenance improve trust?	User and audit studies showing that receipts support error localization, challenge, replay, and governance decisions.
Can executable science add value beyond workflows?	Claim-level reevaluation, theory substitution, incremental review, and dependency-sensitive correction that conventional reruns do not provide.
Can the system abstain appropriately?	Calibration of explicit unknown, unsupported, ambiguous, and truncated statuses against controlled ground truth.

Chapter 12 - Toward Trustworthy Executable Science

12.1 Systematic scientific search with explicit evidence boundaries

The surrounding research landscape has moved rapidly toward autonomous scientific agents. The AI Scientist-v2 describes an end-to-end agentic system that formulates hypotheses, designs and runs experiments, analyzes results, and writes manuscripts; its authors report that one fully AI-generated manuscript crossed the acceptance threshold at an ICLR workshop. FutureHouse publicly frames its mission as automating scientific discovery and provides specialized scientific agents with access to literature and tools. Kosmos describes long-running cycles of literature search, data analysis, and hypothesis generation coordinated through a structured world model. These systems show that the question is no longer whether language-model agents can perform meaningful portions of the scientific workflow. The more difficult question is how much authority their outputs should have and how their reasoning can be audited. [R36][R37][R38]

An independent 2025 evaluation of three KOSMOS-generated hypotheses in radiation biology illustrates why traceability alone is not enough. The evaluators reported one well-supported result, one plausible but uncertain result, and one false hypothesis after testing against simple null baselines. This is only a small case study and cannot characterize KOSMOS as a whole, but it exposes the exact distinction that matters for this programme: a system may provide traceable citations and executable analyses while still selecting weak hypotheses, inappropriate comparisons, or insufficient validation. Trust therefore requires explicit adversarial checks, null models, independent verification circuits, and a status that can remain UNKNOWN or PARTIAL after an apparently coherent discovery path. [R39]

Systematic scientific search should therefore be represented as a succession of epistemic transformations rather than as one agent transcript. Literature discovery produces candidate sources, not accepted premises. Claim extraction produces propositions bound to source spans, not facts. Applicability analysis determines whether a result from one population, instrument, material, or experimental regime bears on the current hypothesis. Conflict analysis preserves incompatible findings. Hypothesis generation proposes relations that may deserve tests. Experimental or computational validation produces new evidence. Only then can a claim move to a stronger status under a declared authority policy.

Long-horizon discovery also requires memory that is not equivalent to a conversational summary. The persistent state should separate observations, imported claims, generated hypotheses, failed tests, method artefacts, and promoted knowledge. Failed hypotheses are scientifically useful because they constrain future search, but they must not be confused with facts. A dynamic circuit can materialize the small part of this research memory relevant to a new objective. The slow acquisition phase can periodically consolidate duplicate hypotheses, derive reusable method circuits, and propose promotions without rewriting historical receipts.

Trustworthy search also benefits from explicit stopping criteria. A research agent can stop because it found a constructive witness, because a predefined evidence threshold was met, because no untested high-priority alternatives remain inside the bounded frontier, or because the budget was exhausted. These are scientifically different outcomes. The final report should not hide them behind a generic statement that the agent “finished researching”. The status of the search frontier is part of the evidence for any claim about novelty, absence of prior work, or robustness.

The SOP programme differentiates itself by optimizing for controllability rather than maximum autonomy. A scientific agent may search broadly and propose hundreds of hypotheses, but a hypothesis enters the trusted knowledge base only through an explicit lifecycle. Literature search produces source objects. Extraction produces claims with spans. Synthesis produces candidate hypotheses. Analysis circuits produce witnesses. Independent validators test the witness against

data and policy. A human or declared authority gate promotes the result to a stronger status. The system may automate most of these steps, but it cannot silently grant itself epistemic authority.

Systematic scientific search can also become more trustworthy when the search state is explicit. Rather than asking an LLM to “think of alternatives” until it seems satisfied, the constructor can define a finite frontier of mechanisms, datasets, parameter regions, counterexamples, or literature clusters and record which regions were explored, pruned, or left untouched due to budget. This does not make open-ended science complete, but it allows a researcher to distinguish “the agent found no counterexample after these declared searches” from “the agent says there is no counterexample”. The difference is epistemically decisive.

Dynamic circuits are especially suitable for research because scientific work is inherently iterative. A failed experiment creates new values that may activate diagnostic templates. A surprising correlation may instantiate confounder checks. A contradiction between papers may activate source-quality and population-difference circuits. A solver failure may trigger a weaker approximation. Rather than encoding one giant workflow for “do science”, the system grows a request-specific SOP circuit as evidence accumulates. The research challenge is to keep this growth bounded and to preserve a receipt for every decision that changed the search frontier.

There is also a social dimension. Scientific results gain authority through communities, replication, peer review, and continued challenge, not through a single internal proof. The SOP architecture should therefore support multiple authorities and versioned acceptance profiles. A laboratory may mark an observation as locally verified. A journal may mark an artifact as reproducible. A meta-analysis may assign a different evidential status after integrating new studies. The same underlying claim can accumulate or lose authority without its historical versions being erased.

This is where the peer-review, specification, and symbolic-language-model use cases converge. A research agent reads literature as source data, compiles relevant theories and protocols into SOP, constructs experiments or analyses, executes the formalizable parts, and returns candidate scientific results. Peer-review circuits then inspect the same structured objects. Because both discovery and review operate over shared semantic artefacts, the review is not an unrelated second LLM opinion. It can reexecute the exact dependency graph and challenge individual assumptions.

Systematic scientific search becomes trustworthy when the search state is an object. The system records what corpora and time ranges were searched, what queries and expansions were used, what sources were admitted, what alternatives were pruned, and what frontier remained when the budget ended. “No evidence found” is then a statement about a documented search, not a universal negative.

Hypothesis generation should operate over evidence-bearing claims and explicit gaps. A hypothesis can cite the observations, analogies, or theory constraints that motivated it while remaining clearly marked as a candidate. Test design can produce computational procedures, formal checks, or protocols for physical experiments. The resulting evidence changes the hypothesis status through declared gates rather than through self-assessment by the generator.

Scientific memory should preserve failures. A rejected hypothesis, an inconclusive test, or a solver counterexample can constrain future search. These records must remain distinct from accepted knowledge. Dynamic activation permits a new question to materialize only the relevant history.

The architecture also supports multiple authorities. A laboratory can verify an observation locally. A journal can certify artifact reproducibility. A formal checker can validate a theorem. A meta-analysis can revise evidential strength. The same claim identity can accumulate versioned statuses without pretending that one machine score replaces scientific institutions.

12.2 Research position in August 2026

The programme belongs naturally to trustworthy AI, but the phrase should be used with precision. NIST describes trustworthy AI through characteristics including validity and reliability, safety, security and resilience, accountability and transparency, explainability and interpretability, privacy enhancement, and fairness with harmful bias managed. SOP Lang circuits directly address only part of this space. Explicit values, receipts, versioned knowledge, deterministic replay, abstention, and authority gates can contribute strongly to validity, reliability, accountability, transparency, explainability, security policy enforcement, and privacy-aware dataflow. They do not automatically make a system fair, safe, or legally compliant. Those properties require domain-specific models, tests, governance, and socio-technical analysis. [R30]

Trustworthy-AI property	Potential SOP contribution and limit
Validity and reliability	Typed commands, independent verifiers, explicit completion status, regression suites, and replayable receipts can make failures observable. They cannot correct a false source or an invalid scientific theory automatically.
Accountability and transparency	Every authoritative result can identify the circuit, knowledge generations, command versions, external proposals, and promotion decisions involved. Accountability still depends on organizational ownership and governance.
Explainability and interpretability	Def-use and provenance paths provide faithful structural explanations of which values produced a result. An opaque neural node remains opaque internally and must be labelled accordingly.
Security and resilience	Inert KB packages, trusted registries, declared effects, isolation of generated commands, and handle-level policies reduce attack surface. They do not eliminate vulnerabilities in command implementations or external dependencies.
Privacy	Values and handles can carry sensitivity labels and restrict which commands or execution locations may consume them. Strong privacy may additionally require MPC, TEEs, differential privacy, or other mechanisms.
Fairness and harmful bias	Circuits can make fairness tests and assumptions explicit, but fairness is not a syntactic property of SOP Lang and must be defined in context.

The regulatory context makes these properties increasingly relevant. As of August 2026, the European Commission states that the majority of AI Act rules are in application and Article 50 transparency obligations apply from 2 August 2026. The SOP programme should not claim automatic

AI Act compliance: legal applicability depends on the system and use case. Nevertheless, machine-readable provenance, explicit disclosure of external model participation, logging, versioned policies, and reproducible result construction are technically aligned with the broader demand for more transparent and governable AI systems. [R31]

The current research position can now be stated more sharply than in the earlier survey. The project has multiple prototypes, each implementing meaningful slices, but the concept is still evolving. The evidence supports feasibility of SOP parsing and execution, persistent catalogs, task-local circuit construction, document-to-SOP synthesis with coding agents, controlled-language symbolic reasoning, bounded methods, receipts, candidate promotion boundaries, abstention, and strategy registries. It does not yet support the claim that one deterministic constructor can select near-optimal circuits from a language-model-scale symbolic library, that unrestricted English can be handled without substantial neural support, or that automatically learned circuits reliably transfer across broad scientific domains.

Status in August 2026	Research position
Demonstrated in prototypes	Executable SOP subsets; persistent indexed knowledge; bounded branch execution; task-local circuit composition; coding-agent generation of candidate/task SOP; executor-owned results; controlled-English symbolic reasoning; explicit abstention and receipts.
Plausible but only partially demonstrated	Hierarchical reusable circuit libraries; multi-round template activation; domain-specific symbolic language models; progressive replacement of recurring LLM reasoning with reviewed SOP; large knowledge bases with lazy materialization.
Central open research	Deterministic circuit construction under hard budget at million-to-billion artefact scale; stable semantics for alternatives and meta-references across profiles; automatic transfer and generalization of learned circuits; multi-method verification.
Long-term hypothesis	Trustworthy automation of substantial portions of scientific research through semantically executable research objects and agentic search whose claims, evidence, computation, and authority remain machine-auditable.

The next maturity step should therefore be unification by contracts rather than a merger of codebases. The SOP parser and core reference semantics need a test suite shared across experiments. Command ABI and metadata should distinguish values, handles, effects, determinism, cost, and authority. Catalog and cache interfaces should support multiple storage engines. Strategy and authority-gate contracts should be formalized. Candidate packages should have one promotion lifecycle. Receipts should map to an interoperable provenance model. Benchmarks should exercise the same tasks through pure symbolic, hybrid, and LLM-heavy profiles so the programme can measure what is actually gained by explicit circuits.

The trustworthy-AI contribution should be stated as a set of mechanisms rather than a global claim. SOP can provide faithful structural provenance because the producer of each @ value and its consumed \$ or ~ dependencies are explicit. It can provide controllable resource use because budgets and exhaustion are execution objects. It can provide stronger separation of duties because proposal, execution, verification, and promotion can be assigned to different commands or host authorities. It can improve security by keeping runtime KB packages inert and preventing generated define bodies from acquiring deployment authority automatically. It can support privacy engineering by carrying labels on values and constraining where handles may be dereferenced. None of these mechanisms establishes fairness, scientific validity, or legal compliance by itself.

This qualification aligns with contemporary trustworthy-AI frameworks. NIST treats trustworthiness as a family of properties that must be balanced in context, not as a synonym for explainability. The EU AI Act adds legal duties whose applicability depends on the system and role; the current transparency requirements reinforce the practical importance of knowing when users interact with AI and when generative components participate in content production. SOP receipts could make such participation easier to disclose because external model calls are named activities in the circuit. The research programme should nevertheless treat standards and law as external requirements to map to, not as properties automatically satisfied by adopting SOP Lang. [R30][R31]

A mature evaluation should therefore combine computational metrics with governance tests. Red-team inputs can attempt to inject command-like text through documents and KB records. Privacy tests can check whether restricted values ever reach forbidden backends. Provenance tests can remove or alter one source and verify that affected outputs are invalidated. Authority tests can ensure that a high-confidence proposal cannot bypass a failed verifier. Reproducibility tests can replay historical receipts against exact artefact generations. Fairness and domain-safety tests must be defined by the application. The common benefit is that failures can be attached to explicit components and contracts rather than diagnosed only from final text.

The most credible near-term contribution is therefore infrastructural. The programme can provide a language and runtime in which trust-relevant distinctions are machine-enforceable: candidate versus promoted artefact, source versus derived value, value versus handle, retrieval evidence versus proof, confidence versus authority, completed versus truncated search, deterministic result versus generative proposal, and current knowledge generation versus historical receipt. These distinctions already appear independently in the three experiments. Stabilizing them across one SOP architecture is a concrete research output even before the system reaches language-model-scale coverage.

For science, the critical benchmark should eventually move beyond toy reasoning. A peer-review corpus can contain papers with known statistical errors, missing artifacts, contradictory claims, citation mismatches, and intentionally ambiguous conclusions. A specification corpus can contain theories and regulations with gold source mappings and exceptions. A literary corpus can test plural interpretations where source grounding matters more than a single gold answer. An agentic research benchmark can ask the system to search a bounded literature and dataset universe and report not only hypotheses but the exact frontier explored and evidence supporting each claim. These use cases collectively test deduction, uncertainty, interpretation, provenance, and resource-bounded search.

The final ambition is an infrastructure in which scientific automation is not trusted because the generating model sounds competent. It is trusted to the degree that relevant operations are explicit, evidence is addressable, transformations are replayable, uncertainty is preserved, and authority is assigned deliberately. SOP Lang is valuable if it can become the compact language in which those responsibilities compose. Dynamic circuits are valuable if they allow the system to construct only the reasoning needed for the current scientific question. Coding agents are valuable if they can extend the library without silently extending their own authority.

Long-term research thesis. A trustworthy AI scientist should not be a single autonomous language model. It should be a governed ecosystem that can use models for discovery and synthesis, compile recurring competence into SOP Lang, construct bounded dynamic circuits for each task, attach evidence and receipts to every authoritative result, abstain when coverage is insufficient, and connect scientific claims to executable or inspectable research objects. The objective is not maximum autonomy; it is scalable scientific agency with explicit epistemic control.

The external landscape in August 2026 reinforces both the opportunity and the caution. Nature publications demonstrate end-to-end and multi-agent scientific automation, while a contemporary position paper argues that current agentic scientists are better understood as co-scientists than as systems built for autonomous discovery. Major conferences are running live experiments with AI-assisted reviewing while explicitly retaining human judgment. These developments validate the importance of the problem without validating a particular solution. [R54][R55][R56][R57][R58]

The state of the SOP programme can be summarized without either understatement or inflation. Several mechanisms are implemented across prototypes: SOP parsing and execution, explicit values and dependencies, persistent catalogues, lazy materialization, bounded methods, task-local generation, candidate promotion boundaries, structured failures, provenance receipts, and abstention. The exact common kernel, broad language coverage, large-scale constructor behaviour, multi-method verification, ontology evolution, and cross-domain transfer are not yet established.

The next research milestone is a shared experimental contract. A common corpus should be executable by the three profiles where their scopes overlap. Receipts should use compatible identities and status categories. Constructor benchmarks should compare strategies on the same persistent model. Candidate artefacts should pass one lifecycle. The aim is not premature standardization but evidence about which semantics deserve to become stable.

The contribution to trustworthy AI is the creation of enforceable distinctions and replayable processes. This is narrower than claiming that the system is trustworthy, and stronger than merely asking a model to explain itself. It provides a substrate on which organizations can implement risk controls, audit, disclosure, human oversight, and domain validation.

The programme should remain open to a hybrid conclusion. Symbolic circuits may absorb stable reasoning while neural models remain essential for interpretation and scientific creativity. The achievement would be the ability to locate that boundary explicitly, move it through validated learning, and prevent the neural component from acquiring authority simply because it produced fluent language.

Established by August 2026	Still open
Readable SOP declarations can be parsed and executed as explicit dependencies in several prototypes.	A final cross-implementation SOP kernel and conformance suite.
Persistent catalogues and lazy loading can avoid scanning every source at startup.	Activation behaviour and storage performance at language-model-scale knowledge.
Coding agents can propose task-specific and knowledge SOP while a host retains promotion authority.	Reliable automatic generalization from one-off generated circuits to reusable capabilities.

Established by August 2026

Bounded symbolic methods can produce witnesses, structured failures, and abstentions.

Scientific and document-analysis use cases can be represented as explicit processing graphs.

Trust-relevant roles can be separated in architecture and receipts.

Still open

Broad natural-language coverage and robust integration of multiple formal methods.

Evidence that these representations outperform simpler workflows or hybrid LLM agents on realistic end-to-end tasks.

Organizational, legal, social, and empirical validation of trustworthy deployment.

Conclusion - A Research Instrument Rather Than an Autonomous Oracle

The programme developed in this book does not begin from the belief that symbolic AI is intrinsically trustworthy or that natural language can be made precise by adding a few markers. It begins from a more practical observation: many failures of current agentic systems are failures of hidden structure and concentrated authority. The model interprets, plans, retrieves, judges, and narrates in one opaque loop.

Dynamic SOP Lang circuits attempt to externalize a useful part of that structure. The language provides a compact human-inspectable representation. Commands permit domain extension without forcing one universal grammar or logic. Values and dependencies support provenance, incremental execution, and validation. Templates and subcircuits support reusable competence. Resource budgets and explicit stopping states make bounded reasoning visible. Candidate and promotion boundaries allow powerful synthesis methods to contribute without becoming authorities automatically.

The research remains difficult because the syntax is the easy part. The constructor must find a relevant circuit inside a very large persistent model. The language interface must preserve ambiguity rather than fabricate precision. Knowledge must evolve without destroying historical reproducibility. Formal solvers must be used without confusing proof of a formalization with truth about the world. Scientific communities must retain control over judgments that cannot be delegated responsibly.

The three prototypes establish that the programme is implementable in pieces. They do not establish that the pieces will scale or converge. That uncertainty is the reason for the programme, not a reason to abandon it. A mature result may be a set of domain-specific symbolic language models, a hybrid agent infrastructure, an executable research-object layer, or a clearer account of where symbolic compilation ceases to be economical.

The final vision is a research environment in which language models can assist discovery, but scientific claims acquire authority only through explicit evidence, methods, validation, and governance. Articles remain readable narratives while gaining executable companions. Peer review becomes partly continuous and replayable. Theories can generate specifications and tests. Agents can search systematically without hiding unvisited alternatives. Recurrent competence can be compiled from prompts into persistent circuits.

Such a system should be approached as a scientific instrument. Instruments extend observation and calculation, but they require calibration, documented limits, maintenance, and interpretation. SOP Lang dynamic circuits have potential if they make those requirements part of the executable system rather than an afterthought in its documentation.

Final position. The objective is not maximum autonomy. It is increasing scientific capability while keeping evidence, computation, uncertainty, and authority separable enough to inspect and govern.

Appendix A - SOP Lang Reference Model

A.1 Declarations, commands, values, and references

The research programme deliberately keeps the globally reserved syntax of SOP Lang small. A declaration begins with an output variable and a command expression. The remainder of the declaration is an ordered argument block that may continue over as many physical lines as needed. The block ends when the parser encounters the next declaration beginning with `@`. This makes line breaks primarily a readability mechanism rather than a semantic boundary inside an argument list.

```
@result command
  atomic-token
  "quoted literal containing spaces"
  $previousValue
  ~previousVariable
  keyword "value understood by this command"

@next otherCommand
  $result
```

The SOP Lang core need not understand the complete grammar after command. At the general level it must preserve declaration boundaries, atomic tokens, quoted literals, and the special variable-reference forms. The command implementation receives the argument representation and is responsible for interpreting it according to its own contract. A restricted runtime may choose a stricter profile, but this should be understood as an implementation constraint rather than a reason to continuously extend the global SOP Lang grammar.

This separation is important because commands may represent very different computational domains. A Datalog command may require relation names and a query form. A theorem-proving command may accept hypotheses followed by a goal. A retrieval command may use command-local keywords for source scope, ranking, or freshness. A command that defines another command may interpret subsequent lines as source code. Requiring every such convention to become global syntax would gradually turn SOP Lang into an aggregation of unrelated language features. Treating command-specific forms as command-owned grammars preserves a small kernel while allowing domain-specific expressiveness.

SOP Lang construct	Operational meaning
<code>@name command ...</code>	Creates a runtime variable named <code>name</code> whose producer is this command instance.
<code>\$name</code>	Requests the semantic value currently associated with variable <code>name</code> . It therefore induces a value dependency on the producer of <code>name</code> .
<code>~name</code>	Passes a reference or handle to the runtime variable itself rather than immediately dereferencing its semantic value. This enables meta-reasoning about alternatives, provenance, state, or variable identity.
Quoted literal	Passes a literal whose whitespace and textual content are preserved by the SOP tokenizer.
Atomic token	Passes a command-interpreted token. Its meaning belongs to the command contract unless reserved by the implementation profile.
Multiple physical lines	Continue the argument block of the same declaration until another <code>@...</code> declaration begins.
Command-owned keyword	A token such as <code>require</code> , <code>source</code> , or <code>policy</code> that has meaning to the selected command but does not need to be a globally reserved SOP Lang keyword.

The distinction between `$name` and `~name` deserves special care. `$name` is a dataflow operation: the consumer asks for the value produced elsewhere in the circuit. `~name` is a meta-level operation: the consumer asks for the variable object or reference. A validator can therefore inspect whether the variable contains alternatives, where it originated, whether it is resolved, what provenance it carries, or which

epoch produced it. Passing `~name` should not implicitly authorize arbitrary mutation. A rigorous runtime should separate a reference that permits inspection from explicit commands that create a new version or otherwise modify system state.

This distinction gives SOP Lang a compact way to represent both ordinary computation and reasoning about the computation. A meta-rational command can inspect a result as an epistemic object rather than being forced to consume only its payload.

Some implementation profiles may permit the command expression itself to contain alternatives separated by `|`:

```
@candidate fastReason | temporalReason | formalReason
  $question
  $context
```

The declaration should not be interpreted merely as textual fallback syntax. It can denote an explicit set of candidate computations associated with the same conceptual output variable. The runtime may execute alternatives sequentially or in parallel, depending on policy and available resources, and may preserve their results rather than immediately collapsing them into a single value. Later commands can receive `~candidate`, inspect the branch structure, compare provenance or cost, and select or aggregate according to their own semantics.

For example:

```
@candidate fastReason | formalReason
  $question
  $context

@quality evaluateCandidates
  ~candidate
  require "provenance"
  require "consistency"

@chosen selectCandidate
  $quality

@answer formulateAnswer
  $chosen
```

This pattern makes alternatives first-class enough to support meta-reasoning without requiring a new globally defined branch syntax. The precise semantics of parallel execution, cancellation, failure, and aggregation remain part of the runtime profile and the relevant commands.

Commands themselves can be introduced using the same declaration mechanism. A general `define` command may own a mini-language for formal parameters and implementation code. For a JavaScript-oriented profile, a definition could look conceptually like this:

```
@newCommand define first second
  const a = Number(first);
  const b = Number(second);
  return a + b;

@sum newCommand
  "12"
  "30"
```

The formal parameters and JavaScript body are not additional universal SOP syntax. They are arguments interpreted by `define`. Another implementation could provide a `define` command that consumes WebAssembly, a safe expression language, a Prolog body, a remote-tool descriptor, or a formally verified implementation. The common SOP Lang layer only needs to know that `@newCommand` is a variable

produced by the define command and can later be resolved as a callable command according to the runtime's command registry.

This approach also permits an important security boundary. A production runtime need not permit unrestricted define. It may accept only signed command packages, sandboxed JavaScript, capability-restricted functions, or previously verified implementations. The language model remains the same while deployment profiles constrain what commands are admissible.

A.2 Circuits, subcircuits, templates, alternatives, and epochs

A SOP Lang circuit is the dependency structure induced by a set of declarations. No separate edge syntax is required. If one declaration contains `$x`, it depends on the value associated with `x`. If it contains `~x`, it depends on the existence and identity of `x` at the meta-level, although whether it also forces evaluation of `x` depends on the command and runtime contract.

Consider the following circuit:

```
@inputText input
@question input

@events extractEvents
    $inputText

@resolved resolveEntities
    $events

@timeline orderEvents
    $resolved

@state computeState
    $timeline

@answer answerQuestion
    $state
    $question
```

The textual order is convenient for reading but is not the primary definition of the execution order. The data dependencies are. If two declarations consume only `$inputText`, they may be evaluated independently. A declaration that needs both outputs must wait until both values are available. The runtime can therefore derive a topological schedule for an acyclic portion of the circuit and can expose parallelism without requiring the author to specify threads or callbacks.

The runtime variable is more than a conventional mutable variable. It has an identity, a producer, and potentially metadata describing its type, provenance, temporal validity, security label, confidence, alternative branches, and execution epoch. The semantic payload is only one part of the runtime object. `$x` normally retrieves the payload according to the command contract; `~x` allows a command to operate on the richer object.

Persistent knowledge identifiers and transient runtime variables should not be conflated. A KB may contain a persistent declaration whose concept identifier survives many sessions, while a task circuit may create temporary variables. A mature implementation should version and namespace these identities so that caching, provenance, and invalidation remain correct.

A subcircuit is a circuit exposed through a stable command contract. Suppose a temporal-reasoning circuit internally extracts events, resolves references, orders them, applies state transitions, and answers a query. Other circuits should not need to reproduce that internal structure. The runtime can package the subcircuit as a callable command whose externally relevant contract describes accepted inputs, produced outputs, effects, cost characteristics, and guarantees.

Conceptually:

```
@temporalAnswer temporalReasoning
  $document
  $question
```

may expand internally into a larger SOP Lang fragment. The outer circuit sees a command instance. Inspection, debugging, provenance, or optimization may still open the subcircuit and examine its internal declarations. This hierarchical composition is essential for scaling the library of learned reasoning procedures.

Template circuits are parameterized SOP Lang fragments equipped with matching conditions. They are not necessarily instantiated when the runtime starts. Instead, the dynamic circuit constructor can discover that the current state satisfies a template's activation conditions and instantiate its declarations with bindings from that state. The newly introduced declarations may generate new variables, which in turn can activate additional templates.

A useful abstract template might state that whenever the current task contains a temporal question and a set of extracted events, a temporal-ordering fragment becomes applicable. The implementation need not add a special `template` keyword to global SOP syntax. Templates can be represented as metadata around ordinary SOP fragments, as entries in a template index, or through commands whose argument grammar describes matching conditions. The critical research requirement is that template applicability and instantiation have precise semantics and can be indexed without scanning the entire library.

This mechanism resembles several established ideas without being identical to any one of them. Logic-programming rules become applicable when a goal or facts unify. Production systems activate rules whose conditions match working memory. Partial-evaluation systems specialize a program under known inputs. Macro systems instantiate parameterized syntax. SOP template circuits borrow the idea of conditional instantiation but produce explicit runtime declarations in the same circuit IR used for normal execution.

Dynamic construction is best understood as a sequence of circuit epochs rather than unrestricted mutation of one graph while it executes. An epoch is a version of the active circuit for which identities and dependencies are stable enough to schedule and reason about. Execution of one epoch produces values and a trace. The constructor then uses the goal, trace, current KB state, remaining budget, and policies to produce a new epoch if needed.

The abstract transition is

$$C(k+1) = R(C(k), \text{trace}(k), G, K, B(k), \text{policy})$$

where (C_k) is the current SOP circuit, $(_k)$ is the execution trace, (G) is the unresolved goal, (K) is accessible knowledge, (B_k) is the remaining resource budget, and $()$ is the policy set. The function (R) is the central subject of the research programme. It must usually behave deterministically for a fixed runtime configuration even when it chooses among heuristic candidates, so that debugging and benchmarking are reproducible.

An epoch transition may instantiate a template, retrieve an additional KB shard, introduce a validator, expand an alternative command, or replace a high-level command with a lower-level subcircuit. Values from the previous epoch need not be discarded. If their producers, inputs, command versions, and relevant policies remain unchanged, they can be reused through cache validation.

This interpretation reconciles dynamic reasoning with SSA-like discipline. Within an epoch, a named result is produced by a specific declaration. Revision creates a new value or a new epoch rather than silently overwriting the historical derivation. Consequently the system can retain an auditable history of how an answer changed as new knowledge or strategies became available.

A.3 Trust roles, strategy contracts, and artefact promotion

The unified architecture adds a small trust vocabulary around ordinary SOP execution. These roles are not intended to create a second programming language. They are contracts attached to commands, subcircuits, registries, and receipts. An implementation may encode them as metadata,

privileged commands, or host-level policy. What matters is that a runtime can distinguish proposal from evidence and evidence from authority.

Role	Reference meaning
Producer	Command or circuit instance that creates an @ value. Its identity is part of the value lineage.
Strategy	Reviewed implementation that can perform one typed responsibility. Several strategies may be alternatives for the same responsibility.
Coordinator	Bounded meta-level component that allocates work among strategies and records candidates, refusals, exhaustion, and results.
Verifier	Component that evaluates whether a witness or result satisfies a declared semantic condition.
Authority gate	Verifier whose acceptance is required to assign a stronger epistemic or operational status; confidence scores cannot override it.
Candidate artefact	Generated command, circuit, rule, mapping, or knowledge package that has not been promoted.
Promoted artefact	Reviewed, versioned artefact admitted to a trusted registry or KB generation for a declared scope.
Receipt	Immutable execution record that identifies inputs, versions, work, branches, checks, outputs, and completion status sufficiently for replay and audit.

A receipt is not the same object as a natural-language explanation. The explanation may summarize why a result is credible to a human. The receipt records operational facts that another program can inspect. A receipt should therefore be stable under changes in explanation wording. Conversely, a good explanation may cite only a small subset of the receipt. This separation helps prevent post-hoc rationalization from being confused with execution provenance.

Authority should also be scoped. A circuit can be authoritative for syntax validation without being authoritative for scientific truth. A theorem prover can be authoritative for derivability in a formal theory but not for whether the theory models the real-world phenomenon accurately. A statistical circuit can be authoritative for recomputing a declared test but not for whether the experimental design was appropriate. SOP result types and receipts should preserve these scopes.

Appendix B - Prerequisite Encyclopedia

B.1 Logic programming, deductive databases, and formal verification

Logic programming treats part of computation as the search for substitutions or structures that make logical relationships hold. Prolog is the canonical example. A Prolog program consists of facts and rules, and a query asks whether a goal can be derived. Unification attempts to make symbolic expressions equal by finding substitutions for variables. Backtracking explores another choice when a selected rule or binding does not lead to a solution. These ideas matter for SOP Lang because a command or a template matcher may use unification internally, while SOP itself remains the orchestration IR that records what inference was requested and what value it produced.

Tabling changes the operational behaviour of logic programming by remembering calls and their answers. Rather than repeatedly exploring the same recursive subproblem, a system such as SWI-Prolog can reuse previously discovered results. This is closely related to memoization in SOP circuits, although the objects being cached differ. A SOP runtime can cache a command instance keyed by its input values, command version, KB version, and policy context. A Prolog backend can additionally table subgoals inside that command.

Datalog is a restricted logic-programming language widely used for deductive databases and program analysis. Its restrictions make bottom-up fixed-point evaluation practical. Starting from extensional facts, rules repeatedly derive intensional facts until no new fact appears. Semi-naïve evaluation improves this process by focusing each iteration on facts newly derived in the previous iteration instead of recomputing all joins. This is directly relevant to dynamic SOP construction: template activation can similarly be driven by deltas rather than rescanning the complete state after every new variable.

SAT asks whether a Boolean formula has any satisfying assignment. SMT generalizes satisfiability by combining Boolean structure with background theories such as integer arithmetic, reals, arrays, bit-vectors, or strings. Z3 is a mature SMT solver. In a SOP circuit, an SMT command is a specialized backend. SOP is responsible for constructing the constraints, preserving their relation to source values, invoking the solver, and interpreting the returned model or unsatisfiability result. The solver's formal correctness does not guarantee that the natural-language statement was translated into the right constraints.

A proof assistant such as Lean takes another approach. Statements are represented in a formal language, and proofs are objects checked by a comparatively small trusted kernel. This provides strong guarantees about derivability in the formal system. Again, the SOP-level problem is broader: deciding when formal proof is warranted, translating or constructing the theorem, feeding it to the backend, and preserving the distinction between a formally verified consequence and an informal premise extracted from text.

Open-world and closed-world assumptions are especially important when natural language is involved. Under a closed-world assumption, failure to find a fact can be treated as evidence that the fact is false. Under an open-world assumption, absence merely means unknown. Ordinary documents are usually incomplete, so treating missing statements as false is dangerous. SOP variables should therefore be capable of representing unknown and unresolved states explicitly rather than collapsing all command failures to Boolean false.

Concept	Practical meaning for SOP Lang research
Unification	Finds bindings that make symbolic structures compatible; useful inside rule, query, and template-matching commands.
Backtracking	Explores alternative choices after failure; can be contained inside a backend or represented explicitly as SOP alternatives.

Concept	Practical meaning for SOP Lang research
Tabling	Memoizes logic-programming subgoals and answers; analogous to, but more specialized than, SOP command-result caching.
Datalog fixed point	Repeatedly derives facts until no new facts appear; suggests delta-driven activation for KB and template circuits.
Semi-naive evaluation	Processes newly derived information rather than recomputing every rule against the complete database.
SAT	Decides whether a Boolean constraint system has a satisfying assignment.
SMT	Extends satisfiability with theories such as arithmetic or bit-vectors; naturally exposed as a SOP command backend.
Proof assistant	Checks explicit formal proof objects in a trusted logical kernel.
Open world	Missing information is unknown rather than false; often the safer default for text-derived knowledge.
Soundness	Accepted formal derivations correspond to valid consequences under the system's semantics and assumptions.

B.2 Graphs, dataflow, SSA, IRs, rewriting, and e-graphs

A directed graph contains nodes connected by directed edges. A directed acyclic graph, or DAG, contains no directed cycle. If an edge means that one computation needs a value produced by another, the graph becomes a dataflow graph. A topological ordering of a DAG schedules every producer before its consumers. SOP Lang's `$var` references induce exactly this kind of dependency for the acyclic portion of a circuit.

A compiler normally does not translate source text directly into final machine code. It introduces intermediate representations, or IRs, whose structure makes analyses and transformations easier. LLVM IR is a well-known typed SSA-based IR. MLIR generalizes the idea through extensible dialects and multiple abstraction levels. The analogy to SOP Lang is not that a natural-language problem is already a deterministic source program. Rather, SOP Lang may play the role of a common executable representation after interpretation has produced enough structure to begin reasoning. High-level SOP commands can subsequently be lowered into specialized circuits or external backend calls.

Static Single Assignment means that each SSA value has a single defining operation. When a source-language variable changes, the IR creates another version instead of silently overwriting the previous one. This property makes def-use relationships explicit. SOP Lang naturally approximates this discipline when each `@var` is treated as the result of one declaration and revisions create new variables or epochs. The research programme should make this invariant explicit rather than relying on naming conventions alone.

A compiler lowering pass replaces a high-level operation with a more concrete implementation while preserving the intended semantics. In SOP Lang, `@answer temporalReasoning ...` might initially be a high-level command. A dynamic constructor may lower it to event extraction, temporal ordering, state transitions, and a query. Some of those commands may then lower further into Datalog or SMT. This makes the compiler analogy operational rather than decorative.

Rewriting systems transform terms according to rules. Maude uses rewriting logic as a foundation for executable specifications of changing and concurrent systems, and its strategy language controls which rewrite rules are applied. SOP dynamic construction can be regarded as rewriting the active SOP program, but the SOP representation retains explicit produced variables and dependencies. This makes Maude particularly relevant as a theoretical and potentially practical backend for controlled circuit transformations.

E-graphs represent many equivalent expressions compactly by storing equivalence classes rather than repeatedly materializing every syntactic variant. Equality saturation applies rewrite rules broadly and delays selecting one expression until extraction according to a cost function. This can be valuable for SOP circuit optimization when equivalence is genuinely known. It is not safe to assume that two natural-language interpretations are equivalent merely because they are similar. E-graphs should therefore be used after semantics have been formalized sufficiently to justify the equivalence relation.

Concept	Practical meaning for SOP Lang research
Dataflow graph	Graph in which edges represent value dependencies; \$var gives SOP Lang a direct dataflow interpretation.
Topological scheduling	Executes producers before consumers and exposes independent commands that can run concurrently.
Intermediate representation	Common internal language that decouples front-end interpretation from backend execution and optimization.
SSA	Associates each value with one definition, simplifying provenance, caching, dependency analysis, and revision history.
Def-use chain	Link from a value-producing declaration to every command that consumes the value.
Lowering	Replaces a high-level SOP command with a more concrete subcircuit or backend-specific operation.
Rewrite rule	Describes a semantics-preserving or policy-driven transformation of a program or symbolic state.
E-graph	Compact representation of formally equivalent expressions or circuit fragments.
Equality saturation	Explores equivalence-preserving rewrites before selecting a low-cost representative.
Dominance pruning	Removes candidates that are no better on relevant quality dimensions and strictly worse on at least one.

For readers coming from ordinary application programming, the most important conceptual distinction is that “graph” names a data structure while dataflow, control flow, knowledge representation, and circuits name different semantics placed on graph-like structures. A directed edge in a knowledge graph can mean “is part of”; a directed edge in a def-use graph means “this value is consumed here”; a directed edge in a control-flow graph means “execution may continue here.” Algorithms that are correct for one interpretation are not automatically meaningful for another.

A hypergraph generalizes an ordinary graph by allowing one edge to connect several vertices at once. This is relevant because a command with several required inputs naturally forms a hyperedge from those inputs to its result. The AND/OR interpretation of circuit construction is therefore more accurately an AND/OR hypergraph than a simple graph. SOP source does not need explicit hypergraph syntax: the ordered list of \$ references in one declaration already identifies a multi-input dependency.

A def-use chain is the compiler relation from one definition to all uses of the resulting value. In SOP Lang the producer of @x and every later \$x form this chain directly. This is one reason the single-producer convention matters. If commands could mutate x invisibly, the runtime would need global alias and state analysis before it could know which value a consumer observes. The current SOP direction deliberately trades some imperative convenience for a simpler dependency model.

Topological scheduling applies to an acyclic dependency graph: if every command receives its required produced values from earlier nodes in the dependency order, any topological ordering is a legal execution order for pure operations. Physical source order is then only one possible schedule.

Commands with external effects or ~ dependencies can impose additional constraints. Templates and epochs deal with graph growth beyond the static DAG, while recursive backends can keep their own loops internally.

Graph-theoretic term	Concrete SOP example
Vertex/node	A concrete command instance such as <code>@events extract_events \$text</code> .
Directed edge	A <code>\$text</code> dependency from the producer of <code>text</code> to <code>extract_events</code> .
Hyperedge	A declaration such as <code>@state update \$previous \$event \$rules</code> depends jointly on several inputs.
Def-use graph	The graph recovered by linking each <code>@</code> definition to all declarations that reference it with <code>\$</code> .
Meta-dependency	A <code>~</code> reference that exposes a variable object and its lineage/branch metadata to a meta-command.
Knowledge edge	A persistent relation such as <code>Paris capital_of France</code> , usually queried by a SOP command rather than interpreted as an execution edge.
Control edge	A possible execution transition. SOP often leaves this inside command semantics or runtime policy instead of encoding a universal CFG.
Rewrite edge	A relation between circuit versions when a planner or template transforms one epoch into another.

These distinctions prevent a common source of confusion in the circuit literature. Drawing boxes and arrows does not create a new computational model. The research content lies in the invariants associated with the boxes and arrows. For SOP Lang those invariants include explicit producers, explicit value and meta references, command contracts, versioned circuit growth, and an execution runtime that can derive scheduling and provenance from the same representation.

B.3 Knowledge representation, provenance, uncertainty, and neuro-symbolic systems

Knowledge representation concerns the forms in which facts, concepts, relations, events, rules, and uncertainty are made explicit enough for computation. A knowledge graph represents entities and relations as a graph. An ontology goes further by specifying conceptual categories, relation meanings, constraints, and often subsumption hierarchies. SOP Lang need not replace either. It can represent the executable relationship between knowledge and reasoning, while KB commands provide indexed access to persistent facts and ontological structures.

Grounding connects a symbol to the evidence or referent that gives it meaning. In a document-oriented system, grounding may link a claim to source spans, document identifiers, extraction procedures, and time. In a sensor system it may link concepts to observations. A symbolic language model becomes fragile if it manipulates internally consistent symbols that were grounded incorrectly. For this reason SOP values should carry provenance close to their creation point rather than attempting to reconstruct explanation after reasoning has finished.

Provenance answers where a value came from and how it was transformed. It is distinct from credibility. A perfectly traced statement may originate in an unreliable source. It is also distinct from logical proof: a proof establishes derivability from premises, not the empirical truth of those premises. SOP Lang can make these distinctions operational by preserving source provenance in variables while separate commands evaluate source credibility, formal validity, or consistency.

Uncertainty has multiple meanings. Probability quantifies uncertainty in a probabilistic model. Fuzzy truth represents graded satisfaction or membership. Neural confidence is typically a score produced by a learned model and may or may not be calibrated as a probability. These quantities should not be silently merged. A SOP runtime can expose them through different value types or metadata contracts so that downstream commands know what they are consuming.

ProbLog represents probabilistic facts inside logic programs. DeepProbLog connects neural predicates to probabilistic logic, demonstrating that learned perception can feed symbolic inference. Scallop provides a Datalog-inspired language with provenance and differentiable reasoning. Logic Tensor Networks interpret logical constraints through differentiable tensor operations. These projects show that symbolic and neural computation can be connected through explicit interfaces, but each chooses a particular integrated semantics. SOP Lang takes a different architectural position: a circuit can call any of these mechanisms while preserving their distinct guarantees and result types.

Controlled natural languages such as Attempto Controlled English reduce the ambiguity of unrestricted natural language and support translation into formal representations. They can play an important role between free text and SOP circuits. A neural front end may propose a controlled-language interpretation; the system can validate or expose that interpretation before expensive reasoning occurs. The CNL is not necessarily SOP Lang itself. SOP represents execution; a CNL represents a constrained form of meaning that can be compiled into executable declarations.

Concept	Practical meaning for SOP Lang research
Ontology	Explicit specification of domain concepts and relations used to interpret or type SOP values.
Knowledge graph	Persistent relational structure that can be queried by SOP commands rather than necessarily loaded into the active circuit.
Grounding	Link from symbolic values to source text, observations, datasets, or other evidence.
Provenance	Record of the origin and derivation path of a value through commands and sources.
Probability	Quantified uncertainty under a probabilistic semantics; not interchangeable with neural confidence.
Fuzzy truth	Degree of membership or satisfaction in a many-valued logic.
Calibration	Agreement between a reported confidence or probability and observed correctness frequencies.
Neuro-symbolic AI	Architectures combining learned numerical components with explicit symbolic structures or reasoning.
Controlled natural language	Restricted natural language designed to reduce ambiguity and permit reliable formalization.

Concept	Practical meaning for SOP Lang research
Epistemic status	Classification such as observed, assumed, inferred, probabilistic, formally proved, contradicted, or unresolved.

B.4 Search complexity, planning, caching, and cryptographic circuits

The dynamic construction of a SOP circuit is fundamentally a search problem. If every available command can feed every compatible command, the number of possible compositions can grow exponentially with circuit depth. Adding template instantiation, KB retrieval, alternative commands, and multiple interpretations increases the branching factor further. A useful symbolic language model cannot explore this space exhaustively for ordinary tasks.

A heuristic estimates which candidate is promising without completely solving it. Best-first search expands candidates by priority. Beam search keeps only a bounded number of candidates at a given stage. An anytime algorithm can return the best result currently available and improve it if additional time remains. These established ideas are relevant, but the SOP research problem adds a strict requirement for reproducibility. A budget-aware deterministic agenda can rank candidates by a stable score using semantic relevance, type compatibility, historical success, expected cost, provenance requirements, and remaining budget. Equal scores should be resolved by stable tie-breaking rather than uncontrolled concurrency.

Memoization stores the result of a computation so that equivalent later requests can reuse it. Correct cache keys are difficult in a semantic system. They must include not only the argument values but also command version, KB snapshot, ontology version, policy, and relevant external state. A cached retrieval result may expire with time, while a formally verified pure computation may remain valid indefinitely.

Cryptographic circuit systems provide another important analogy. Secure multiparty computation evaluates functions while protecting private inputs. ABY demonstrates mixed-protocol computation in which different parts of one function can use different secure representations. SOP circuits similarly benefit from backend heterogeneity: a command can be chosen according to semantics, cost, trust, or privacy requirements. The analogy is architectural; a general SOP circuit is not itself a secure MPC circuit.

Zero-knowledge proof systems often arithmetize a computation into constraints. Circom and Halo2 exemplify this circuit-oriented discipline. A witness contains secret and intermediate values satisfying the constraints. The verifier checks the proof without necessarily observing the complete computation. A SOP execution trace can be thought of as an epistemic witness in a loose architectural sense: it records the intermediate values that justify a conclusion. This analogy must not be confused with cryptographic zero knowledge. Only suitably deterministic and arithmetized subcircuits could eventually be compiled into actual proof circuits.

Cryptographic engineering also illustrates the danger of under-constrained designs. A circuit can be internally valid yet fail to constrain the property the designer intended. The semantic analogue is a validator that merely checks that a citation exists without checking that the cited material entails the claim. Explicit structure does not automatically produce meaningful guarantees; the invariants themselves must capture the intended property.

Concept	Practical meaning for SOP Lang research
Combinatorial explosion	Rapid growth of possible circuit compositions, template instances, knowledge activations, and alternative branches.
Heuristic	Stable estimate used to prioritize candidates before complete evaluation.
Beam bound	Hard limit on the number of competing candidates retained at a stage.

Concept	Practical meaning for SOP Lang research
Memoization	Reuse of a command result when all semantically relevant inputs and versions match.
Anytime algorithm	Procedure that can return a valid current result within a budget and improve it when more budget is available.
Circuit witness	In cryptography, values satisfying circuit constraints; metaphorically useful for the explicit SOP trace supporting a result.
MPC	Secure computation over private inputs held by multiple parties.
Zero knowledge	Cryptographic proof property that hides the witness beyond what the verified statement reveals.
Arithmetization	Translation of computation into algebraic constraints suitable for proof systems.
Under-constrained design	Specification that technically validates executions not intended by the designer; a warning equally relevant to semantic validation.

B.5 Research objects, reproducibility, provenance, and trustworthy AI

A research object is a package or conceptual unit that groups the artefacts required to understand or reproduce a piece of research. It may contain data, software, workflows, notebooks, documentation, identifiers, and metadata. RO-Crate is a current community specification that packages such material using JSON-LD and linked-data principles. The value for SOP Lang is interoperability: an ingest command can receive a crate as a structured source rather than reverse-engineering an ad hoc project directory. [R33]

Reproducibility terminology varies between communities, but the general distinction is between reusing the original artefacts to obtain a corresponding result and independently recreating the result with different artefacts or implementation. ACM artifact review and badging is an example of a mature practice that evaluates availability and functionality of research artefacts and recognizes reproducibility-related properties. A SOP peer-review circuit should preserve the definitions used by the target venue rather than inventing its own universal badge semantics. [R35]

Provenance is information about entities, activities, and agents involved in producing a thing. W3C PROV defines an interoperable model for this information. A SOP receipt can be more execution-specific, but many of its relationships map naturally to PROV: an @ value is an entity, a command execution is an activity, a coding agent or human reviewer can be an agent, and derivation connects outputs to inputs. This mapping is valuable because provenance can then cross tool boundaries. [R32]

Concept	Why it matters to SOP Lang
Research object	Provides the larger package within which SOP files, data, code, source documents, and execution receipts can be distributed together.
Computational notebook	Provides executable code plus narrative and results; SOP can point to or invoke notebook computations while making claim dependencies explicit.

Concept	Why it matters to SOP Lang
Provenance	Makes origin and derivation exchangeable across systems; SOP adds command-level def-use and branch details that can be exported to a provenance model.
Artifact evaluation	Provides an existing institutional model for checking whether research artefacts are available, functional, reusable, or reproducible.
Trustworthy AI	Treats trust as a family of measurable socio-technical properties rather than one scalar confidence score. SOP contributes technical mechanisms to only a subset of those properties.
Authority boundary	Defines who or what can change the status of a claim, artefact, or action. It is a governance concept expressed operationally through gates and promotion receipts.

Trustworthy AI is broader than symbolic explainability. NIST emphasizes validity and reliability, safety, security and resilience, accountability and transparency, explainability and interpretability, privacy enhancement, and fairness. A symbolic circuit can make dependencies visible yet still be unsafe, biased, insecure, or built on invalid assumptions. The purpose of explicit SOP circuits is to create better handles for measurement and governance, not to collapse trust into a property of syntax. [R30]

Appendix C - Acronyms and Abbreviations

Acronym	Meaning
ACE	Attempto Controlled English
AGI	Artificial General Intelligence
AI	Artificial Intelligence
API	Application Programming Interface
CFG	Control-Flow Graph
CNL	Controlled Natural Language
DAG	Directed Acyclic Graph
DRS	Discourse Representation Structure
DSL	Domain-Specific Language
EqSat	Equality Saturation
FOL	First-Order Logic
IR	Intermediate Representation
KB	Knowledge Base
LLM	Large Language Model
LTN	Logic Tensor Network
MLIR	Multi-Level Intermediate Representation
MPC	Secure Multiparty Computation
NARS	Non-Axiomatic Reasoning System
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
PLP	Probabilistic Logic Programming
RAG	Retrieval-Augmented Generation
SAT	Boolean Satisfiability Problem
SMT	Satisfiability Modulo Theories
SOP Lang	The command-and-variable language used in this research programme to represent executable dynamic circuits
SSA	Static Single Assignment
ZK	Zero Knowledge
AI RMF	Artificial Intelligence Risk Management Framework.
PROV	W3C family of specifications for interoperable provenance.
RO-Crate	Research Object Crate, a machine-readable package for research data and related artefacts.
AOT	Ahead-of-time compilation.
JIT	Just-in-time compilation.
ABI	Application Binary Interface; used here by analogy for stable command/runtime contracts.
TEEs	Trusted Execution Environments.

References

- [R1] LLVM Project. LLVM Language Reference Manual. Official LLVM documentation, current edition.
- [R2] LLVM Project. MLIR Overview and MLIR Language Reference. Official MLIR documentation.
- [R3] Soufflé Project. Soufflé Documentation and Tutorial. Official project documentation.
- [R4] SWI-Prolog. Tabling and SLG Resolution. Official SWI-Prolog documentation.
- [R5] Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., and Talcott, C. All About Maude. Springer; together with current Maude documentation and strategy-language literature.
- [R6] Goertzel, B. et al. Reflective Metagraph Rewriting as a Foundation for an AGI Language. 2021; and OpenCog Hyperon/MeTTa official documentation.
- [R7] De Raedt, L., Kimmig, A., and Toivonen, H. ProbLog literature; and official ProbLog documentation.
- [R8] Aditya, D. et al. PyReason: Software for Open World Temporal Logic. 2023.
- [R9] Wang, P. NARS: A Non-Axiomatic Reasoning System and official introductory material on reasoning under insufficient knowledge and resources.
- [R10] Fuchs, N. E., Kaljurand, K., and Kuhn, T. Literature on Attempto Controlled English; and official Attempto project material.
- [R11] de Moura, L., and Bjørner, N. Z3: An Efficient SMT Solver; and the Microsoft Z3 Guide.
- [R12] Lean Project. Theorem Proving in Lean 4. Official documentation on dependent type theory and proof construction.
- [R13] Manhaeve, R., Dumancic, S., Kimmig, A., Demeester, T., and De Raedt, L. DeepProbLog: Neural Probabilistic Logic Programming. 2018.
- [R14] Li, Z., Huang, J., and Naik, M. Scallop: A Language for Neurosymbolic Programming. 2023.
- [R15] Badreddine, S. et al. Logic Tensor Networks. Foundational literature on Real Logic and differentiable first-order reasoning.
- [R16] Willsey, M. et al. egg: Fast and Extensible Equality Saturation. POPL 2021.
- [R17] Zhang, Y. et al. Better Together: Unifying Datalog and Equality Saturation. Work introducing egglog.
- [R18] ONNX Project. ONNX Intermediate Representation Specification. Official documentation.
- [R19] Demmler, D., Schneider, T., and Zohner, M. ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation. NDSS 2015.
- [R20] Circom Project. Circom Language and Constraint Generation Documentation. Official documentation.
- [R21] Halo2 Project. The Halo2 Book: PLONKish Arithmetization. Official documentation.
- [R22] Yin, Y. et al. EggMind: LLM-Guided Strategy Synthesis for Equality Saturation. 2026 preprint.

[R23] Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., and Zadeck, F. K. Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. ACM TOPLAS, 1991.

[R24] Forgy, C. L. Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem. Artificial Intelligence, 1982.

[R25] Cohen, W. W. TensorLog: A Differentiable Deductive Database. 2016.

[R26] Rocktaschel, T., and Riedel, S. End-to-End Differentiable Proving. 2017.

[R27] AssistOS-AI. SOPLangCircuits. Public GitHub repository and maintained documentation, state consulted 18 August 2026. <https://github.com/AssistOS-AI/SOPLangCircuits>

[R28] AssistOS-AI. dynamicCircuits. Public GitHub repository and maintained documentation, state consulted 18 August 2026. <https://github.com/AssistOS-AI/dynamicCircuits>

[R29] AssistOS-AI. eslm: Executable Symbolic Language Model. Public GitHub repository and maintained documentation, state consulted 18 August 2026. <https://github.com/AssistOS-AI/eslm>

[R30] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework and AI RMF Playbook. 2023-current. <https://www.nist.gov/itl/ai-risk-management-framework>

[R31] European Commission. AI Act implementation and transparency guidance. State consulted 18 August 2026. <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>

[R32] World Wide Web Consortium. PROV family of provenance recommendations. <https://www.w3.org/TR/prov-overview/>

[R33] RO-Crate Community. RO-Crate Metadata Specification 1.3 and official use-case material. 2026. <https://www.researchobject.org/ro-crate/>

[R34] Project Jupyter. Jupyter documentation and Notebook format/use-case documentation. <https://jupyter.org/>

[R35] Association for Computing Machinery. Artifact Review and Badging policy and reproducibility guidance.

[R36] Yamada, Y. et al. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066, 2025.

[R37] FutureHouse. Research mission and platform material on scientific research agents. <https://www.futurehouse.org/>

[R38] Mitchener, L. et al. Kosmos: An AI Scientist for Autonomous Discovery. arXiv:2511.02824, 2025.

[R39] Nusrat, H. and Nusrat, O. When AI Does Science: Evaluating the Autonomous AI Scientist KOSMOS in Radiation Biology. arXiv:2511.13825, 2025.

[R40] Kuhn, T. A Survey and Classification of Controlled Natural Languages. Computational Linguistics 40(1), 2014.

[R41] Schwitter, R., and Fuchs, N. E. Attempto: From Specifications in Controlled Natural Language towards Executable Specifications. arXiv:cmp-lg/9603004.

[R42] Sowa, J. F. Common Logic Controlled English and related Common Logic material.

[R43] Object Management Group. Semantics of Business Vocabulary and Business Rules, Version 1.5. <https://www.omg.org/spec/SBVR/1.5/About-SBVR>

[R44] Ranta, A. Grammatical Framework and official GF documentation. <https://www.grammaticalframework.org/>

[R45] Caruso, S., Dodaro, C., Maratea, M., Mochi, M., and Riccio, F. CNL2ASP: Converting Controlled Natural Language Sentences into ASP. TPLP 24(2), 2024.

[R46] Borroto, M., Kareem, I., and Ricca, F. Towards Automatic Composition of ASP Programs from Natural Language Specifications. IJCAI 2024.

[R47] Nelson, G. Inform 7: a natural-language-based programming language for interactive fiction. Official project site. <https://ganelson.github.io/inform-website/>

[R48] Cucumber Project. Gherkin Reference and Cucumber documentation on executable specifications. <https://cucumber.io/docs/gherkin/>

[R49] Whole Tale Project. Open-source platform and executable research-object model. <https://wholetale.org/>

[R50] Common Workflow Language Project. Official standard and documentation. <https://www.commonwl.org/>

[R51] REANA Project. Reusable Analyses platform. <https://www.reana.io/>

[R52] Jaradeh, M. Y. et al. Open Research Knowledge Graph: Next Generation Infrastructure for Semantic Scholarly Knowledge. K-CAP 2019; and ORKG system documentation.

[R53] Kuhn, T. et al. Nanopublication literature and current nanopublication documentation.

[R54] Lu, C. et al. Towards end-to-end automation of AI research. Nature, 2026. <https://www.nature.com/articles/s41586-026-10265-5>

[R55] Gottweis, J. et al. Accelerating scientific discovery with Co-Scientist. Nature, 2026. <https://www.nature.com/articles/s41586-026-10644-y>

[R56] Ghareeb, A. E. et al. A multi-agent system for automating scientific discovery. Nature, 2026; together with Bisht, H. et al. Agentic AI Scientists Are Not Built For Autonomous Scientific Discovery, 2026 position paper.

[R57] AAAI-26 AI Review Pilot. AI-Assisted Peer Review at Scale: The AAAI-26 AI Review Pilot Program. arXiv:2604.13940, 2026.

[R58] NeurIPS 2026. AI-Assisted Reviewing Experiment. Official conference documentation. <https://neurips.cc/Conferences/2026/ai-reviewing-experiment>

[R59] Nguyen, T. H., and Ahmadi, Z. LLM-Based Scientific Peer Review: Methods, Benchmarks, and Reliability Challenges. arXiv:2606.25057, 2026; and Can AI Be a Good Peer Reviewer? A Survey of Emerging Trends and Challenges. arXiv:2604.27924, 2026.

[R60] Edelman, B., and Skolnick, J. Valsci: An Open-Source, Self-Hostable Literature Review Utility for Automated Large-Batch Scientific Claim Verification Using Large Language Models. BMC Bioinformatics 26, 140, 2025. DOI: 10.1186/s12859-025-06159-4.

[R61] Liu, H., et al. Retrieval Augmented Scientific Claim Verification. JAMIA Open 7(1), ooae021, 2024; and Sadeghi, S., et al. DeepSciVerify: Verifying Scientific Claim-Citation Alignment via LLM-Driven Evidence Escalation. arXiv:2605.27710, 2026.

[R62] ISO/IEC 42001:2023. Information technology - Artificial intelligence - Management system. <https://www.iso.org/standard/42001>

[R63] ISO/IEC 42006:2025. Requirements for bodies providing audit and certification of artificial intelligence management systems. <https://www.iso.org/standard/42006>

[R64] OECD. Recommendation of the Council on Artificial Intelligence and OECD AI Principles, updated framework consulted August 2026. <https://oecd.ai/en/ai-principles>

[R65] Cerna, D. M. et al. Integration of Controlled Natural Language in Formal Mathematics. EuroProofNet white paper, 2025.